

Data Structures

โครงสร้างข้อมูล

254251

1st semester 2017

Info

- Instructor
 - Thanathorn Phoka (ธนธร พ่อคำ)
 - Thanathornp[!@#\$%^&]nu.ac.th
 - Room no. 417
- TA
- Programming language
 - Python
- Online resources
 - https://github.com/fomapavlov/254251_1_2017
 - <http://interactivepython.org/runestone/static/pythonds/index.html>
 - <http://pythontutor.com/visualize.html>
 - Youtube 2110101 Computer Programming (Python) บทที่ 1 - 8

Course description

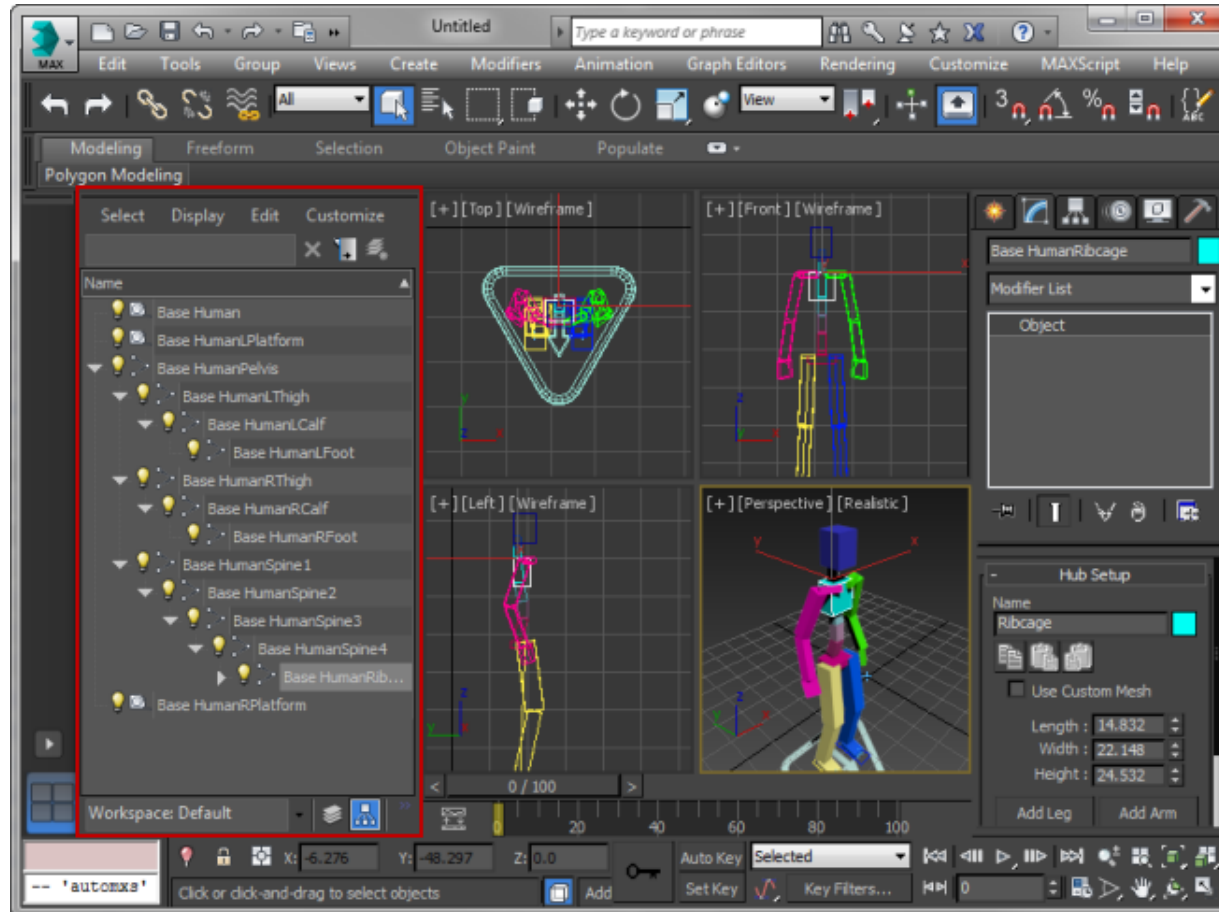
- โครงสร้างข้อมูลพื้นฐาน แอเรย์ สแตคและแถวคอย รายการโยง การเวียนบังเกิด ต้นไม้ค้นหาแบบทวิภาค ต้นไม้เอวีแอล ฮีป ตารางแฮช กราฟ การเรียงลำดับและการค้นหา
- Basic data structure, array, stacks, queues, linked lists, recursion, binary search trees, AVL trees, heaps, graphs, and hash tables, application to sorting and searching

Why data structures?



<http://mathematica.stackexchange.com/questions/11673/how-to-play-with-facebook-data-inside-mathematica>

Why data structures?



<https://knowledge.autodesk.com/support/3ds-max/learn-explore/caas/CloudHelp/cloudhelp/2017/ENU/3DSMax/files/GUID-2569461E-C859-4D54-BAFF-C8BD078B53AC-htm.html>

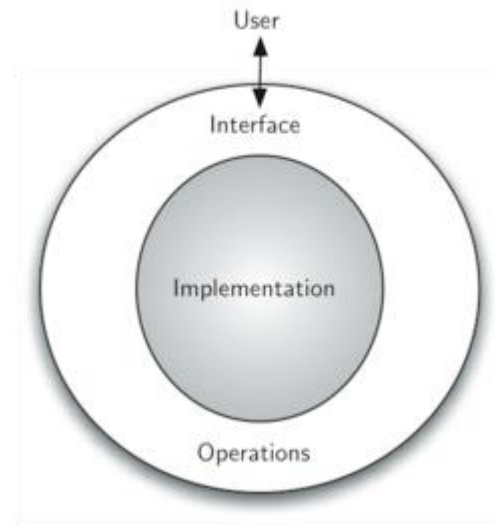
Data structures and abstract data types

- Abstract Data Types (ADT)

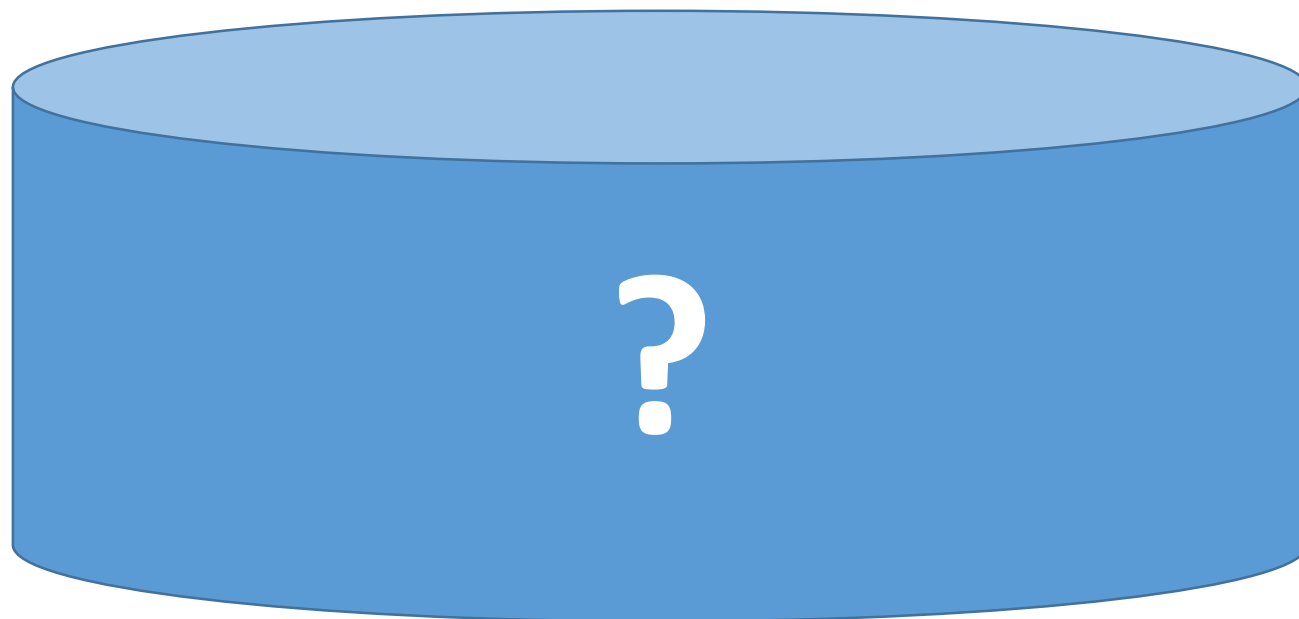
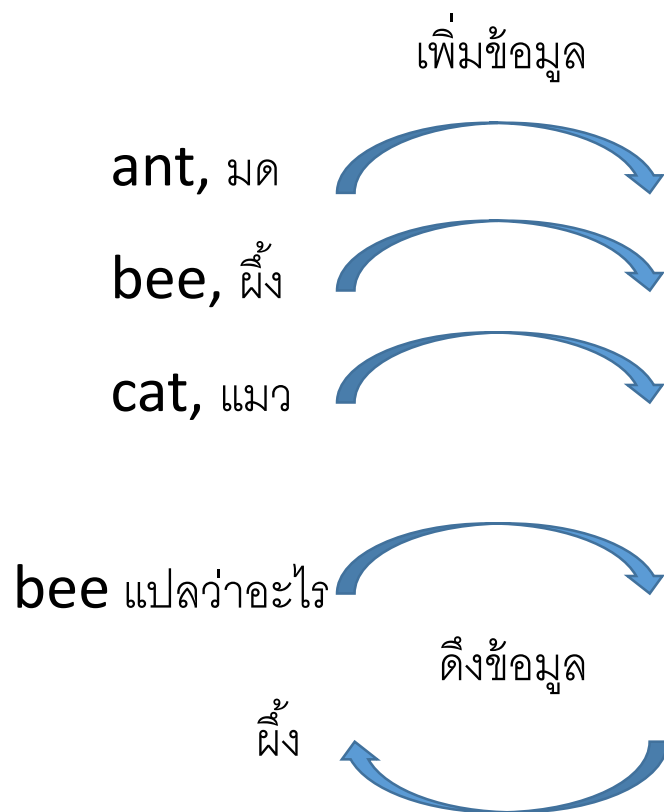
- เป็นมุมมองของผู้ใช้งาน Data structure ที่ไม่จำเป็นต้องรู้ว่า implementation เป็นอย่างไร

- Data structures

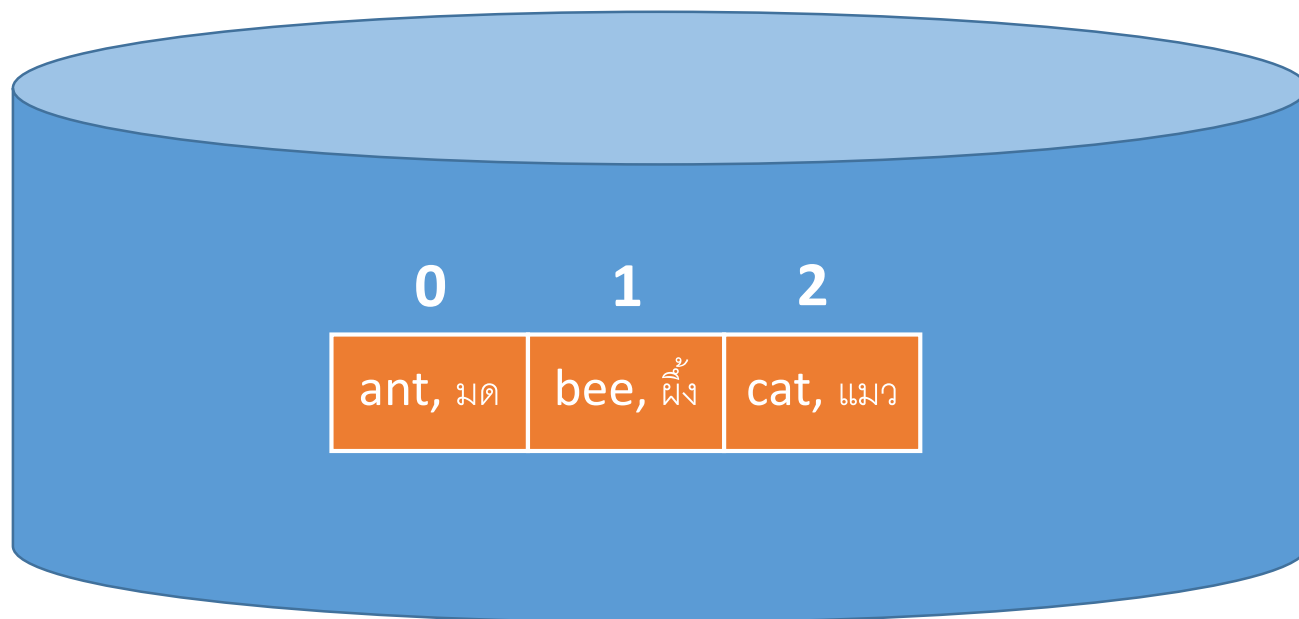
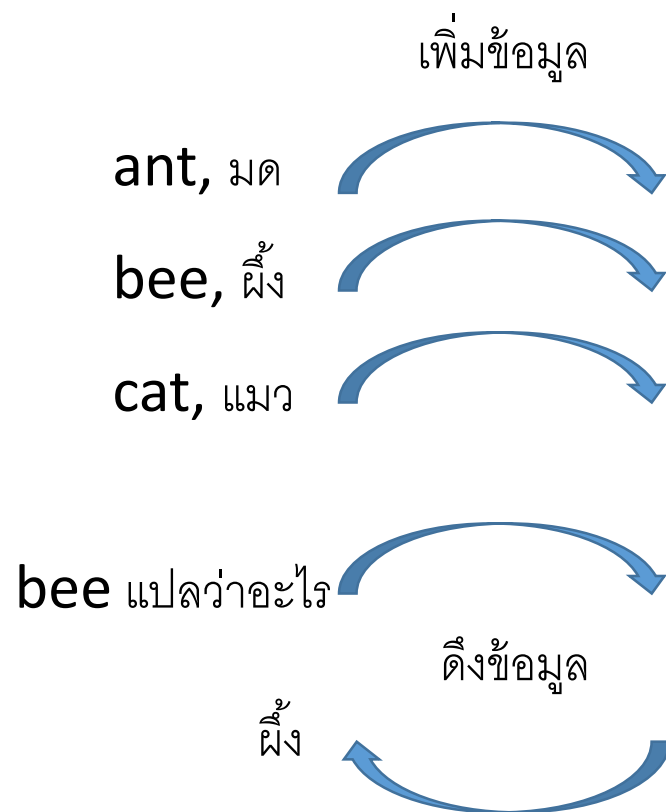
- เป็นส่วน implementation



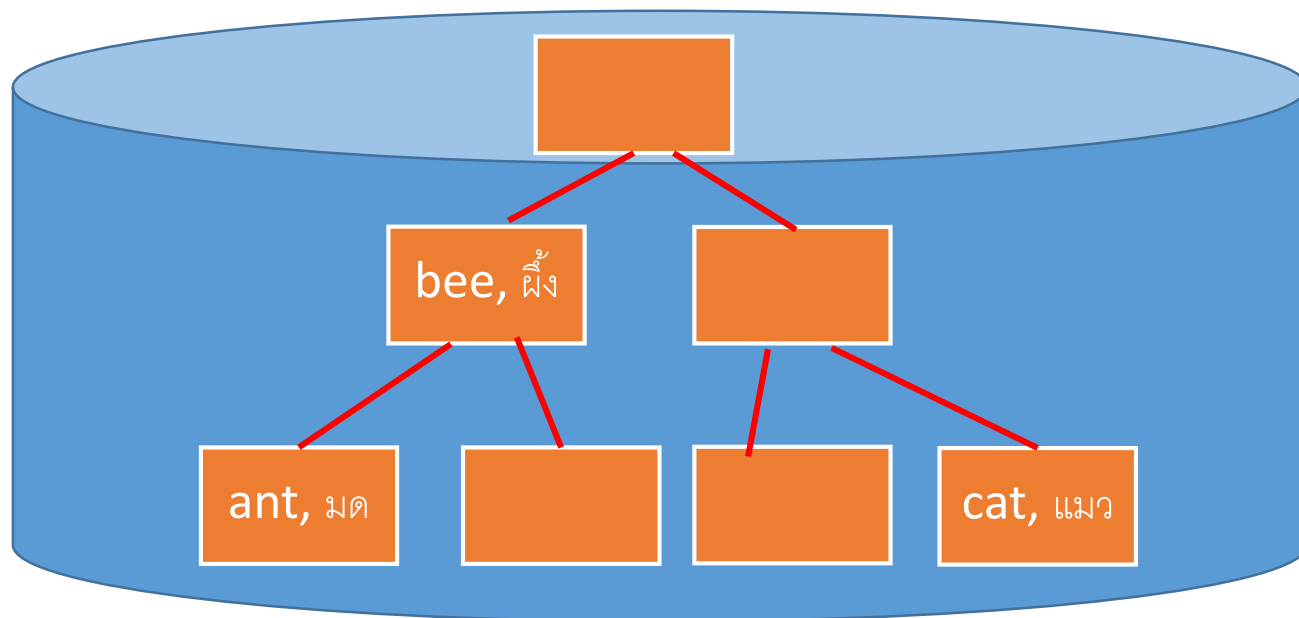
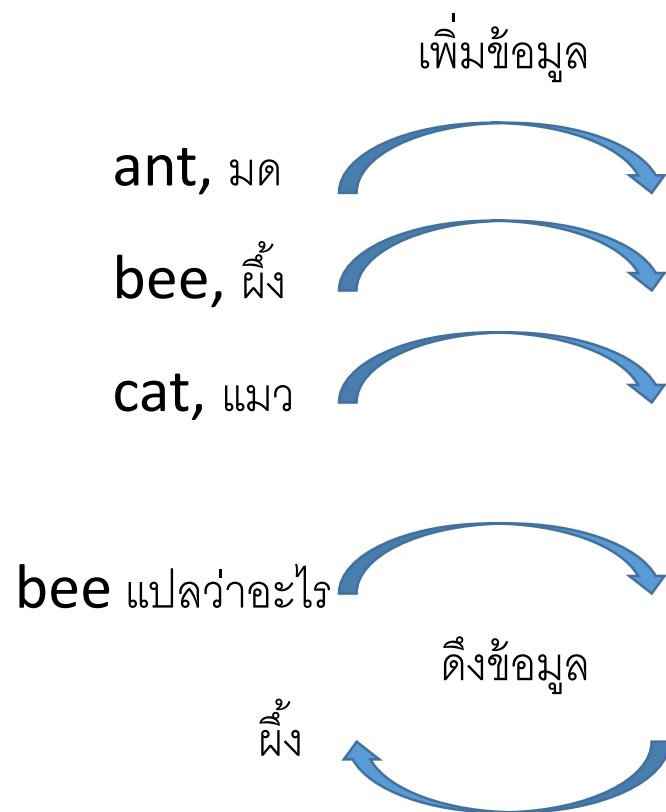
Dictionary



Dictionary



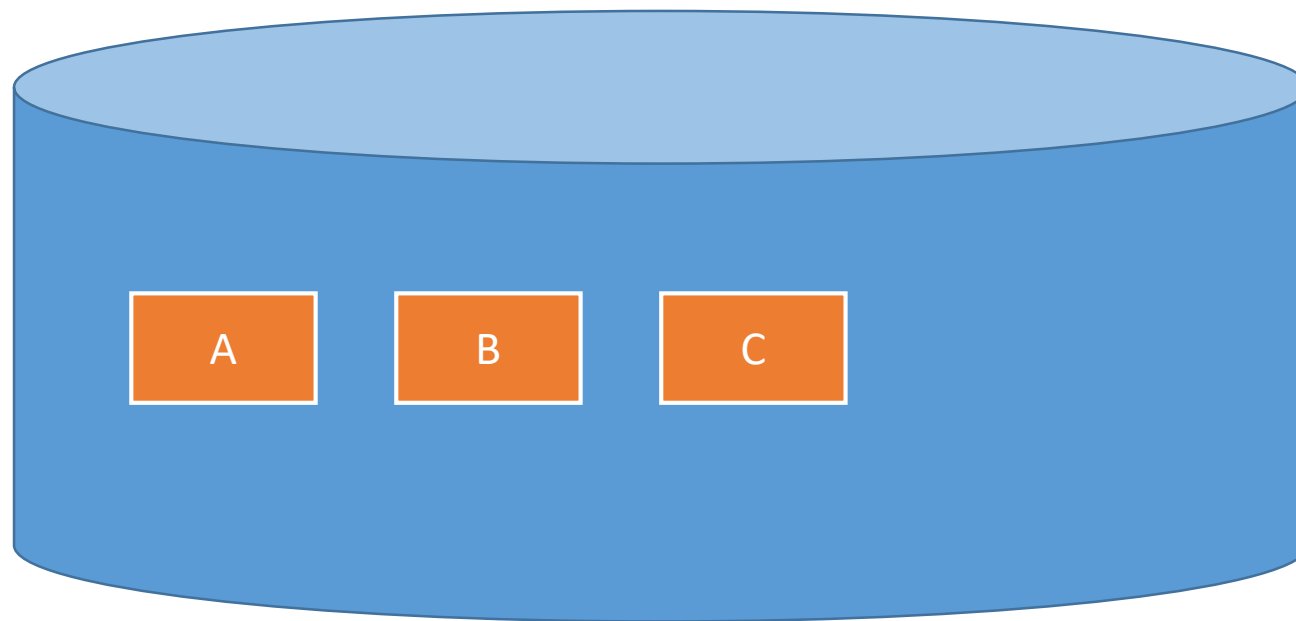
Dictionary



ที่เก็บข้อมูลพื้นฐาน

- Set

- ไม่มีอันดับ ซ้ำไม่ได้



ที่เก็บข้อมูลพื้นฐาน

- List
 - มีอันดับ เข้าได้



ที่เก็บข้อมูลพื้นฐาน

- List
 - มีอันดับ เข้าได้



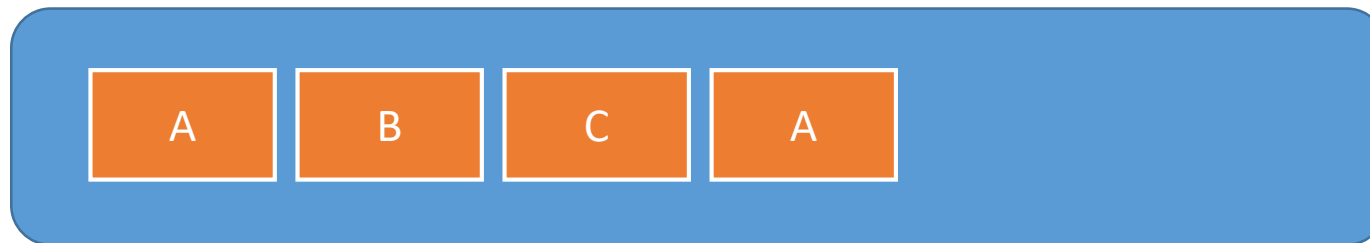
ที่เก็บข้อมูลพื้นฐาน

- List
 - มีอันดับ เข้าได้



ที่เก็บข้อมูลพื้นฐาน

- List
 - มีอันดับ เข้าได้



ที่เก็บข้อมูลพื้นฐาน

- Queue

- มีอันดับ เข้าก่อน ออกก่อน



ที่เก็บข้อมูลพื้นฐาน

- Queue

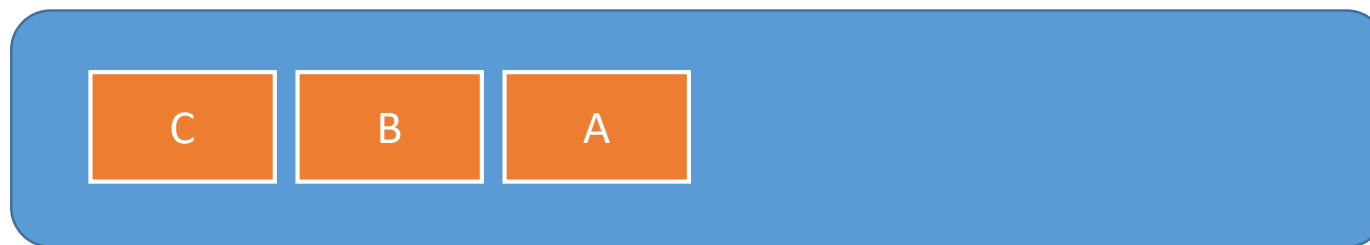
- มีอันดับ เข้าก่อน ออกก่อน



ที่เก็บข้อมูลพื้นฐาน

- Queue

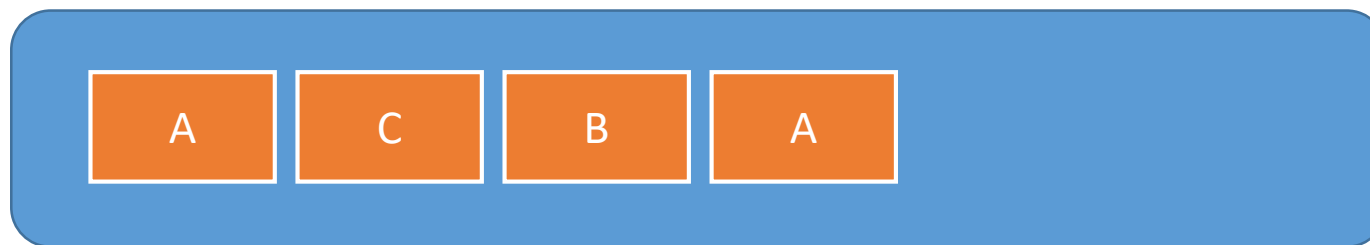
- มีอันดับ เข้าก่อน ออกก่อน



ที่เก็บข้อมูลพื้นฐาน

- Queue

- มีอันดับ เข้าก่อน ออกก่อน



ที่เก็บข้อมูลพื้นฐาน

- Queue

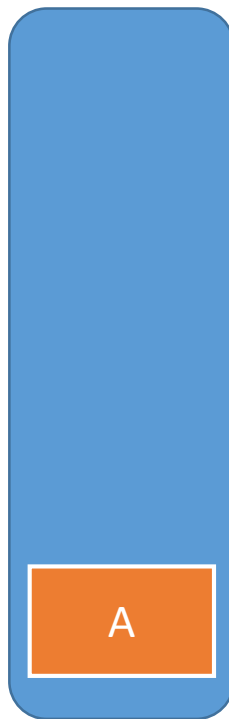
- มีอันดับ เข้าก่อน ออกก่อน



ที่เก็บข้อมูลพื้นฐาน

- Stack

- มีอันดับ เข้าหลัง ออกก่อน



ที่เก็บข้อมูลพื้นฐาน

- Stack

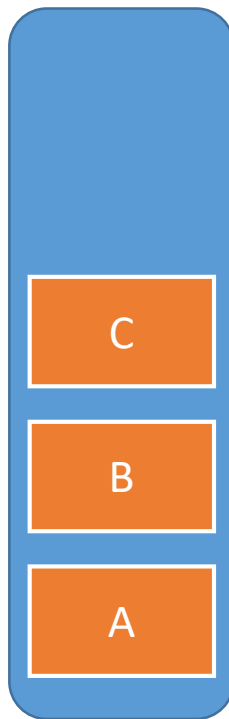
- มีอันดับ เข้าหลัง ออกก่อน



ที่เก็บข้อมูลพื้นฐาน

- Stack

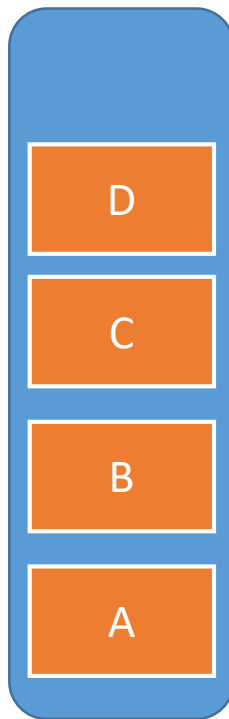
- มีอันดับ เข้าหลัง ออกก่อน



ที่เก็บข้อมูลพื้นฐาน

- Stack

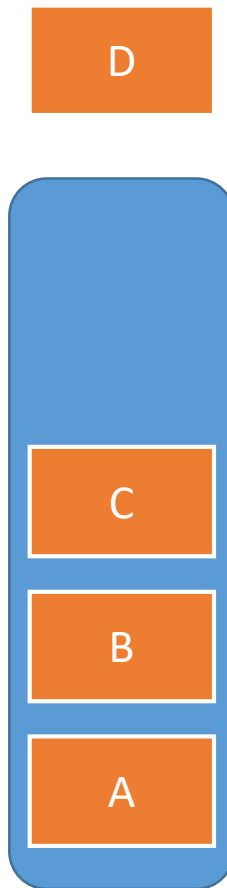
- มีอันดับ เข้าหลัง ออกก่อน



ที่เก็บข้อมูลพื้นฐาน

- Stack

- มีอันดับ เข้าหลัง ออกก่อน

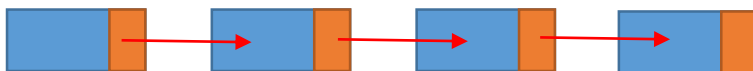


วิธีสร้างที่เก็บข้อมูล

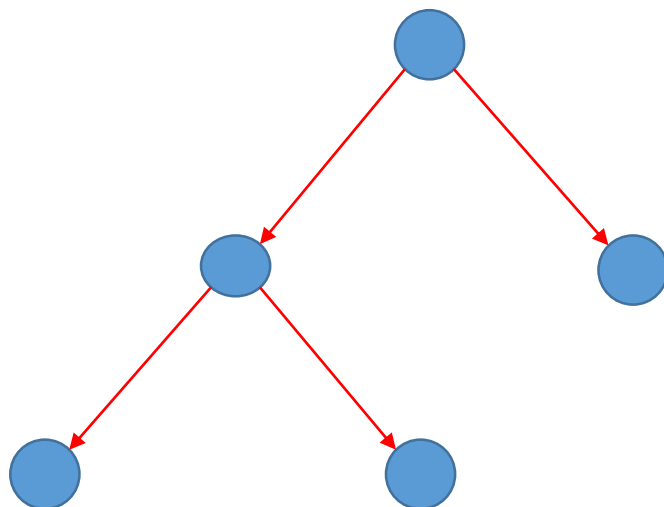
- ใช้ array



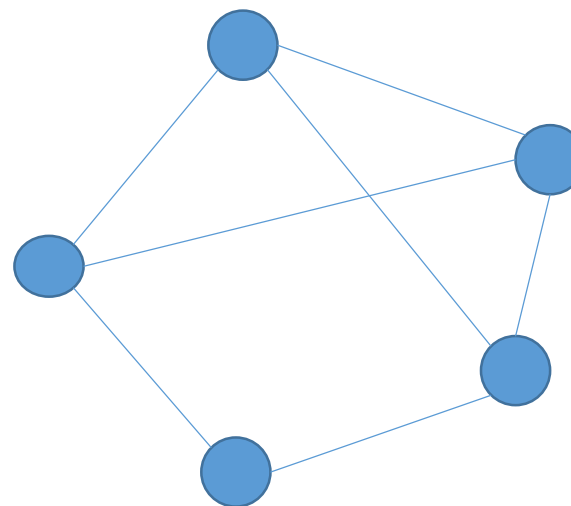
- ใช้การโยง



- ใช้ต้นไม้



- ใช้กราฟ



- ใช้ตาราง

วัตถุประสงค์

- เลือกใช้ให้เหมาะสมกับปัญหา
- รู้ว่าจะใช้งานแต่ละ **data structure** อย่างไร
- สามารถ **implement** ได้

Basic Python

Our editors & shell

- <http://pythontutor.com/visualize.html>
- <https://repl.it/languages/python3>

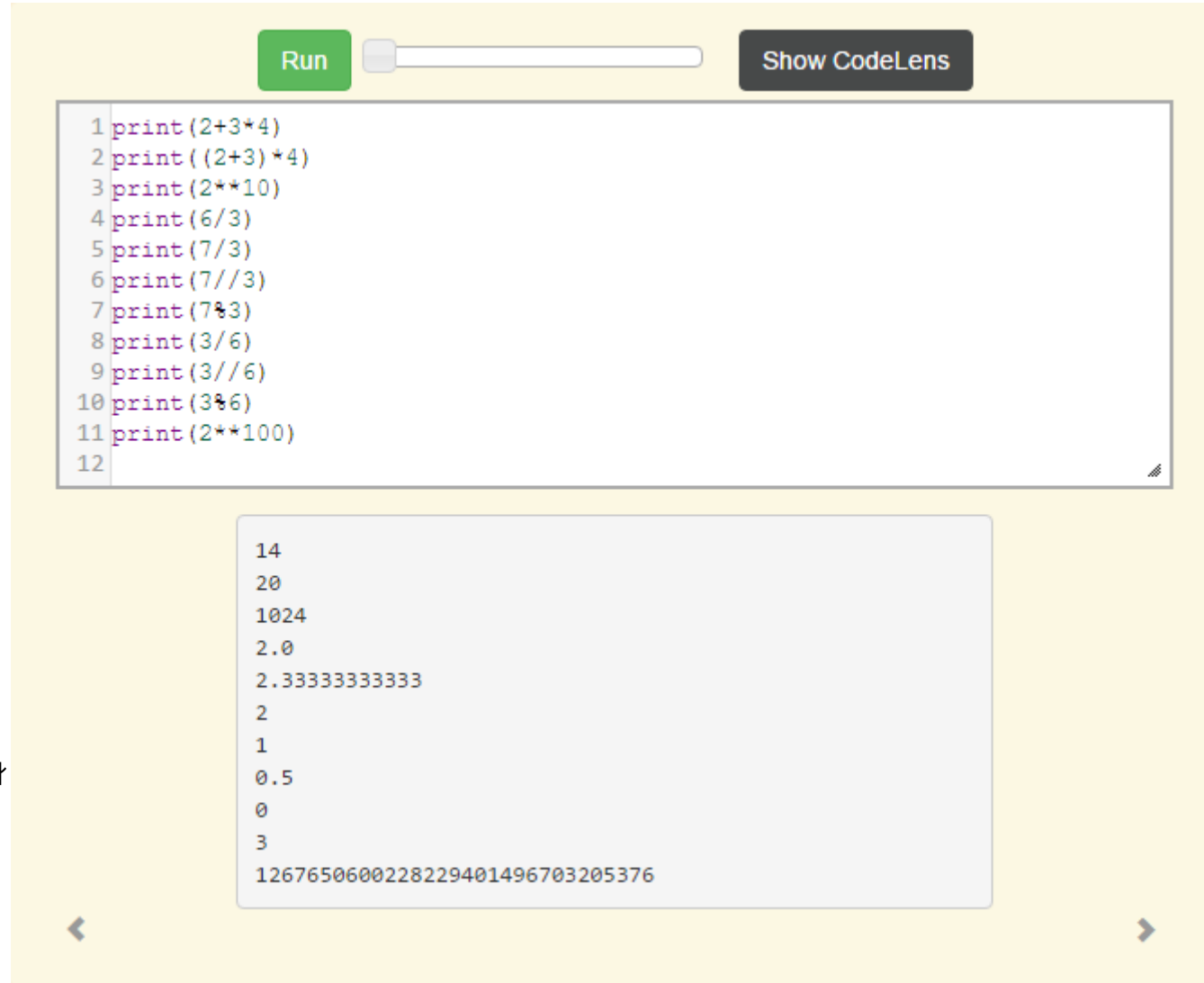
Indentation based

- ไม่มี `begin end` หรือ `{ }` เป็นตัวบอก scope
- ใช้ `space` หรือ `tab` เข้าไปแทน
 - ระวังเรื่องการใช้ `space` และ `tab` (บาง editor ไม่แปลง `tab` เป็น `space` ให้)

```
def fib(n):  
    ↑ print 'n =', n  
    ↑ if n > 1:  
        ↑ return n * fib(n - 1)  
    else:  
        ↑ print 'end of the line'  
        ↑ return 1
```

Numeric Data types

- int – จำนวนเต็ม
- float – จำนวนทศนิยม
- Operation
 - +
 - -
 - *
 - /
 - ** - ยกกำลัง
 - % - หารจำนวนเต็มเอาเศษ
 - // - หารจำนวนเต็ม



The screenshot shows a Python code editor with a 'Run' button and a 'Show CodeLens' button. The code consists of 11 lines of print statements. The output is displayed in a separate window below the code editor.

```
1 print(2+3*4)
2 print((2+3)*4)
3 print(2**10)
4 print(6/3)
5 print(7/3)
6 print(7//3)
7 print(7%3)
8 print(3/6)
9 print(3//6)
10 print(3%6)
11 print(2**100)
```

Output:

```
14
20
1024
2.0
2.333333333333333
2
1
0.5
0
3
1267650600228229401496703205376
```

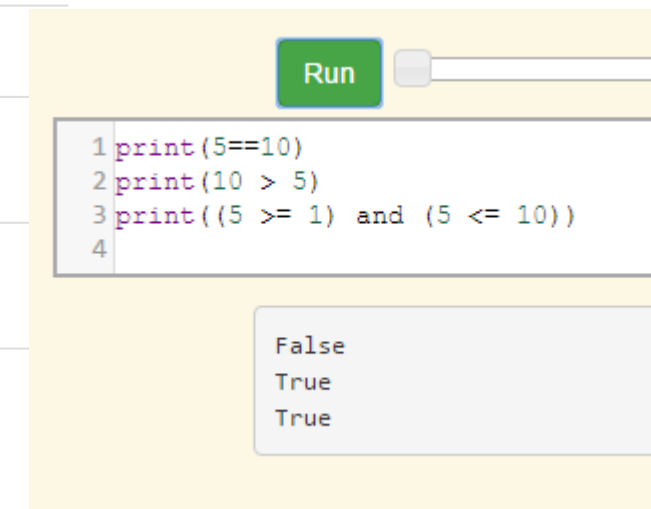
Boolean

- True
- False
- Operation
 - and
 - Or
 - not

```
>>> True
True
>>> False
False
>>> False or True
True
>>> not (False or True)
False
>>> True and True
True
```

Logical operators

Operation Name	Operator	Explanation
less than	<	น้อยกว่า
greater than	>	มากกว่า
less than or equal	<=	น้อยกว่าเท่ากับ
greater than or equal	>=	มากกว่าเท่ากับ
equal	==	เท่ากันหรือไม่
not equal	!=	ไม่เท่ากันหรือไม่
logical and	and	True and True = True
logical or	or	True ตัวใดตัวหนึ่ง = True
logical not	not	นิเสธ



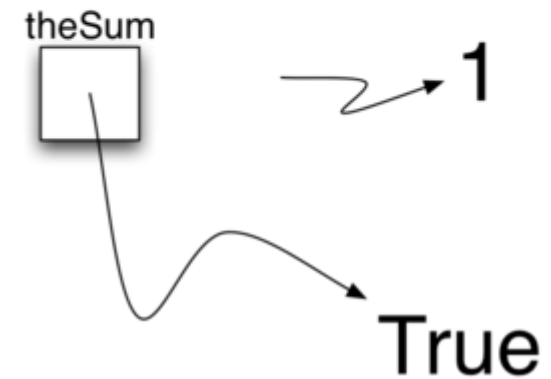
```
1 print(5==10)
2 print(10 > 5)
3 print((5 >= 1) and (5 <= 10))
4
```

False
True
True

การกำหนดค่าแบบอ้างอิง

- ไม่ยึดติดกับ data type

```
int → >>> theSum = 0
      >>> theSum
      0
      >>> theSum = theSum + 1
      >>> theSum
      1
boolean → >>> theSum = True
          >>> theSum
          True
```



Data structures พื้นฐานใน python

- มีอันดับ
 - List (หรือ array), string, tuple
- ไม่มีอันดับ
 - Set, dictionary

List

- สร้างด้วย [] คั่นด้วย ,
- ไม่จำเป็นต้องเป็น **data type** แบบเดียวกัน
- [] คือ **list** ว่าง

```
>>> [1,3,True,6.5]
[1, 3, True, 6.5]
>>> myList = [1,3,True,6.5]
>>> myList
[1, 3, True, 6.5]
```

- เริ่มต้น **list** ด้วยค่าเดียวกัน

```
>>> myList = [0] * 6
>>> myList
[0, 0, 0, 0, 0, 0]
```

List's operators

Operation Name	Operator	Explanation
indexing	[]	ดึงค่าจากตำแหน่งที่กำหนดใน []
concatenation	+	รวมลำดับ
repetition	*	รวมลำดับแบบทำซ้ำเป็นจำนวนครั้ง
membership	in	ถามว่ามี item ใน list หรือไม่
length	len	จำนวน item ใน list
slicing	[:]	ดึงลำดับมาบางส่วน

Run

Show CodeLens

```
1 myList = [1,2,3,4]
2 A = [myList]*3
3 print(A)
4 myList[2]=45
5 print(A)
6
7
```

[[1, 2, 3, 4], [1, 2, 3, 4], [1, 2, 3, 4]]
[[1, 2, 45, 4], [1, 2, 45, 4], [1, 2, 45, 4]]

List's methods

Method Name	Use	Explanation
append	<code>alist.append(item)</code>	เพิ่ม item ท้าย list
insert	<code>alist.insert(i,item)</code>	เพิ่ม item ที่ตำแหน่ง i
pop	<code>alist.pop()</code>	คืนค่า item ที่ตำแหน่งสุดท้ายแล้วเอาออกด้วย
pop	<code>alist.pop(i)</code>	คืนค่า item ที่ตำแหน่ง i แล้วเอาออกด้วย
sort	<code>alist.sort()</code>	เรียงลำดับใน list
reverse	<code>alist.reverse()</code>	ย้อนลำดับใน list
del	<code>del alist[i]</code>	ลบ item ที่ตำแหน่ง i
index	<code>alist.index(item)</code>	ค้นหา item แล้วคืนค่าตำแหน่งที่เจอมาให้
count	<code>alist.count(item)</code>	นับจำนวนที่มีค่าเดียวกับ item ใน list
remove	<code>alist.remove(item)</code>	ลบตัวที่มีค่าเดียวกับ item ตัวแรกที่เจอ

ตัวอย่าง

Run

Show CodeLens

```
1 myList = [1024, 3, True, 6.5]
2 myList.append(False)
3 print(myList)
4 myList.insert(2, 4.5)
5 print(myList)
6 print(myList.pop())
7 print(myList)
8 print(myList.pop(1))
9 print(myList)
10 myList.pop(2)
11 print(myList)
12 myList.sort()
13 print(myList)
14 myList.reverse()
15 print(myList)
16 print(myList.count(6.5))
17 print(myList.index(4.5))
18 myList.remove(6.5)
19 print(myList)
20 del myList[0]
21 print(myList)
22
```

```
[1024, 3, True, 6.5, False]
[1024, 3, 4.5, True, 6.5, False]
False
[1024, 3, 4.5, True, 6.5]
3
[1024, 4.5, True, 6.5]
[1024, 4.5, 6.5]
[4.5, 6.5, 1024]
[1024, 6.5, 4.5]
1
2
[1024, 4.5]
[4.5]
```

range

- เป็น **function** ที่ใช้สร้างลำดับของตัวเลข
- ใช้บ่อยใน **for loop**
- การทำงานต่างกันตามจำนวน **parameter**
 - **range(n)** สร้าง 0 ถึง n-1
 - **range(n, m)** สร้าง n ถึง m-1
 - **range(n, m, o)** สร้าง n ด้วย step o และไม่เกิน m

```
>>> range(10)
range(0, 10)
>>> list(range(10))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> range(5,10)
range(5, 10)
>>> list(range(5,10))
[5, 6, 7, 8, 9]
>>> list(range(5,10,2))
[5, 7, 9]
>>> list(range(10,1,-1))
[10, 9, 8, 7, 6, 5, 4, 3, 2]
>>>
```

String

Method Name	Use	Explanation
center	<code>astring.center(w)</code>	สร้าง string ความยาว w โดยมีค่าเดิมอยู่ตรงกลาง
count	<code>astring.count(item)</code>	จำนวนที่ค้นหาค่าเดียวกับ item เจอ
ljust	<code>astring.ljust(w)</code>	สร้าง string ความยาว w โดยมีค่าเดิมอยู่ด้านซ้าย
lower	<code>astring.lower()</code>	สร้าง string ที่เป็น lower case
rjust	<code>astring.rjust(w)</code>	สร้าง string ความยาว w โดยมีค่าเดิมอยู่ด้านขวา
find	<code>astring.find(item)</code>	คืนค่า index แรกที่เจอตัวที่มีค่าเดียวกับ item
split	<code>astring.split(schar)</code>	แบ่ง string ด้วยตัวคั่น schar (ใช้บ่อย)

```
>>> "David"
'David'
>>> myName = "David"
>>> myName[3]
'i'
>>> myName*2
'DavidDavid'
>>> len(myName)
5
>>>
```

```
>>> myName
'David'
>>> myName.upper()
'DAVID'
>>> myName.center(10)
'  David  '
>>> myName.find('v')
2
>>> myName.split('v')
['Da', 'id']
```

Mutability

- ความสามารถในการเปลี่ยนแปลงค่า
- List ทำได้
- String ทำไม่ได้

```
>>> myList
[1, 3, True, 6.5]
>>> myList[0]=2**10
>>> myList
[1024, 3, True, 6.5]
>>>
>>> myName
'David'
>>> myName[0]='X'

Traceback (most recent call last):
  File "<pyshell#84>", line 1, in -toplevel-
    myName[0]='X'
TypeError: object doesn't support item assignment
>>>
```

Tuple

- คล้าย **list** แต่ **immutable**
- สร้างโดยอยู่ใน () คั่นด้วย ,

```
>>> myTuple = (2,True,4.96)
>>> myTuple
(2, True, 4.96)
>>> len(myTuple)
3
>>> myTuple[0]
2
>>> myTuple * 3
(2, True, 4.96, 2, True, 4.96, 2, True, 4.96)
>>> myTuple[0:2]
(2, True)
>>>
```

```
>>> myTuple[1]=False
```

```
Traceback (most recent call last):
```

```
  File "<pysHELL#137>", line 1, in -toplevel-
```

```
    myTuple[1]=False
```

```
TypeError: object doesn't support item assignment
```

```
>>>
```

Set

- Immutable, ไม่มีอันดับ
- สร้างโดยอยู่ใน { } คั่นด้วย ,

```
>>> {3,6,"cat",4.5,False}
{False, 4.5, 3, 6, 'cat'}
>>> mySet = {3,6,"cat",4.5,False}
>>> mySet
{False, 4.5, 3, 6, 'cat'}
>>>
```

Set's operators

Operation Name	Operator	Explanation
membership	in	ถามว่ามีค่านั้นๆ ใน set หรือไม่
length	len	ขนาดของ set
	aset otherset	union ของ 2 sets
&	aset & otherset	Intersection ของ 2 sets
-	aset - otherset	Difference ของ 2 sets คือ ตัวที่มีใน set แรกแต่ไม่มีใน set ที่สอง
<=	aset <= otherset	ถามว่าสมาชิกใน set แรกน้อยกว่าเท่ากับสมาชิกใน set ที่สอง ทั้งหมดหรือไม่

```
>>> mySet
{False, 4.5, 3, 6, 'cat'}
>>> len(mySet)
5
>>> False in mySet
True
>>> "dog" in mySet
False
>>>
```


Set's methods

Method Name	Use	Explanation
union	aset.union(otherset)	union ของ 2 sets
intersection	aset.intersection(otherset)	Intersection ของ 2 sets
difference	aset.difference(otherset)	Difference ของ 2 sets คือ ตัวที่มีใน set แรกแต่ไม่มีใน set ที่สอง
issubset	aset.issubset(otherset)	ถามว่าเป็น subset ของ otherset หรือไม่
add	aset.add(item)	เพิ่ม item เข้าใน set
remove	aset.remove(item)	ลบ item ออกจาก set
pop	aset.pop()	คืนค่าและลบทิ้งสมาชิก 1 ตัว
clear	aset.clear()	ลบ item ทั้งหมด

```
>>> mySet
{False, 4.5, 3, 6, 'cat'}
>>> yourSet = {99,3,100}
>>> mySet.union(yourSet)
{False, 4.5, 3, 100, 6, 'cat', 99}
>>> mySet | yourSet
{False, 4.5, 3, 100, 6, 'cat', 99}
>>> mySet.intersection(yourSet)
{3}
>>> mySet & yourSet
{3}
>>> mySet.difference(yourSet)
{False, 4.5, 6, 'cat'}
>>> mySet - yourSet
{False, 4.5, 6, 'cat'}
>>> {3,100}.issubset(yourSet)
True
>>> {3,100}<=yourSet
True
>>> mySet.add("house")
>>> mySet
{False, 4.5, 3, 6, 'house', 'cat'}
>>> mySet.remove(4.5)
>>> mySet
{False, 3, 6, 'house', 'cat'}
>>> mySet.pop()
False
>>> mySet
{3, 6, 'house', 'cat'}
>>> mySet.clear()
>>> mySet
set()
>>>
```

Dictionary

- Mutable, ไม่มีอันดับ
- คู่ลำดับของข้อมูล
 - Key
 - Value
- สร้างโดยอยู่ใน { } มี : คั่นระหว่าง key:value มี , คั่นระหว่างคู่ลำดับ

```
>>> capitals = {'Iowa': 'DesMoines', 'Wisconsin': 'Madison'}  
>>> capitals  
{ 'Wisconsin': 'Madison', 'Iowa': 'DesMoines' }  
>>>
```

Dictionary's operators

Operator	Use	Explanation
[]	myDict[key]	คืนค่าที่สัมพันธ์กับ key
in	key in dict	มี key อยู่ใน dictionary หรือไม่
del	del dict[key]	ลบคู่ลำดับที่สัมพันธ์กับ key

Run

Hide Codelens

```
1 capitals = {'Iowa': 'DesMoines', 'Wisconsin': 'Madison'}
2 print(capitals['Iowa'])
3 capitals['Utah'] = 'SaltLakeCity'
4 print(capitals)
5 capitals['California'] = 'Sacramento'
6 print(len(capitals))
7 for k in capitals:
8     print(capitals[k], " is the capital of ", k)
9
```

DesMoines
{'Iowa': 'DesMoines', 'Wisconsin': 'Madison', 'Utah': 'SaltLakeCity'}
4
DesMoines is the capital of Iowa
Madison is the capital of Wisconsin
SaltLakeCity is the capital of Utah
Sacramento is the capital of California

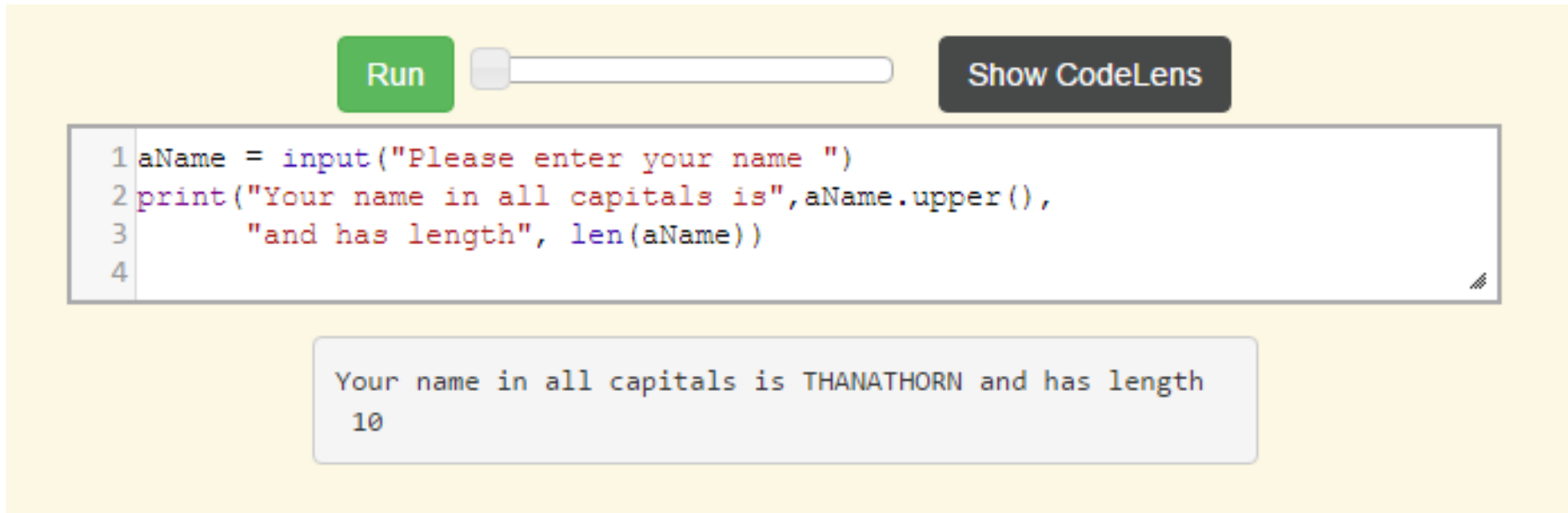
Dictionary's methods

Method Name	Use	Explanation
keys	dict.keys()	คืนค่า key ทั้งหมด
values	dict.values()	คืนค่า value ทั้งหมด
items	dict.items()	คืนค่าคู่ลำดับทั้งหมด
get	dict.get(k)	คืนค่าที่สัมพันธ์กับ key k ถ้าไม่เจอจะคืนค่า None
get	dict.get(k,alt)	คืนค่าที่สัมพันธ์กับ key k ถ้าไม่เจอจะคืนค่า alt

```
>>> phoneext={'david':1410,'brad':1137}
>>> phoneext
{'brad': 1137, 'david': 1410}
>>> phoneext.keys()
dict_keys(['brad', 'david'])
>>> list(phoneext.keys())
['brad', 'david']
>>> phoneext.values()
dict_values([1137, 1410])
>>> list(phoneext.values())
[1137, 1410]
>>> phoneext.items()
dict_items([('brad', 1137), ('david', 1410)])
>>> list(phoneext.items())
[('brad', 1137), ('david', 1410)]
>>> phoneext.get("kent")
>>> phoneext.get("kent","NO ENTRY")
'NO ENTRY'
>>>
```

Input และ output

- in = input(string)
- Output ใช้ print(string)



The screenshot shows a Python code editor interface. At the top, there is a green 'Run' button, a progress slider, and a dark grey 'Show CodeLens' button. Below these is a code editor with the following Python code:

```
1 aName = input("Please enter your name ")
2 print("Your name in all capitals is", aName.upper(),
3       "and has length", len(aName))
4
```

Below the code editor, the output of the program is displayed in a light grey box:

```
Your name in all capitals is THANATHORN and has length
10
```

Input ข้อควรระวัง

- `in = input(string)`
 - สิ่งที่คืนค่าให้กับ `in` คือ `string`
 - ก่อนนำไปใช้ต้องรู้ว่าเราจะเอาไปใช้งานเป็น `data type` แบบไหน
 - ใช้เป็น `float` ก็ต้องแปลงเป็น `float` ก่อน



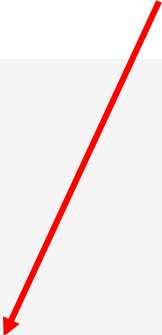
```
sradius = input("Please enter the radius of the circle ")  
radius = float(sradius)  
diameter = 2 * radius
```

String format

- Python 3 เท่านั้น !!!!

```
>>> print("Hello")
Hello
>>> print("Hello","World")
Hello World
>>> print("Hello","World", sep="***")
Hello***World
>>> print("Hello","World", end="***")
Hello World***>>>
```

ตัวคั่น



ลงท้าย



String format

- % ที่อยู่ใน “ ” บอกเราว่าจะพิมพ์ออกเป็น data type ไหน
 - %d จำนวนเต็ม
 - %f จำนวนทศนิยม
 - %s string
 - อื่นๆ ลองไปศึกษาดู
- % ที่อยู่ข้างนอกบอกว่าตัวแปรข้างหลังจะนำมาแทนที่ % ที่อยู่ใน “ ” ตามลำดับ

```
> aName = "Eiei"
=> None
> age = 16
=> None
> print("%s is %d years old." % (aName, age))
Eiei is 16 years old.
```


String format

Modifier	Example	Description
number	%20d	พิมพ์ค่าโดยจองที่ให้ 20 ตัว ชิดขวา
-	%-20d	พิมพ์ค่าโดยจองที่ให้ 20 ตัว ชิดซ้าย
+	%+20d	พิมพ์ค่าโดยจองที่ให้ 20 ตัว ชิดซ้าย
0	%020d	พิมพ์ค่าโดยจองที่ให้ 20 ตัว เต็ม 0 ให้ข้างหน้าจนเต็ม (ใช้ทำอะไร ลองคิดดู)
.	%20.2f	พิมพ์ค่าโดยจองที่ให้ 20 ตัว ทศนิยม 2 ตำแหน่ง
(name)	%(name)d	นำค่าจาก dictionary มาพิมพ์ตาม key name

String format

```
>>> price = 24
>>> item = "banana"
>>> print("The %s costs %d cents"%(item,price))
The banana costs 24 cents
>>> print("The %+10s costs %5.2f cents"%(item,price))
The      banana costs 24.00 cents
>>> print("The %+10s costs %10.2f cents"%(item,price))
The      banana costs      24.00 cents
>>> itemdict = {"item":"banana","cost":24}
>>> print("The %(item)s costs %(cost)7.1f cents"%itemdict)
The banana costs      24.0 cents
>>>
```

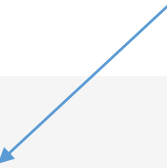
Control Structures

- while
- for
- If elif else

while

- ทำไปเรื่อยๆ เมื่อเงื่อนไขยังเป็นจริงอยู่

เงื่อนไข



```
>>> counter = 1
>>> while counter <= 5:
...     print("Hello, world")
...     counter = counter + 1
```

```
Hello, world
Hello, world
Hello, world
Hello, world
Hello, world
```

while

- ลองใน <http://pythontutor.com/visualize.html>

The screenshot displays the Python Tutor interface for Python 2.7. The code being executed is a while loop that prints 'hello' five times. The execution is currently at step 2, where the loop condition is being checked. The 'Print output' area is empty. The 'Frames' panel shows the 'Global frame' with the variable 'counter' set to 1. The 'Objects' panel is empty. The 'Print output' area is a large rectangle with a handle in the bottom right corner for resizing. The 'Frames' and 'Objects' panels are located below the 'Print output' area. The 'Frames' panel has a title bar with 'Global frame' and a list of variables: 'counter' with a value of 1. The 'Objects' panel is currently empty. The 'Print output' area is a large rectangle with a handle in the bottom right corner for resizing. The 'Frames' and 'Objects' panels are located below the 'Print output' area. The 'Frames' panel has a title bar with 'Global frame' and a list of variables: 'counter' with a value of 1. The 'Objects' panel is currently empty.

Python 2.7

```
→ 1 counter = 1
→ 2 while counter <= 5:
3     print("hello")
4     counter = counter + 1
```

[Edit code](#) | [Live programming](#)

→ line that has just executed
→ next line to execute

NEW! Click on a line of code to set a breakpoint. Then use the Forward and Back buttons to jump there.

Print output (drag lower right corner to resize)

Frames Objects

Global frame

counter 1

<< First < Back Step 2 of 17 Forward > Last >>

for

- ทำงานกับทุก **item** ใน **list**

```
>>> for item in [1,3,6,2,5]:  
...     print(item)  
...  
1  
3  
6  
2  
5
```

[0, 1, 2, 3, 4]



```
>>> for item in range(5):  
...     print(item**2)  
...  
0  
1  
4  
9  
16  
>>>
```

for ซ้อน for

Run

Hide Codelens

```
1 wordlist = ['cat', 'dog', 'rabbit']
2 letterlist = [ ]
3 for aword in wordlist:
4     for aletter in aword:
5         letterlist.append(aletter)
6 print(letterlist)
7
```

Python 3.3

```
1 wordlist = ['cat', 'dog', 'rabbit']
2 letterlist = [ ]
→ 3 for aword in wordlist:
→ 4     for aletter in aword:
5         letterlist.append(aletter)
6 print(letterlist)
```

→ line that has just executed
→ next line to execute

< Back Step 11 of 34 Forward >

Print output (drag lower right corner to resize)

Frames

Objects

Global frame

wordlist	
letterlist	
aword	"cat"
aletter	"t"

list

0	1	2
"cat"	"dog"	"rabbit"

list

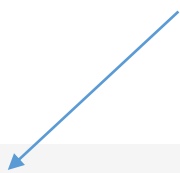
0	1	2
"c"	"a"	"t"

Visualized using [Online Python Tutor](#) by Philip Guo

if else

- ทำใน **if** ถ้าตรงกับเงื่อนไข
- ทำใน **else** ถ้าไม่ตรงกับเงื่อนไข

เงื่อนไข



```
if n<0:
    print("Sorry, value is negative")
else:
    print(math.sqrt(n))
```

```
if score >= 90:
    print('A')
else:
    if score >= 80:
        print('B')
    else:
        if score >= 70:
            print('C')
        else:
            if score >= 60:
                print('D')
            else:
                print('F')
```


if elif else

- ทำใน **if** ถ้าตรงกับเงื่อนไข
- ทำใน **elif** ถ้าตรงกับอีกเงื่อนไข
- ทำใน **else** ถ้าไม่ตรงกับเงื่อนไขไหนเลย

```
if score >= 90:  
    print('A')  
elif score >= 80:  
    print('B')  
elif score >= 70:  
    print('C')  
elif score >= 60:  
    print('D')  
else:  
    print('F')
```

for ในการสร้าง list

- Loop เพิ่ม item

```
>>> sqllist=[]
>>> for x in range(1,11):
        sqllist.append(x*x)

>>> sqllist
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
>>>
```

- อีกวิธี

```
>>> sqllist=[x*x for x in range(1,11)]
>>> sqllist
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
>>>
```

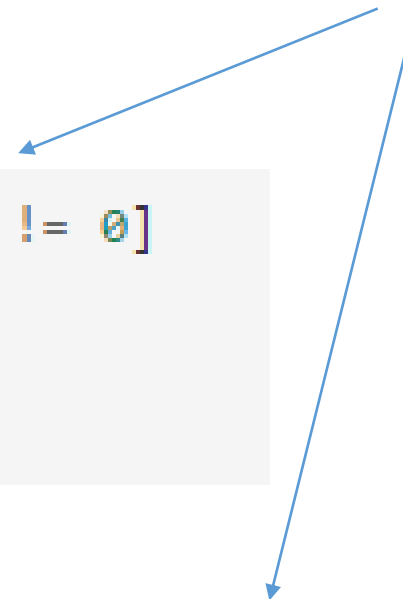
for และ if ในการสร้าง list

- เพิ่มใน **list** เมื่อตรงกับเงื่อนไข

```
>>> sqlist=[x*x for x in range(1,11) if x%2 != 0]
>>> sqlist
[1, 9, 25, 49, 81]
>>>
```

```
>>>[ch.upper() for ch in 'comprehension' if ch not in 'aeiou']
['C', 'M', 'P', 'R', 'H', 'N', 'S', 'N']
>>>
```

เงื่อนไข



Exception handling

- **Exception** คือความผิดพลาดทาง logic
- ไม่จัดการกับ exception

```
>>> anumber = int(input("Please enter an integer "))
Please enter an integer -23
>>> print(math.sqrt(anumber))
Traceback (most recent call last):
  File "<pyshell#102>", line 1, in <module>
    print(math.sqrt(anumber))
ValueError: math domain error
>>>
```

- จัดการกับ exception
 - **try** : ทำภายใต้ส่วน try
 - **except** : เกิด exception โยนมาส่วนนี้

```
>>> try:
    print(math.sqrt(anumber))
except:
    print("Bad Value for square root")
    print("Using absolute value instead")
    print(math.sqrt(abs(anumber)))
```

```
Bad Value for square root
Using absolute value instead
4.79583152331
>>>
```

Function

- def ชื่อ function (ส่งค่า parameter)
- คืนค่าด้วย return

```
>>> def square(n):  
...     return n**2  
...  
>>> square(3)  
9  
>>> square(square(3))  
81  
>>>
```

Class

- ชื่อ class
- Constructor (ตัวสร้าง)
 - `__init__` เป็นชื่อ method
 - `self` เป็นตัวแปรที่อ้างถึงตัวมันเอง
- Method พื้นฐาน
 - ขึ้นต้นและลงท้ายด้วย `__`
- Method อื่นๆ

ตัวอย่าง Fraction

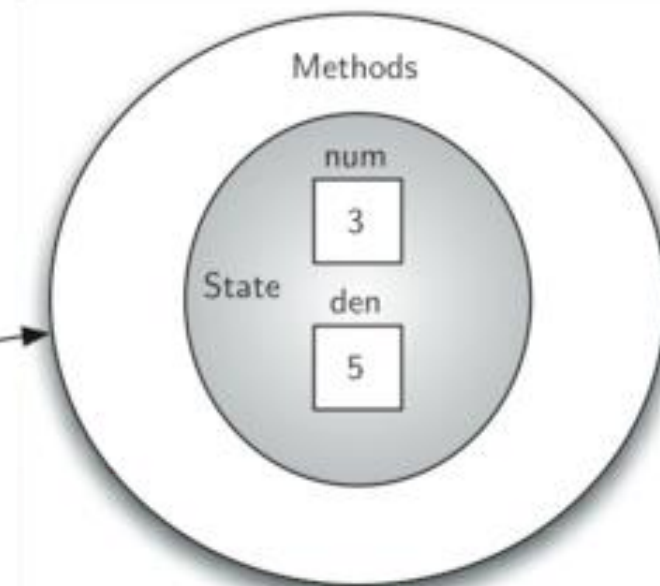
constructor

```
1 class Fraction:
2
3     def __init__(self, top, bottom):
4
5         self.num = top
6         self.den = bottom
7
8 myfraction = Fraction(3,5)
9 print(myfraction)
```

<__main__.Fraction object>

Print ไม่ออก

myfraction



ตัวอย่าง Fraction

```
1 class Fraction:
2
3     def __init__(self, top, bottom):
4         self.num = top
5         self.den = bottom
6
7     def show(self):
8         print(self.num, "/", self.den)
9
10 myfraction = Fraction(3, 5)
11 myfraction.show()
12 print(myfraction)
```

สร้าง method show มาแสดงผล

```
3 / 5
<__main__.Fraction object>
```

แสดงผลแบบนี้ใช้งานยาก

ตัวอย่าง fraction

- ใช้การสร้าง method พื้นฐาน (override)
 - `__str__`

```
1 class Fraction:
2
3     def __init__(self, top, bottom):
4         self.num = top
5         self.den = bottom
6
7     def __str__(self):
8         return str(self.num) + "/" + str(self.den)
9
10 myfraction = Fraction(3,5)
11 print(myfraction)
```

3/5

ตัวอย่าง fraction

- `__add__` บวก
- `__eq__` เปรียบเทียบ

```
def __add__(self, otherfraction):  
    newnum = self.num*otherfraction.den + self.den*otherfraction.num  
    newden = self.den * otherfraction.den  
    common = gcd(newnum,newden)  
    return Fraction(newnum//common,newden//common)
```

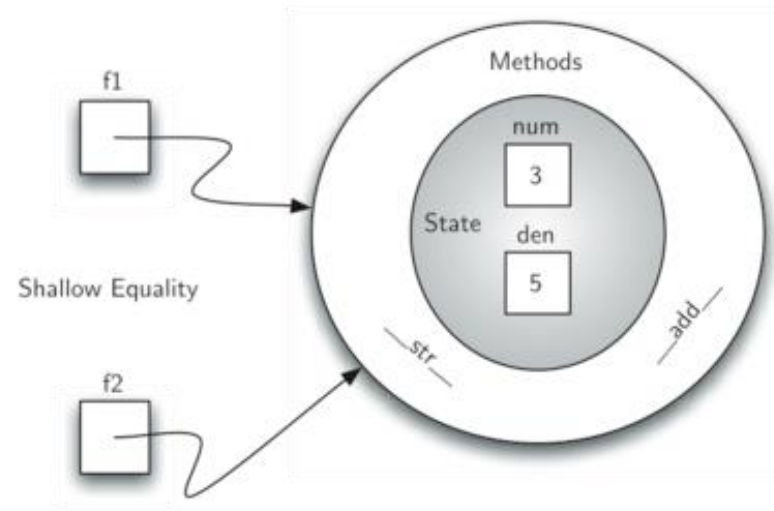
gcd เขียนเองไว้หาหรม.

```
>>> f1=Fraction(1,4)  
>>> f2=Fraction(1,2)  
>>> f3=f1+f2  
>>> print(f3)  
3/4  
>>>
```

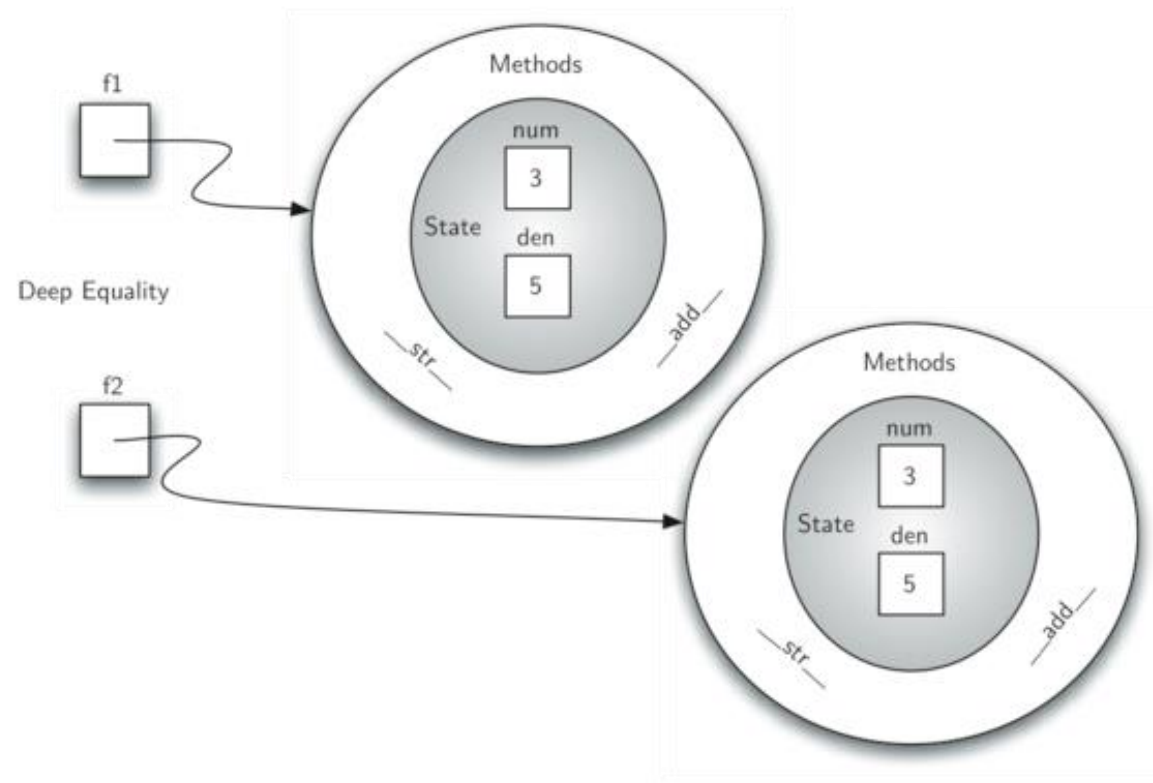
```
def __eq__(self, other):  
    firstnum = self.num * other.den  
    secondnum = other.num * self.den  
  
    return firstnum == secondnum
```

ตัวอย่าง fraction

- ทำไมต้องสร้าง `__eq__`



เปรียบเทียบ reference



แบบฝึกหัด

- <http://interactivepython.org/runestone/static/pythonds/Introduction/ProgrammingExercises.html>
- <http://codingbat.com/python>
- <http://10.31.26.54/grader/main/list>

Big Oh

Big Oh คืออะไร

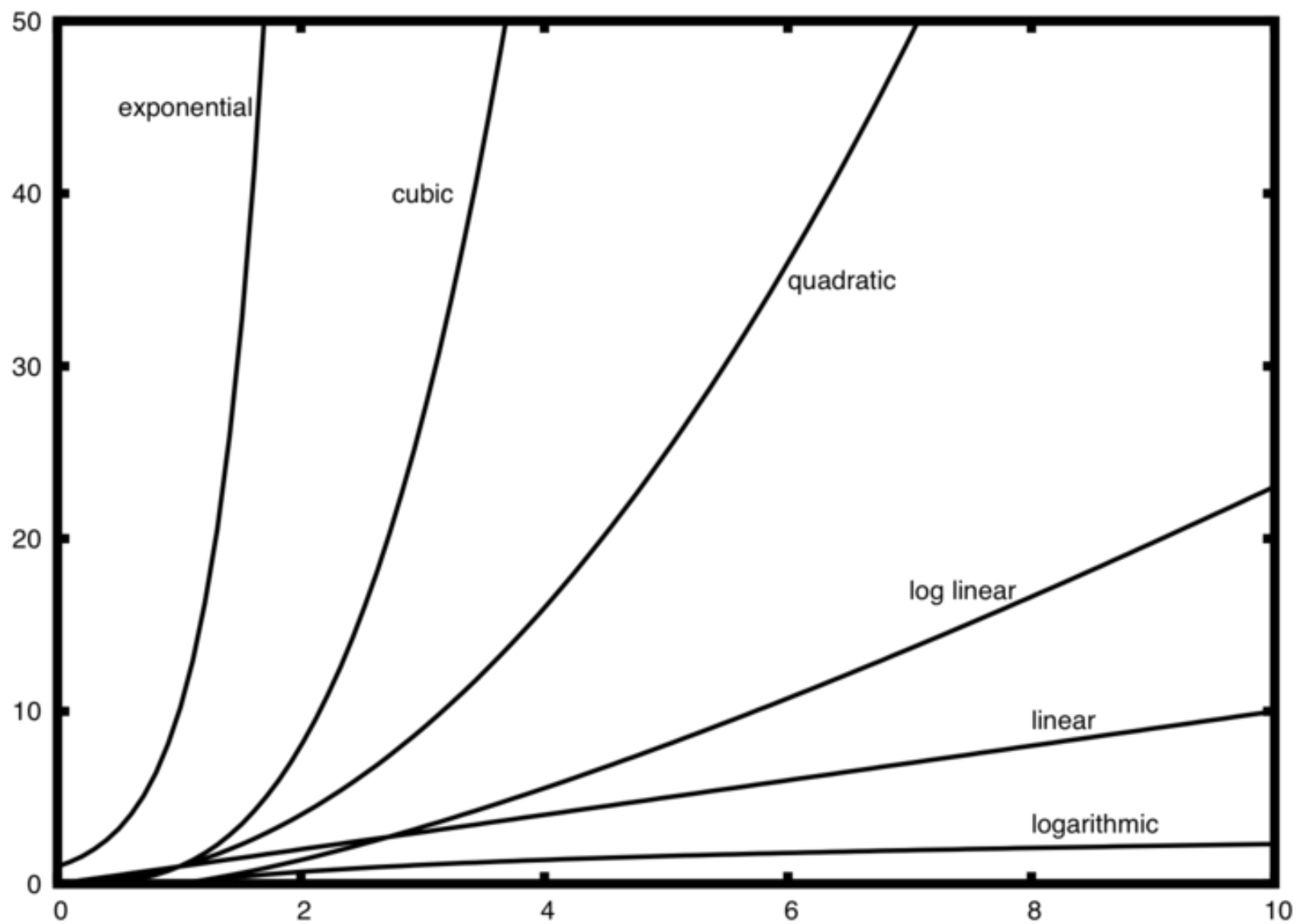
- ตัววัดประสิทธิภาพการทำงาน
- วัดตามจำนวนข้อมูล n
- ตัวอย่าง **sum of n**
 - จำนวน **step** ในการทำงาน $T(n) = 2 + n$
 - $O(n)$
 - ดูตัวที่ยกกำลังเยอะสุด
- $T(n) = 5n^2 + 3n$
 - $O(n^2)$
 - ดูตัวที่ยกกำลังเยอะสุดและไม่ต้องดูสัมประสิทธิ์

```
1 n = 10
2 thesum = 0
3 for i in range(1, n + 1) :
4     thesum = thesum + i
5 print(thesum)
```

ชื่อเรียก

f(n)	Name
1	Constant
$\log n$	Logarithmic
n	Linear
$n \log n$	Log Linear
n^2	Quadratic
n^3	Cubic
2^n	Exponential

กราฟอัตราการโต



Big Oh สำคัญอย่างไร?

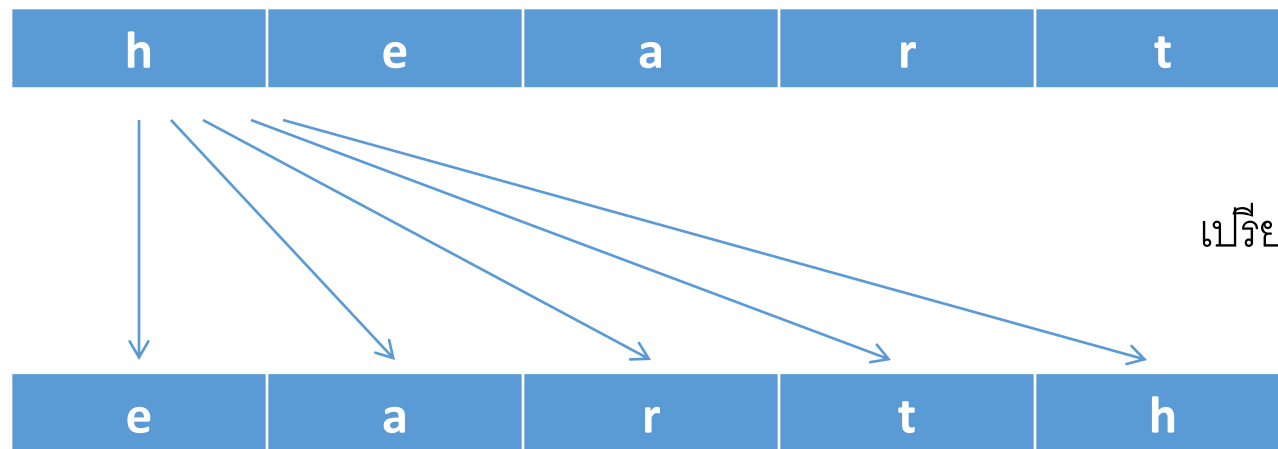
- รู้ว่าประสิทธิภาพของแต่ละ **data structure** เป็นอย่างไร
- เลือก **data structure** ที่มีประสิทธิภาพเหมาะกับปัญหา
- คิด **algorithm** ที่จะทำให้ใช้ **data structure** ได้อย่างมีประสิทธิภาพ

Anagram detection

- String 2 ตัวประกอบด้วยชุดตัวอักษรเดียวกันหรือไม่
- heart กับ earth → true
- python กับ typhon → true
- แก้ปัญหาได้หลายวิธี O ต่างกัน

Check off

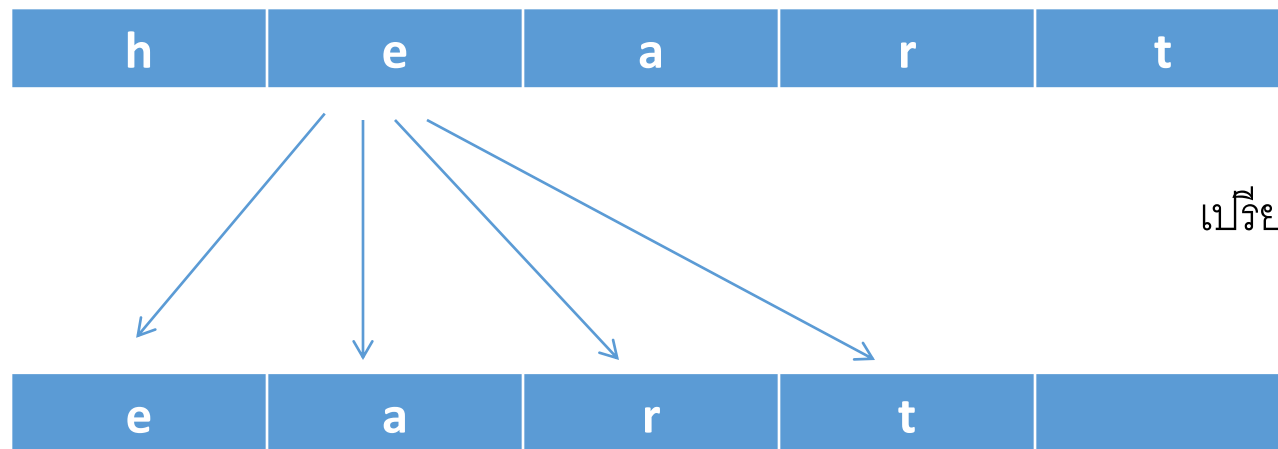
- ทดสอบทุกคู่



เปรียบเทียบ **n** ครั้ง

Check off

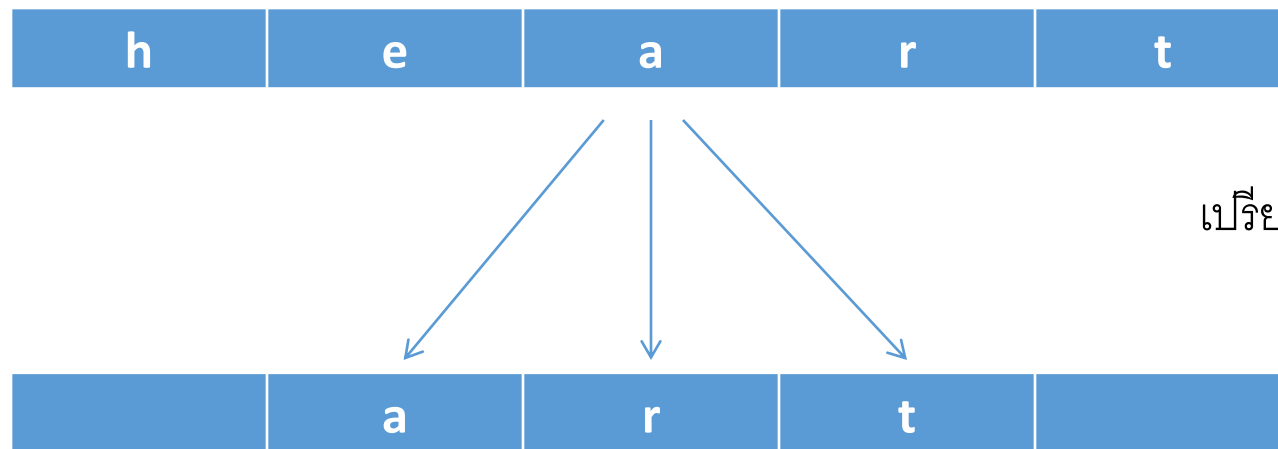
- ทดสอบทุกคู่



เปรียบเทียบ **n-1** ครั้ง

Check off

- ทดสอบทุกคู่



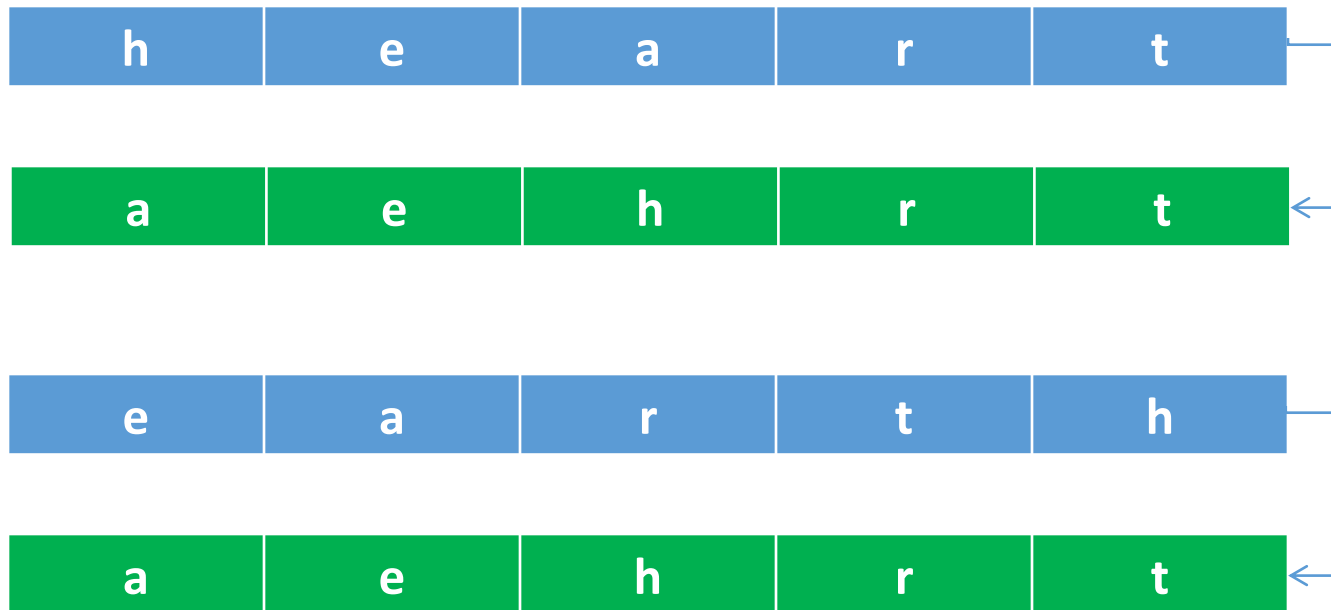
เปรียบเทียบ $n-2$ ครั้ง

Check off

- $n + (n - 1) + (n - 2) + \dots + 1$
- ออนุกรมเลขคณิต (ม. ปลาย)
- $n * (n + 1) / 2$
- $T(n) = \frac{1}{2} * n^2 + \frac{1}{2} * n = O(n^2)$

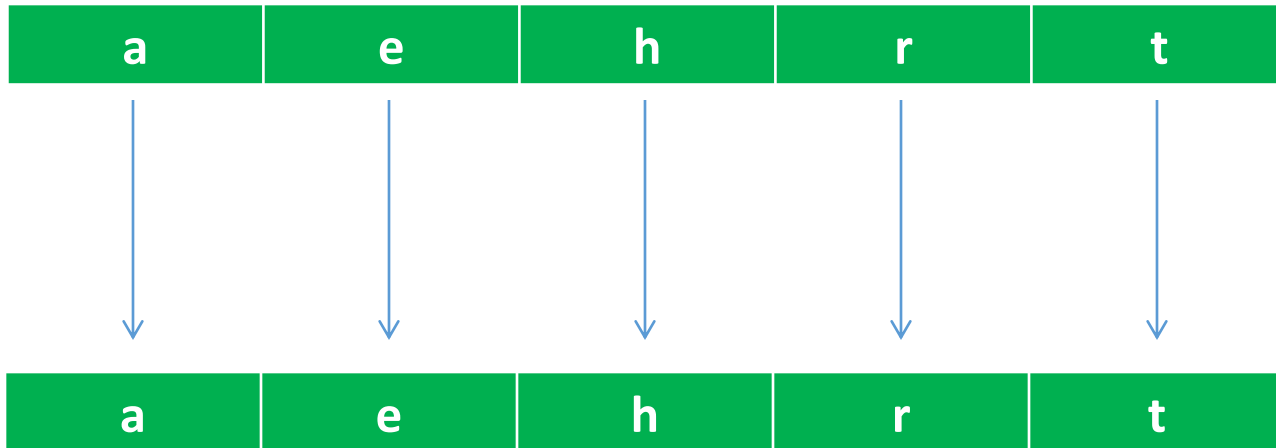
Sort & compare

- เรียงลำดับแล้วเปรียบเทียบเป็นคู่ๆ



Sort & compare

- เรียงลำดับแล้วเปรียบเทียบเป็นคู่ๆ



Sort & compare

- เรียงลำดับ
 - $O(n \log n)$
 - $O(n^2)$
- เปรียบเทียบเป็นคู่ๆ
 - $O(n)$
- รวม $O(n \log n) + O(n) \rightarrow O(n \log n)$

Count & compare

- นับแล้วเปรียบเทียบ
- ตัวอักษรทั้งหมด 26 ตัว

a	b	c	d	e	f	g	h	i	j
k	l	m	n	o	p	q	r	s	t
u	v	w	x	y	z				

Count & compare

- นับว่าแต่ละ **string** มีตัวอักษรตัวไหนเท่าไร
- นับ heart

a(1)	b	c	d	e(1)	f	g	h(1)	i	j
k	l	m	n	o	p	q	r(1)	s	t(1)
u	v	w	x	y	z				

- นับ earth

a(1)	b	c	d	e(1)	f	g	h(1)	i	j
k	l	m	n	o	p	q	r(1)	s	t(1)
u	v	w	x	y	z				

Count & compare

- เปรียบเทียบตัวอักษรเป็นคู่ๆ

heart

a(1)	b	c	d	e(1)	f	g	h(1)	i	j
k	l	m	n	o	p	q	r(1)	s	t(1)
u	v	w	x	y	z				

earth

a(1)	b	c	d	e(1)	f	g	h(1)	i	j
k	l	m	n	o	p	q	r(1)	s	t(1)
u	v	w	x	y	z				

Count & compare

- เปรียบเทียบตัวอักษรเป็นคู่ๆ

heart

a(1)	b	c	d	e(1)	f	g	h(1)	i	j
k	l	m	n	o	p	q	r(1)	s	t(1)
u	v	w	x	y	z				

earth

a(1)	b	c	d	e(1)	f	g	h(1)	i	j
k	l	m	n	o	p	q	r(1)	s	t(1)
u	v	w	x	y	z				

Count & compare

- เปรียบเทียบตัวอักษรเป็นคู่ๆ

heart

a(1)	b	c	d	e(1)	f	g	h(1)	i	j
k	l	m	n	o	p	q	r(1)	s	t(1)
u	v	w	x	y	z				

earth

a(1)	b	c	d	e(1)	f	g	h(1)	i	j
k	l	m	n	o	p	q	r(1)	s	t(1)
u	v	w	x	y	z				

Count & compare

- นับตัวอักษร
 - $T(n) = n * 2$
- เปรียบเทียบตัวอักษรเป็นคู่ๆ
 - $T(n) = 26$
- รวม $T(n) = n * 2 + 26 \rightarrow O(n)$

แบบฝึกหัด

- <http://interactivepython.org/runestone/static/pythonds/AlgorithmAnalysis/AnAnagramDetectionExample.html>

List

Operation	Big-O Efficiency
index []	O(1)
index assignment	O(1)
append	O(1)
pop()	O(1)
pop(i)	O(n)
insert(i,item)	O(n)
del operator	O(n)
iteration	O(n)
contains (in)	O(n)
get slice [x:y]	O(k)
del slice	O(n)
set slice	O(n+k)
reverse	O(n)
concatenate	O(k)
sort	O(n log n)
multiply	O(nk)

slice

```
1 L = ['a','b','c','d','e','f']
2 print(L[3:5])
3 L[3:5] = []
4 print(L)
```

```
['d', 'e']
['a', 'b', 'c', 'f']
```

Dictionary

operation	Big-O Efficiency
copy	$O(n)$
get item	$O(1)$
set item	$O(1)$
delete item	$O(1)$
contains (in)	$O(1)$
iteration	$O(n)$

Linear structure

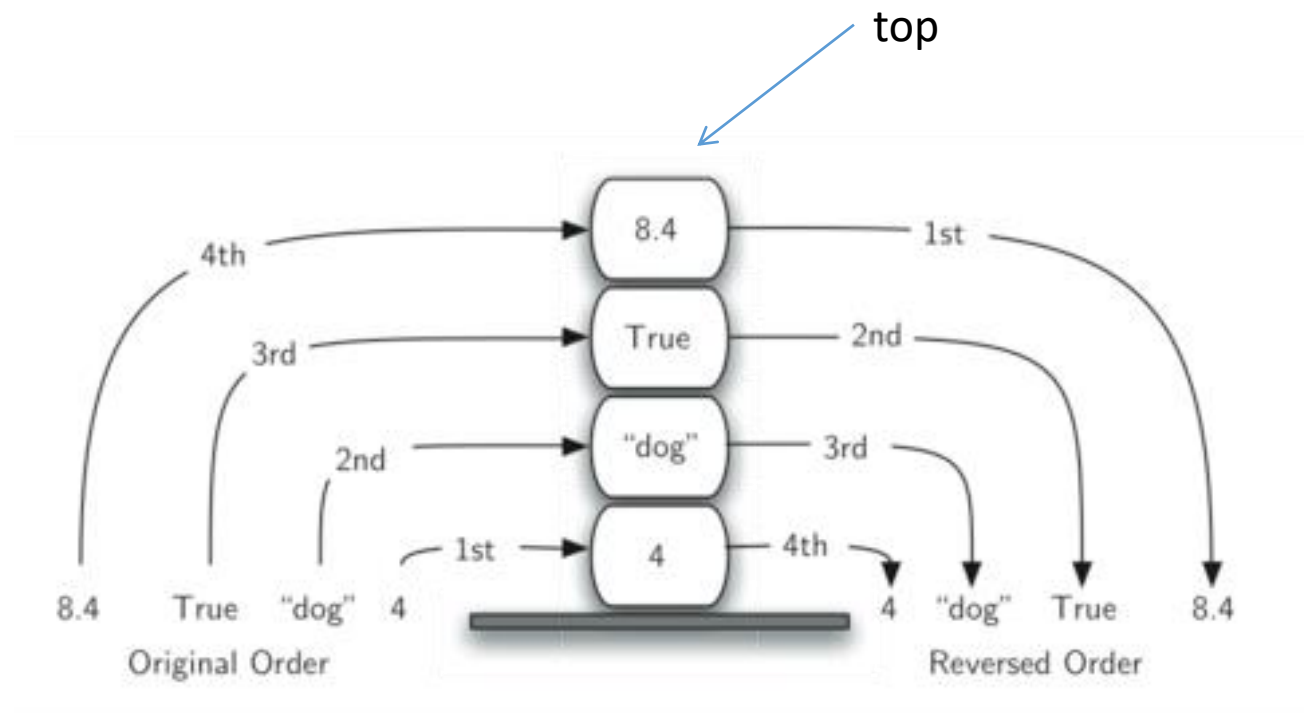
Linear structure คือ?

- โครงสร้างมีลำดับ
- มีหัว มีท้าย หรือ มีบน มีล่าง
- list, stack, queue
 - ต่างกันตรงวิธีเพิ่มและลบข้อมูล



Stack

- มีลำดับแบบ **last-in-first-out**
- ลำดับที่สำคัญคือ **top**



Stack ADT

- `stack` มีการทำงานอย่างไร ควรจะมี `method` อะไรบ้าง
- `Stack()` สร้าง `stack` ว่างๆ และ `return stack` มาให้
- `push(item)` เพิ่ม `item` และ `ไม่ return`
- `pop()` เอา `item` ที่อยู่บนสุดออกมาและ `return item` นั้น
- `peek()` `return item` ที่อยู่บนสุด
- `isEmpty()` `return` ค่าว่า `stack` ว่างหรือไม่
- `size()` `return` จำนวน `item` ใน `stack`

Stack ADT

Stack Operation	Stack Contents	Return Value
s.isEmpty()	[]	True
s.push(4)	[4]	
s.push('dog')	[4,'dog']	
s.peek()	[4,'dog']	'dog'
s.push(True)	[4,'dog',True]	
s.size()	[4,'dog',True]	3
s.isEmpty()	[4,'dog',True]	False
s.push(8.4)	[4,'dog',True,8.4]	
s.pop()	[4,'dog',True]	8.4
s.pop()	[4,'dog']	True
s.size()	[4,'dog']	2

Stack's implementation

```
1 class Stack:
2     def __init__(self):
3         self.items = []
4
5     def isEmpty(self):
6         return self.items == []
7
8     def push(self, item):
9         self.items.append(item)
10
11    def pop(self):
12        return self.items.pop()
13
14    def peek(self):
15        return self.items[len(self.items)-1]
16
17    def size(self):
18        return len(self.items)
19
```


การใช้งาน stack



The image shows a Python IDE interface with a yellow background. At the top, there is a green 'Run' button and a progress bar. Below this is a code editor with 16 lines of Python code. The code demonstrates the use of a 'Stack' class from a module named 'pythonds.basic.stack'. The operations performed are: creating a stack, checking if it's empty, pushing elements (4, 'dog', True, 8.4), checking its size, popping elements, and checking if it's empty again. Below the code editor is a text area showing the output of the program, which matches the expected results of the stack operations.

```
1 from pythonds.basic.stack import Stack
2
3 s=Stack()
4
5 print(s.isEmpty())
6 s.push(4)
7 s.push('dog')
8 print(s.peek())
9 s.push(True)
10 print(s.size())
11 print(s.isEmpty())
12 s.push(8.4)
13 print(s.pop())
14 print(s.pop())
15 print(s.size())
16
```

True
dog
3
False
8.4
True
2

Stack in action

Python 2.7

```
7
8     def push(self, item):
9         self.items.insert(0,item)
10
11     def pop(self):
12         return self.items.pop(0)
13
14     def peek(self):
15         return self.items[0]
16
17     def size(self):
18         return len(self.items)
19
20 → s = Stack()
21 s.push('hello')
22 s.push('true')
23 print(s.pop())
```

<< First

< Back

Step 2 of 17

Forward >

Last >>

→ line that has just executed

→ next line to execute

Frames

Global frame
Stack

Objects

Stack class
hide attributes

__init__	function __init__(self)
isEmpty	function isEmpty(self)
peek	function peek(self)
pop	function pop(self)
push	function push(self, item)
size	function size(self)

Balanced parentheses

- วงเล็บเปิดวงเล็บปิดถูกหรือไม่ (สมมติว่ามีแค่วงเล็บ)
- ตัวอย่างที่ถูก
 - (() () () ())
 - (((())))
 - (() ((()) ()))
- ตัวอย่างที่ผิด
 - ((((((
 -)))
 - ((())(

Implementation

```
1 from pythonds.basic.stack import Stack
2
3 def parChecker(symbolString):
4     s = Stack()
5     balanced = True
6     index = 0
7     while index < len(symbolString) and balanced:
8         symbol = symbolString[index]
9         if symbol == "(":
10            s.push(symbol) ← เจอ ( ยัดเข้า stack
11        else:
12            if s.isEmpty():
13                balanced = False ← stack ว่างเจอ ) ไม่ balanced แน่นนอน
14            else:
15                s.pop() ← เจอ ) pop ( ออก
16
17        index = index + 1
18
19    if balanced and s.isEmpty():
20        return True
21    else:
22        return False
```

Balanced symbols

- $[\{ () \}]$ ถูกหรือไม่
- ตัวอย่างที่ถูก
 - $\{ \{ ([] []) \} () \}$
 - $[[\{ \{ (()) \} \}]]$
 - $[] [] [] () \{ \}$
- ตัวอย่างที่ผิด
 - $([])$
 - $((()]))$
 - $[\{ ()]$

Implementation

```
1 def parChecker(symbolString):
2     s = Stack()
3     balanced = True
4     index = 0
5     while index < len(symbolString) and balanced:
6         symbol = symbolString[index]
7         if symbol in "([{":
8             s.push(symbol)
9         else:
10            if s.isEmpty():
11                balanced = False
12            else:
13                top = s.pop()
14                if not matches(top, symbol):
15                    balanced = False
16            index = index + 1
17    if balanced and s.isEmpty():
18        return True
19    else:
20        return False

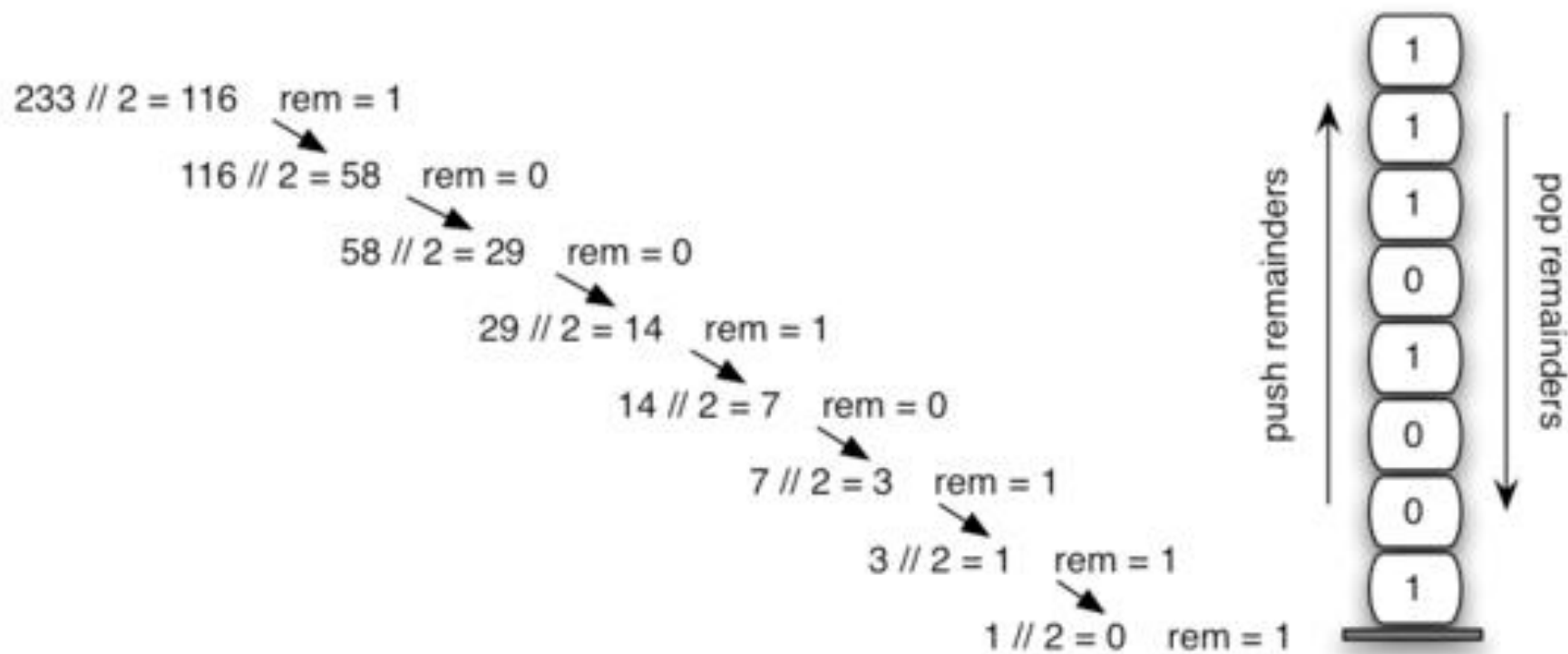
1 def matches(open, close):
2     opens = "([{"
3     closers = ")]}"
4     return opens.index(open) == closers.index(close)
```

แปลงเลขฐาน 2

- แปลงจากฐาน 10 ไปเป็นฐาน 2
- ตัวอย่างฐาน 10 แยกหลัก
 - 512 หาร 10 เศษ 2 ยัดลง stack
 - ผลหาร 51 หาร 10 เศษ 1 ยัดลง stack
 - ผลหาร 5 หาร 10 เศษ 5 ยัดลง stack
 - pop ออกมาจนหมด

5
1
2

แปลงเลขฐาน 2



Implementation

```
1 from pythonds.basic.stack import Stack
2
3 def divideBy2(decNumber):
4     remstack = Stack()
5
6     while decNumber > 0:
7         rem = decNumber % 2
8         remstack.push(rem)
9         decNumber = decNumber // 2
10
11     binString = ""
12     while not remstack.isEmpty():
13         binString = binString + str(remstack.pop())
14
15     return binString
16
17 print(divideBy2(42))
```

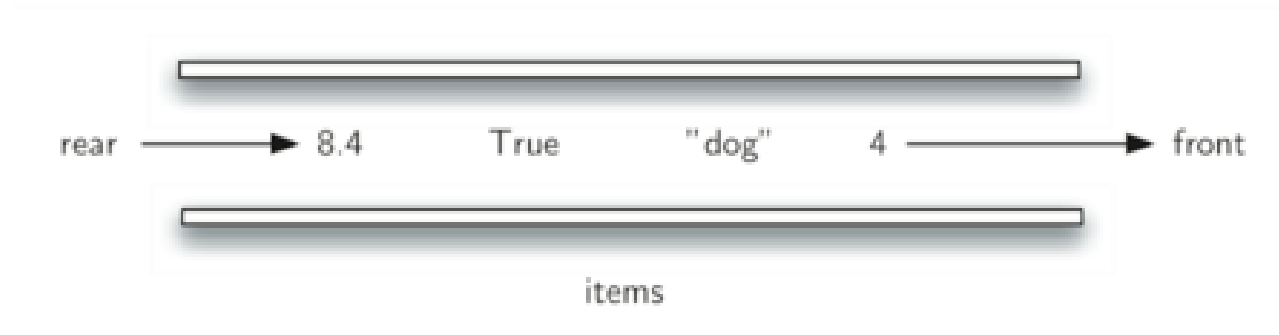
← หาร 2 ยัดเศษลง stack

แปลงเลขฐานอื่นๆ

- ลองทำดู
- เฉลย
 - <http://interactivepython.org/runestone/static/pythonds/BasicDS/ConvertingDecimalNumberstoBinaryNumbers.html>

Queue

- มีลำดับแบบ **first-in-first-out**
- ลำดับที่สำคัญคือ **rear** กับ **front**



Queue ADT

- Queue() สร้าง queue ว่างๆ และ return queue มาให้
- enqueue(item) เพิ่ม item และไม่ return
- dequeue() เอา item ที่อยู่บนหน้าออกมาและ return item นั้น
- isEmpty() return ค่าว่า queue ว่างหรือไม่
- size() return จำนวน item ใน queue

Queue ADT

Queue Operation	Queue Contents	Return Value
q.isEmpty()	[]	True
q.enqueue(4)	[4]	
q.enqueue('dog')	['dog',4]	
q.enqueue(True)	[True,'dog',4]	
q.size()	[True,'dog',4]	3
q.isEmpty()	[True,'dog',4]	False
q.enqueue(8.4)	[8.4,True,'dog',4]	
q.dequeue()	[8.4,True,'dog']	4
q.dequeue()	[8.4,True]	'dog'
q.size()	[8.4,True]	2

Queue's implementation

```
class Queue:
    def __init__(self):
        self.items = []

    def isEmpty(self):
        return self.items == []

    def enqueue(self, item):
        self.items.insert(0,item)

    def dequeue(self):
        return self.items.pop()

    def size(self):
        return len(self.items)
```

Queue in action

```
Python 2.7
6     return self.items == []
7
8     def enqueue(self, item):
9         self.items.insert(0,item)
10
11     def dequeue(self):
12         return self.items.pop()
13
14     def size(self):
15         return len(self.items)
16
17 q=Queue()
18
19 q.enqueue(4)
20 q.enqueue('dog')
21 q.enqueue(True)
22 print(q.size())
```

<< First

< Back

Step 13 of 21

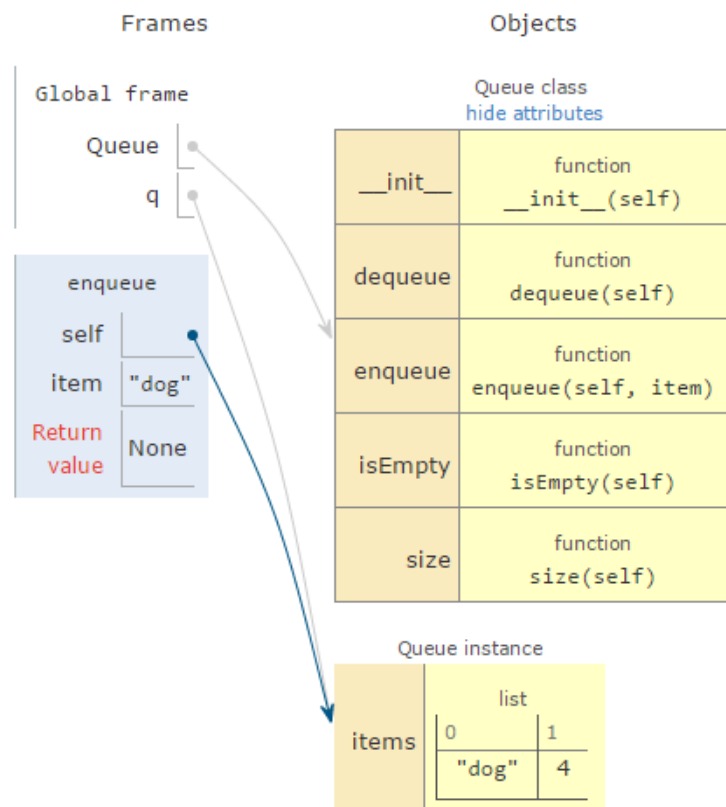
Forward >

Last >>

→ line that has just executed

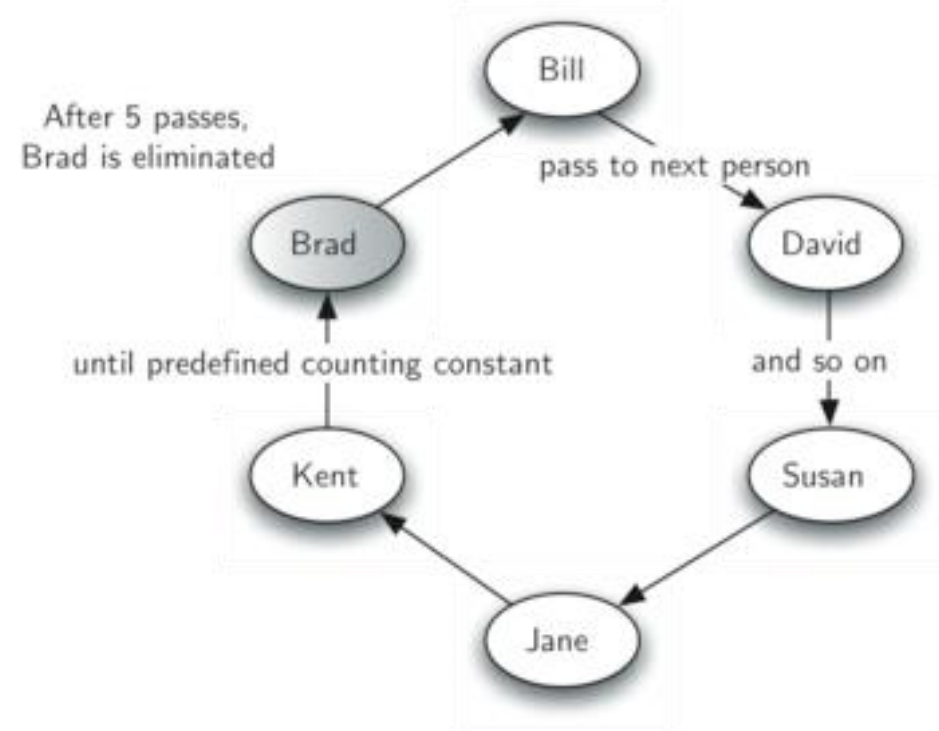
→ next line to execute

Visualized using Online Python Tutor by Philip Guo



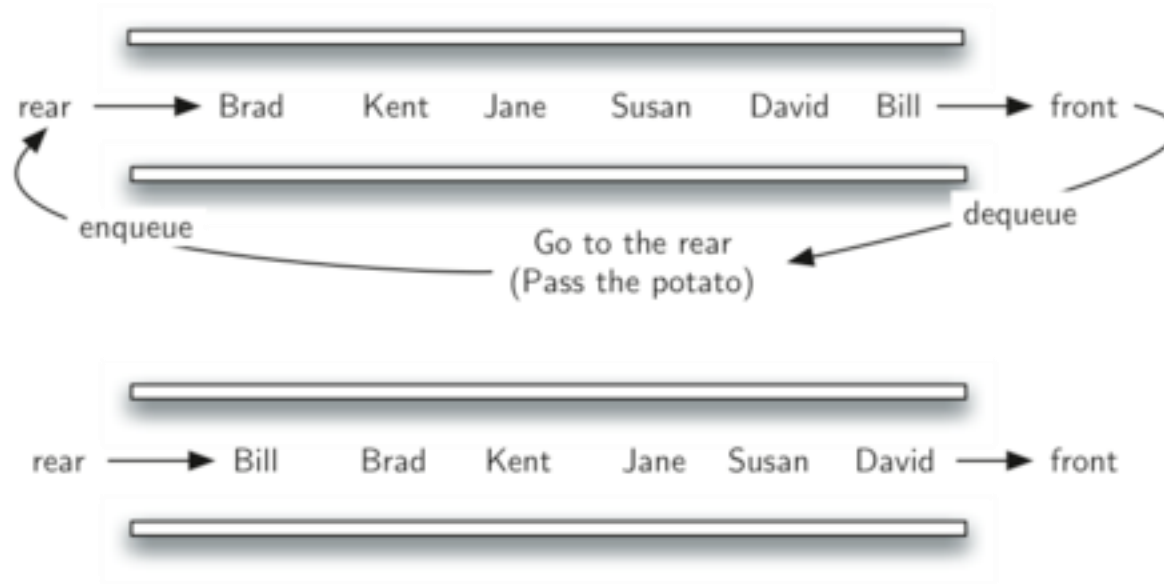
Hot potato

- ส่ง **potato** ไปให้คนข้างๆ หยุดที่ใครคนนั้นออกจากเกม คนสุดท้ายชนะ
- เล่นแบบง่ายๆ ส่งเป็นจำนวนครั้งแล้วหยุด
 - ในตัวอย่างนี้ส่ง 5 ครั้ง



คิดแบบ queue

- คนที่อยู่หัว queue คือคนที่ถือ potato
- เมื่อส่ง potato ให้คนข้างๆ ก็ไปต่อท้าย queue



Implementation

```
1 from pythonds.basic.queue import Queue
2
3 def hotPotato(namelist, num):
4     simqueue = Queue()
5     for name in namelist:
6         simqueue.enqueue(name)
7
8     while simqueue.size() > 1:
9         for i in range(num):
10             simqueue.enqueue(simqueue.dequeue())
11
12         simqueue.dequeue()
13
14     return simqueue.dequeue()
15
16 print(hotPotato(["Bill", "David", "Susan", "Jane", "Kent", "Brad"], 7))
17
```

เริ่มใส่ชื่อคนเล่น

ส่งเป็นจำนวนครั้ง คนที่ส่งแล้วไปต่อท้าย

ครบจำนวนใครอยู่หน้า queue (ถือ potato) ถูกกำจัด

เหลือคนสุดท้ายคือผู้ชนะ

Printing task

- จำลองสถานการณ์เพื่อคำนวณเวลารอในการพิมพ์
- ความสามารถของ **printer** คือ จำนวนหน้าต่อนาที (ppm)
- ความน่าจะเป็นของการเกิด **task = 1** งานใน **180** วินาที
 - สุ่ม **1** ใน **180** ทุกๆ **1** วินาที (tick)
- จำนวนหน้าในแต่ละงาน **1 – 20** หน้าโอกาสเท่ากันหมด

Printer class

- Variables
 - ppm
 - Task ปัจจุบัน
 - เวลาที่เหลือในการทำ **task** ปัจจุบัน
- Methods
 - ตรวจสอบการทำงานทุกๆ 1 tick
 - Printer ทำงานอยู่หรือไม่ (busy)
 - เริ่ม **task** ใหม่

Task class

- Variables

- เวลาที่ส่งงานเข้า queue
- จำนวนหน้า

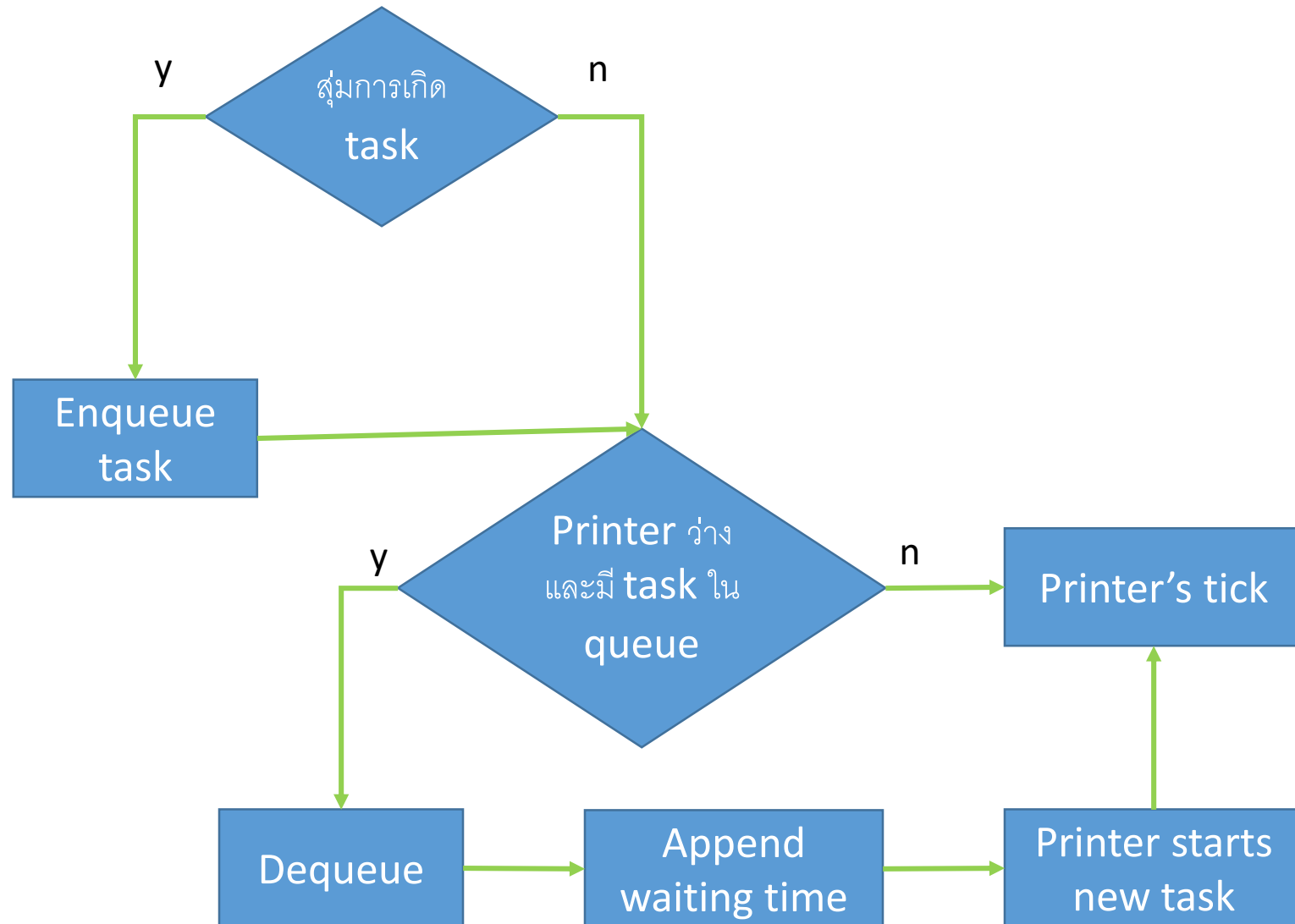
- Method

- คำนวณเวลาที่รอ queue

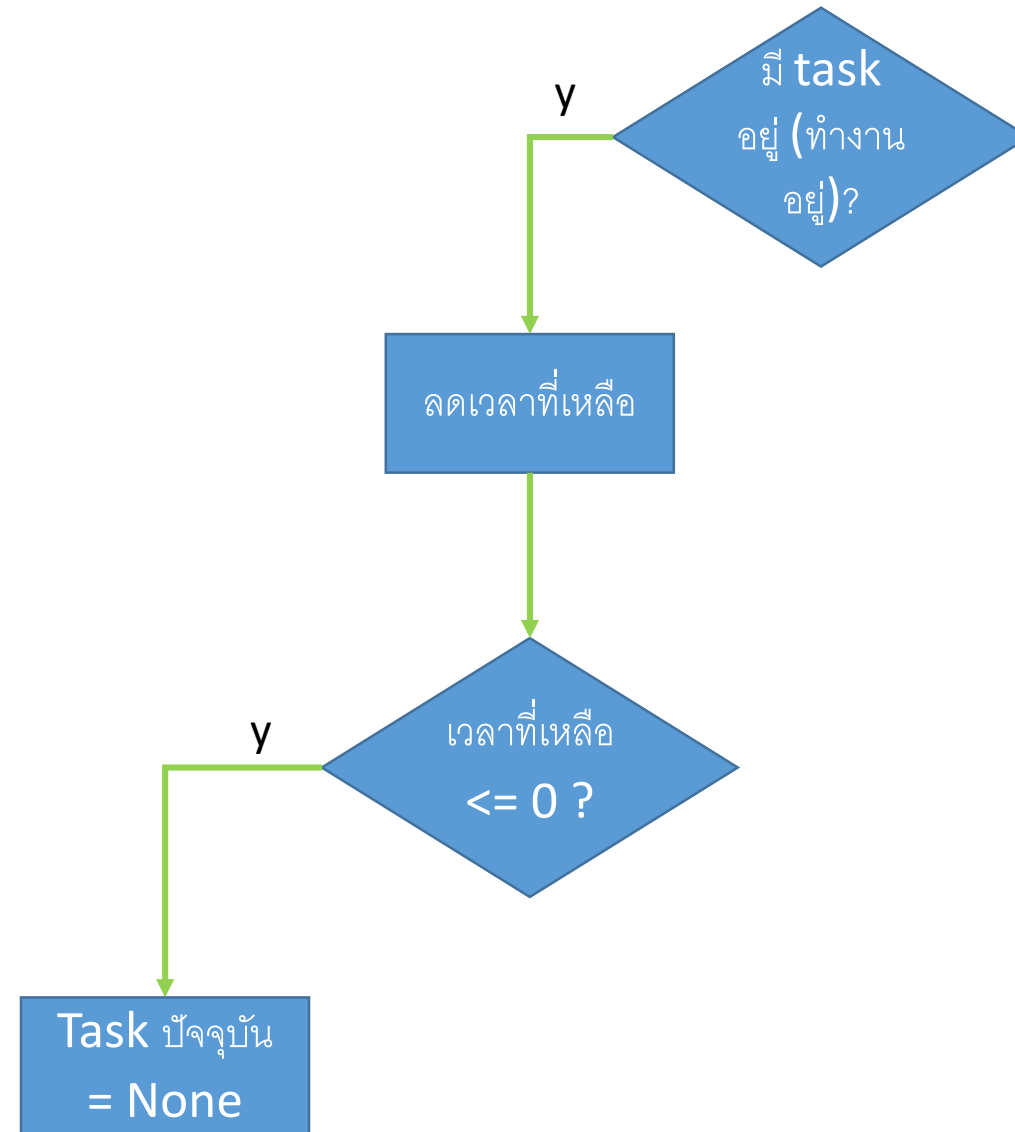
Random

- สุ่มตัวเลข
- `random.randrange(n, m)`
 - สุ่มจำนวนเต็มระหว่าง n ถึง $m - 1$

Simulation



Printer's tick



Printer class implementation

```
1  class Printer:
2      def __init__(self, ppm):
3          self.pagerate = ppm
4          self.currentTask = None
5          self.timeRemaining = 0
6
7      def tick(self):
8          if self.currentTask != None:
9              self.timeRemaining = self.timeRemaining - 1
10             if self.timeRemaining <= 0:
11                 self.currentTask = None
12
13     def busy(self):
14         if self.currentTask != None:
15             return True
16         else:
17             return False
18
19     def startNext(self, newtask):
20         self.currentTask = newtask
21         self.timeRemaining = newtask.getPages() * 60/self.pagerate
```

Task class implementation

```
import random

class Task:
    def __init__(self, time):
        self.timestamp = time
        self.pages = random.randrange(1, 21)

    def getStamp(self):
        return self.timestamp

    def getPages(self):
        return self.pages

    def waitTime(self, currenttime):
        return currenttime - self.timestamp
```

Simulation และ สุ่มการเกิด task

```
from pythonds.basic.queue import Queue

import random

def simulation(numSeconds, pagesPerMinute):

    labprinter = Printer(pagesPerMinute)
    printQueue = Queue()
    waitingtimes = []

    for currentSecond in range(numSeconds):

        if newPrintTask():
            task = Task(currentSecond)
            printQueue.enqueue(task)

        if (not labprinter.busy()) and (not printQueue.isEmpty()):
            nexttask = printQueue.dequeue()
            waitingtimes.append(nexttask.waitTime(currentSecond))
            labprinter.startNext(nexttask)

        labprinter.tick()

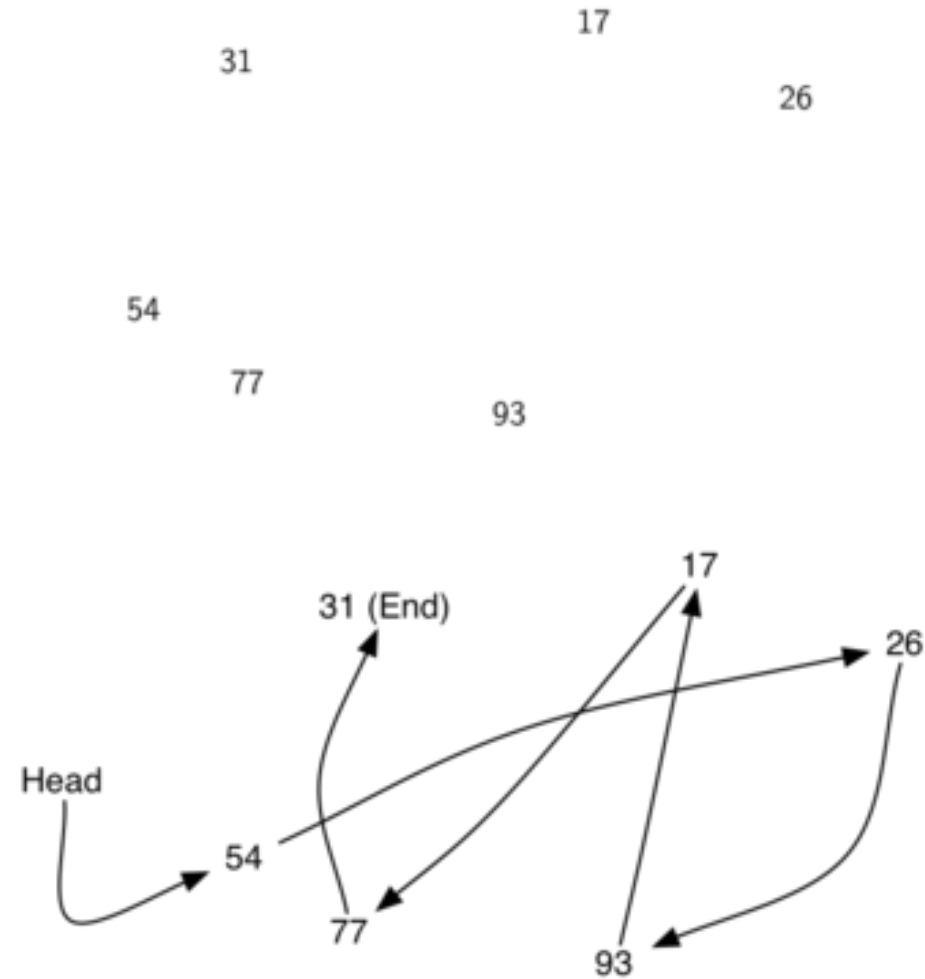
    averageWait=sum(waitingtimes)/len(waitingtimes)
    print("Average Wait %.2f secs %3d tasks remaining."%(averageWait,printQueue.size()))

def newPrintTask():
    num = random.randrange(1,181)
    if num == 180:
        return True
    else:
        return False
```

Linked List

List

- Python list
- รายการแบบโยง (linked list)
 - แบบไม่มีลำดับ (unordered list)
 - แบบมีลำดับ (ordered list)
 - น้อยไปมาก
 - มากไปน้อย

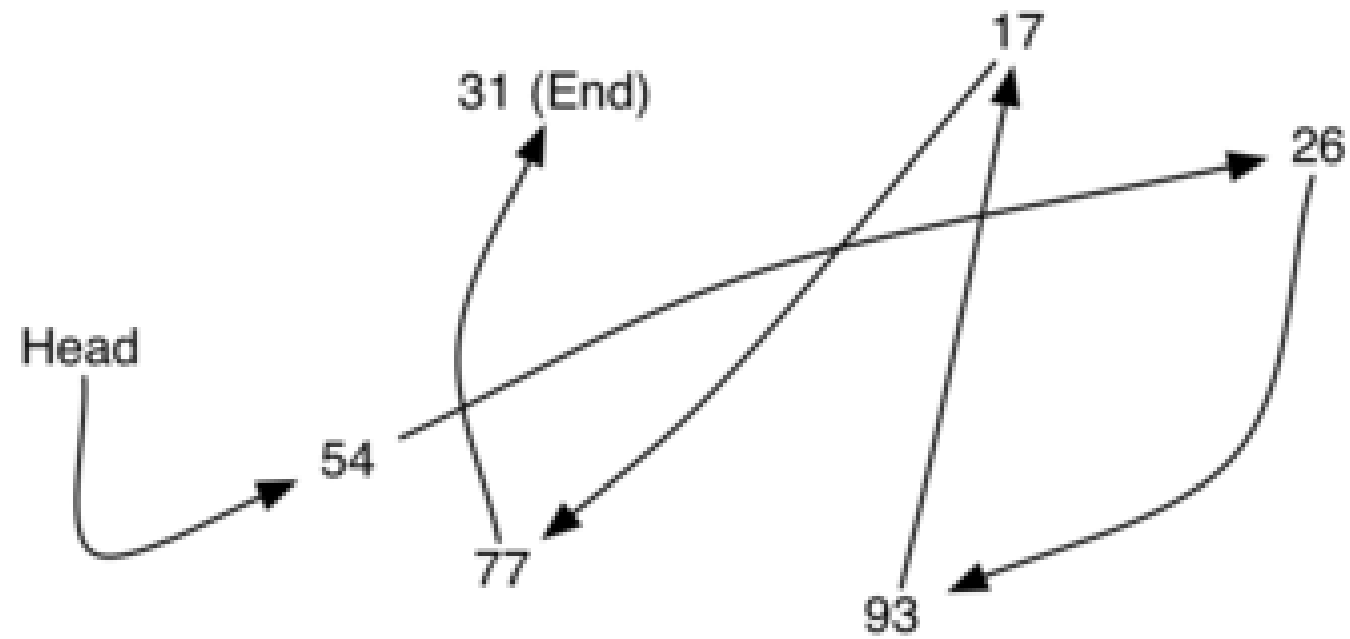


แบบไม่มีลำดับ

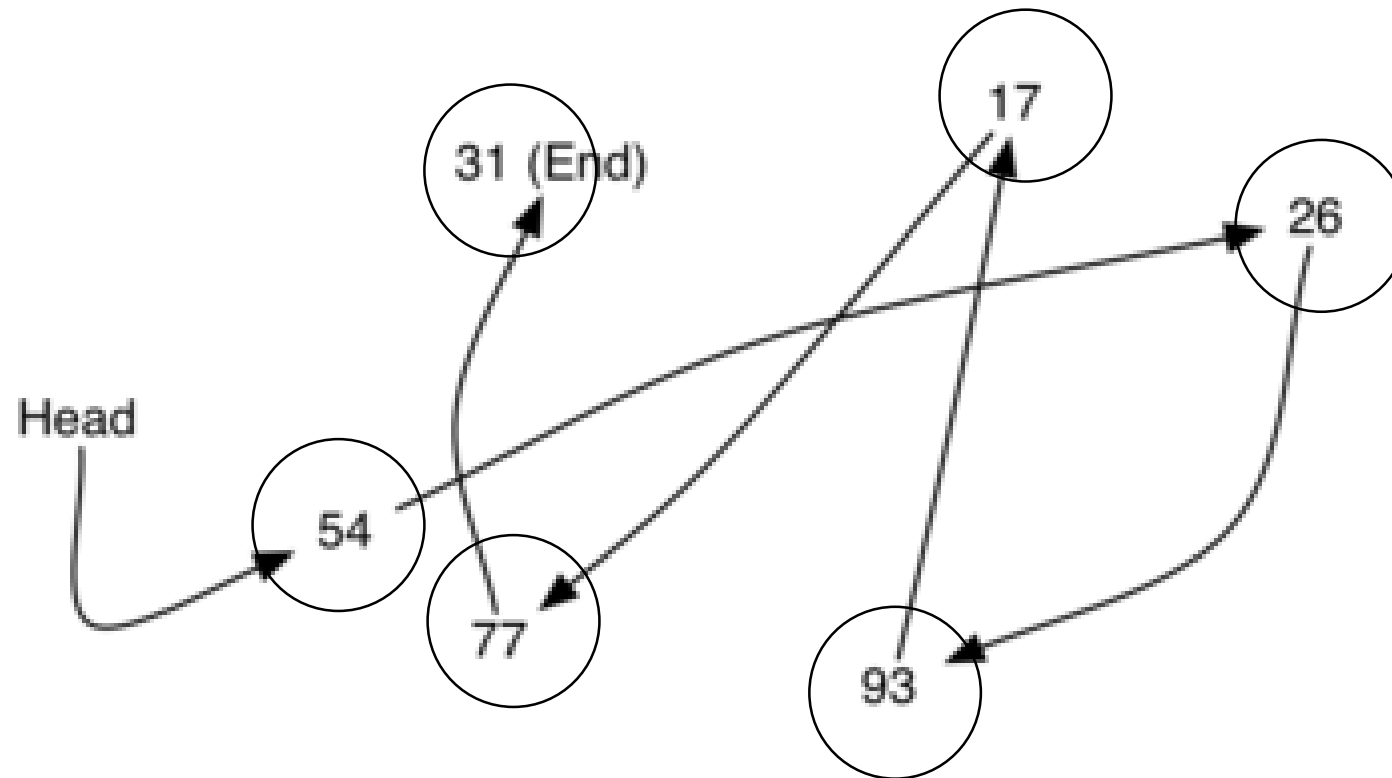
ADT ของ UnorderedList

- `UnorderedList()` ทำการสร้าง `UnorderedList` ใหม่แล้วคืนค่าเป็น `UnorderedList` ว่าง
- `add(item)` รับ parameter คือ `item` เพื่อเพิ่มเข้าใน `UnorderedList`
- `remove(item)` ค้นหา `item` แล้วเอาออกจาก `UnorderedList`
- `search(item)` ค้นหา `item` แล้วคืนค่าว่าเจอหรือไม่ เป็น `True` กับ `False`
- `isEmpty()` ตรวจสอบแล้วคืนค่าว่างหรือไม่
- `size()` คืนค่าขนาดของ `UnorderedList`
- `append(item)` เพิ่ม `item` โดยต่อท้ายใน `UnorderedList`
- `index(item)` หาดัชนีของ `item` ใน `UnorderedList`
- `insert(pos, item)` เพิ่ม `item` ณ ดัชนีที่ `pos`
- `pop()` นำข้อมูลตัวสุดท้ายออกจาก `UnorderedList` และคืนค่านั้นด้วย
- `pop(pos)` นำข้อมูล ณ ดัชนีที่ `pos` ออกจาก `UnorderedList` และคืนค่านั้นด้วย

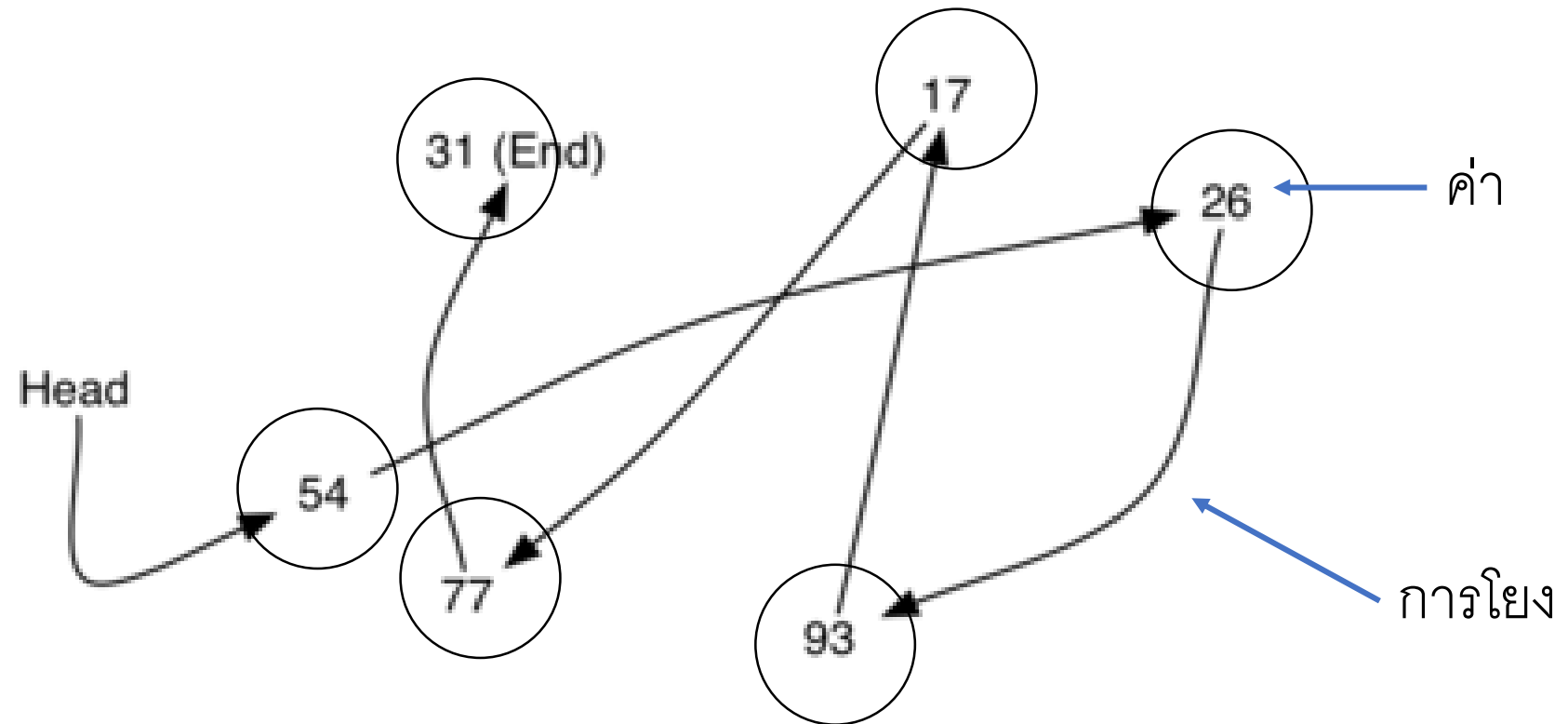
Node



Node



Node



```
class Node:
```

```
    def __init__(self,initdata):
```

```
        self.data = initdata
```

```
        self.next = None
```

```
    def getData(self):
```

```
        return self.data
```

```
    def getNext(self):
```

```
        return self.next
```

```
    def setData(self,newdata):
```

```
        self.data = newdata
```

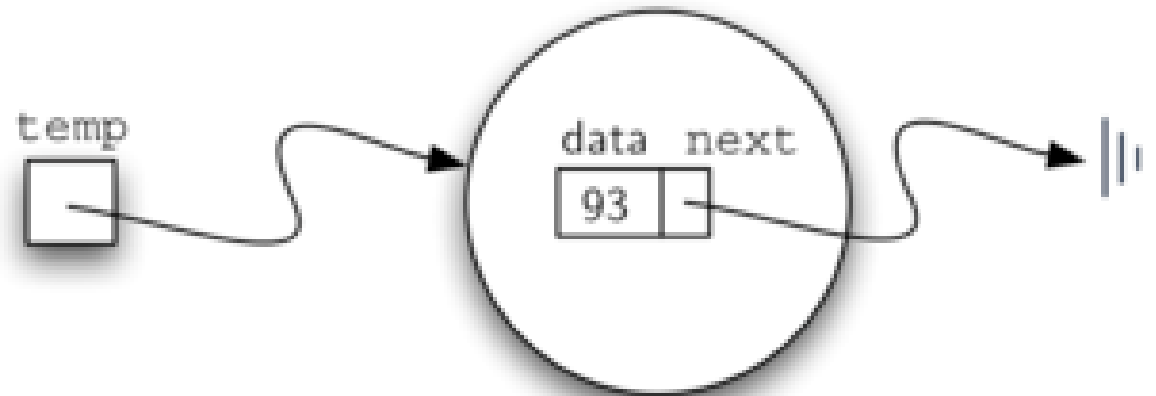
```
    def setNext(self,newnext):
```

```
        self.next = newnext
```

```
>>> temp = Node(93)
```

```
>>> temp.getData()
```

```
93
```



Class ของ UnorderedList

```
class UnorderedList:
```

```
    def __init__(self):  
        self.head = None
```

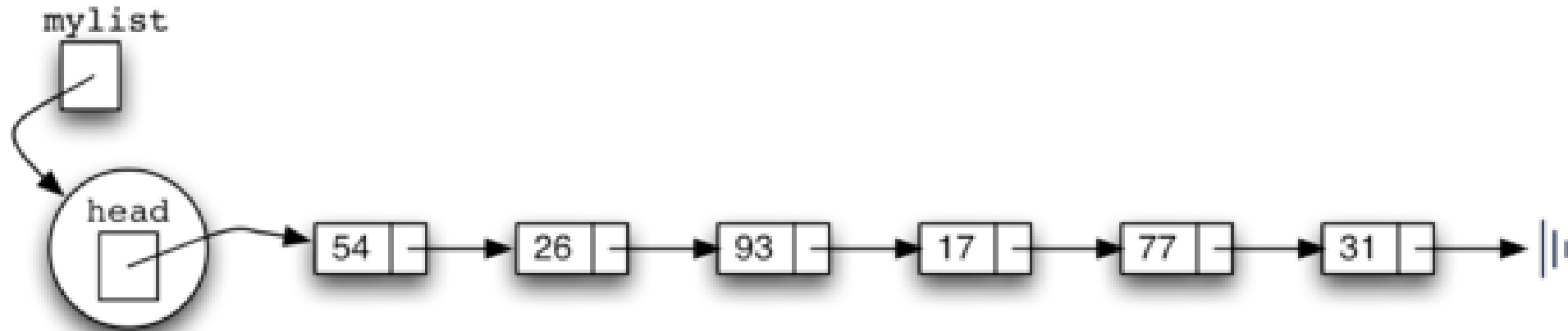
```
    def isEmpty(self):  
        return self.head == None
```

```
>>> mylist = UnorderedList()
```



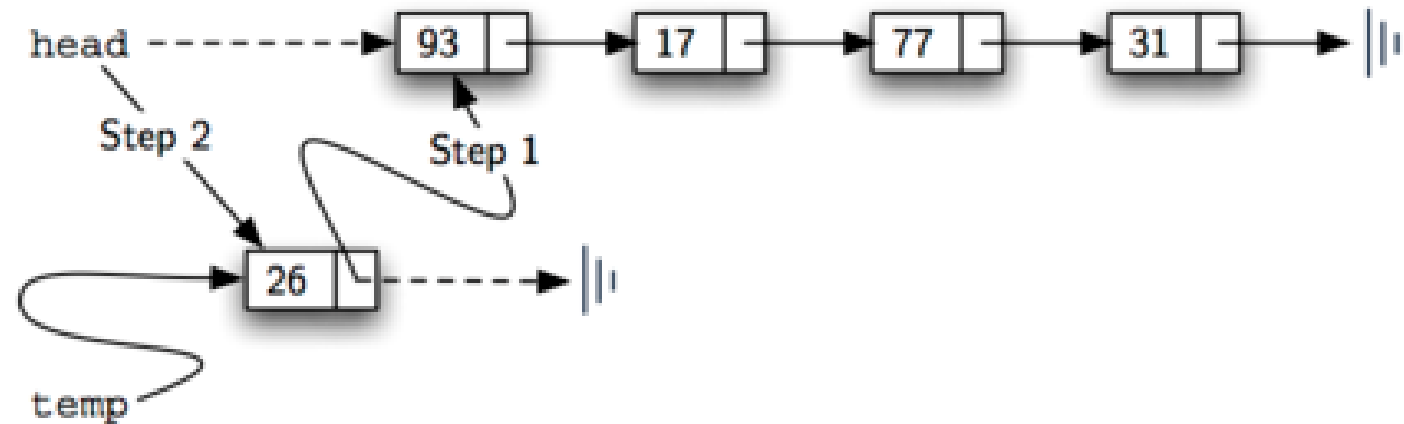
add

```
>>> mylist.add(31)
>>> mylist.add(77)
>>> mylist.add(17)
>>> mylist.add(93)
>>> mylist.add(26)
>>> mylist.add(54)
```



```
def add(self, item):  
    temp = Node(item)  
    temp.setNext(self.head)  
    self.head = temp
```

เพิ่มที่ส่วนหัวของ list

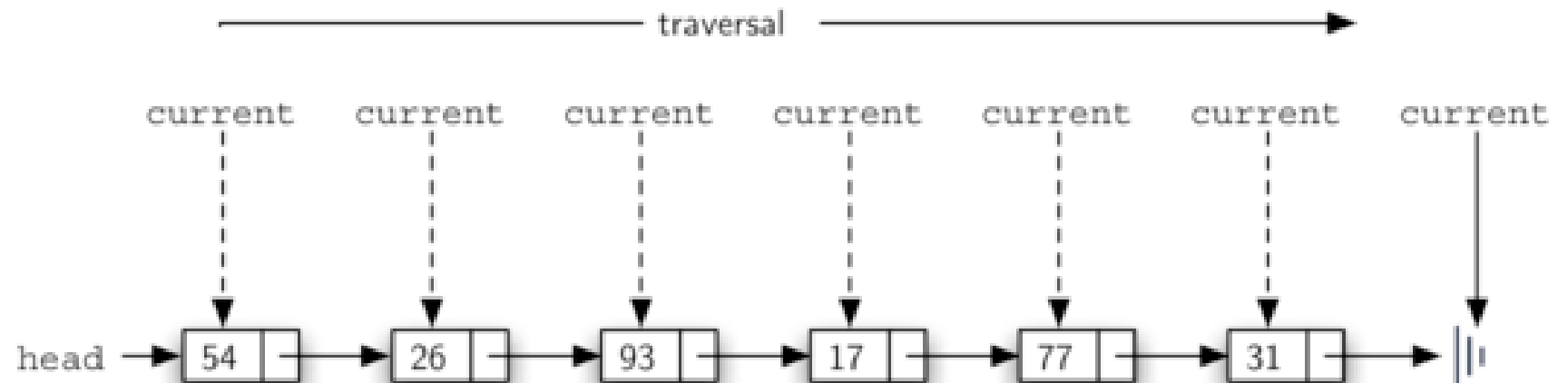


size

```
def size(self):  
    current = self.head  
    count = 0  
    while current != None:  
        count = count + 1  
        current = current.getNext()
```

วิ่งทั่ว list เพื่อนับ

```
return count
```



search

- วิ่งทั่ว **list** จนกว่า
 - เจอ $\rightarrow$ **True**
 - หมด **list** แล้วไม่เจอ $\rightarrow$ **False**
- ถูกนำไปดัดแปลงใน **remove**

```
def search(self,item):
```

```
    current = self.head
```

```
    found = False
```

```
    while current != None and not found:
```

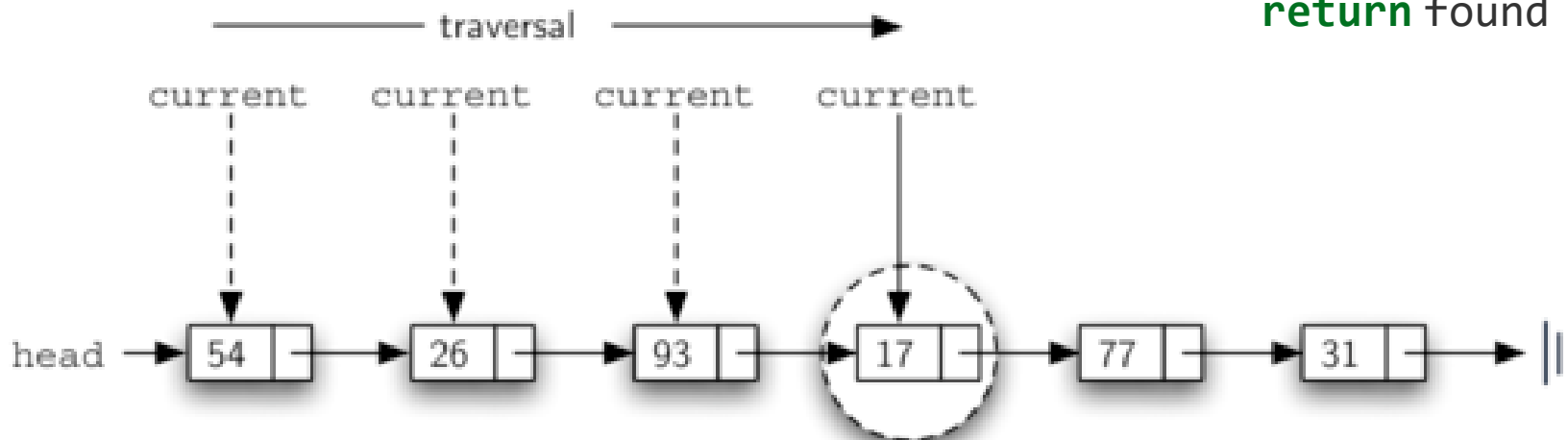
```
        if current.getData() == item:
```

```
            found = True
```

```
        else:
```

```
            current = current.getNext()
```

```
    return found
```



remove

```
def remove(self, item):
```

```
    current = self.head
```

```
    previous = None
```

```
    found = False
```

```
    while not found:
```

```
        if current.getData() == item:
```

```
            found = True
```

```
        else:
```

```
            previous = current
```

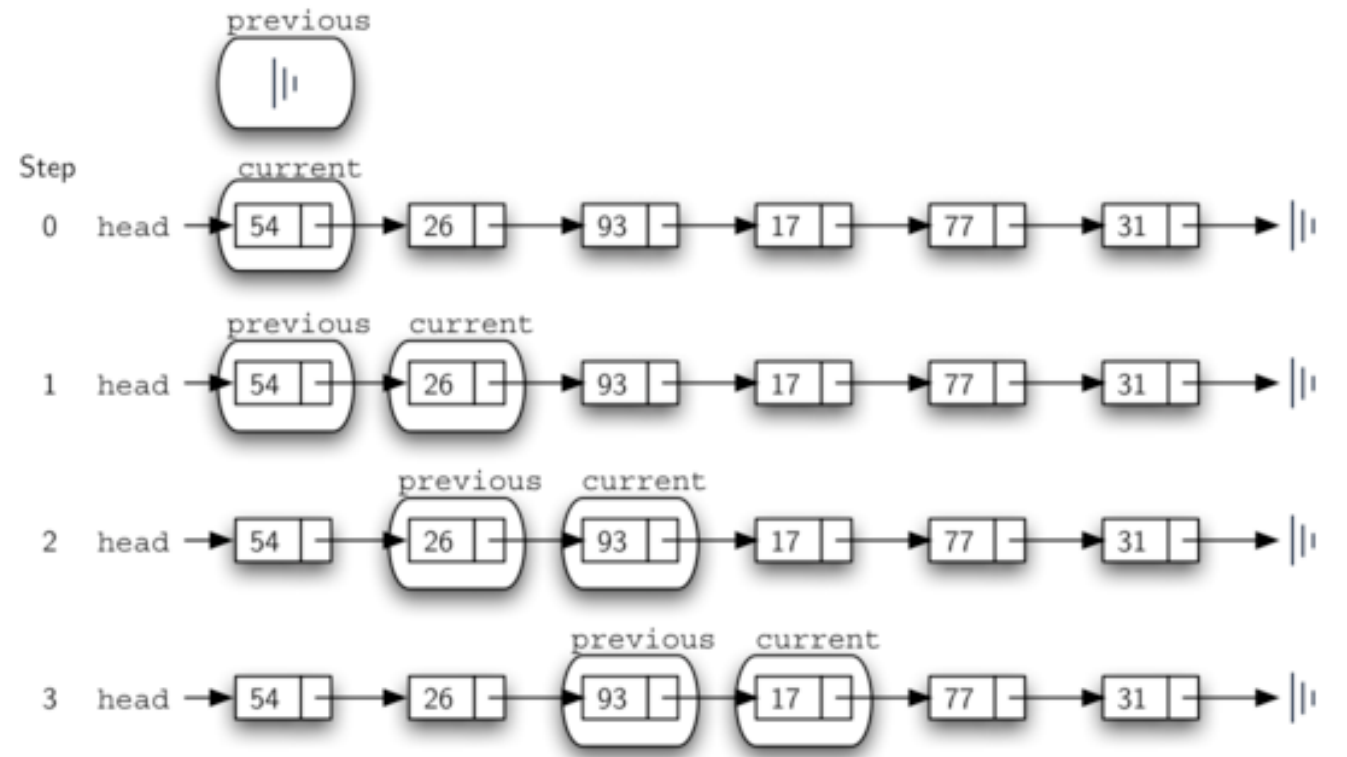
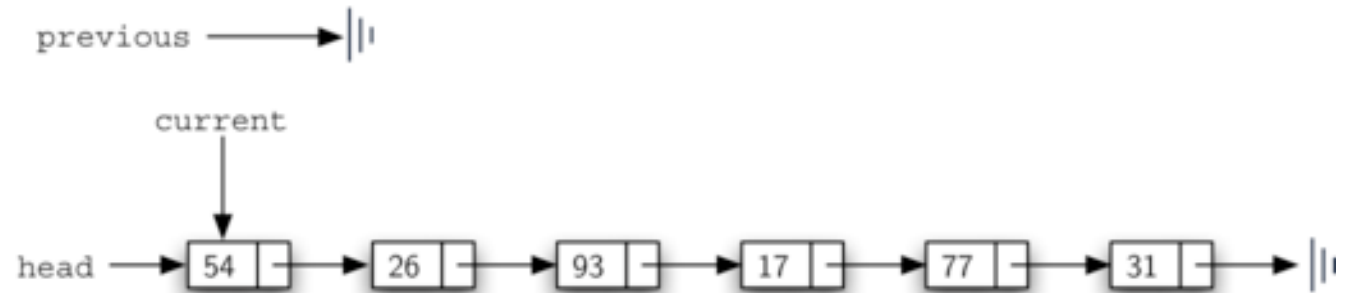
```
            current = current.getNext()
```

```
    if previous == None:
```

```
        self.head = current.getNext()
```

```
    else:
```

```
        previous.setNext(current.getNext())
```



remove

```
def remove(self, item):
```

```
    current = self.head
```

```
    previous = None
```

```
    found = False
```

```
    while not found:
```

```
        if current.getData() == item:
```

```
            found = True
```

```
        else:
```

```
            previous = current
```

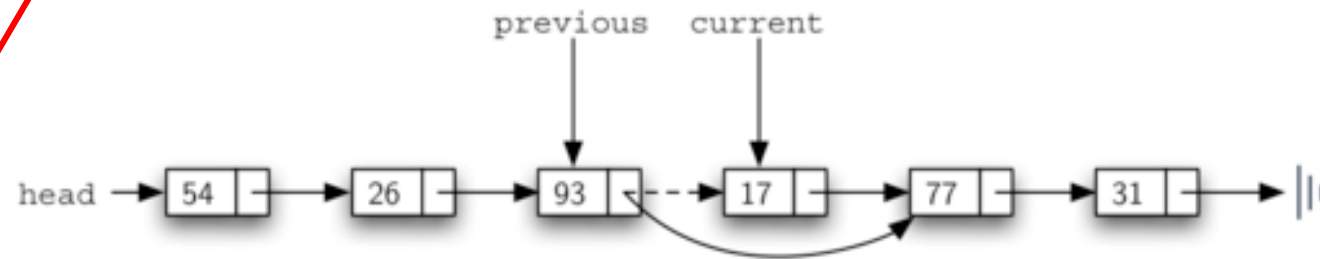
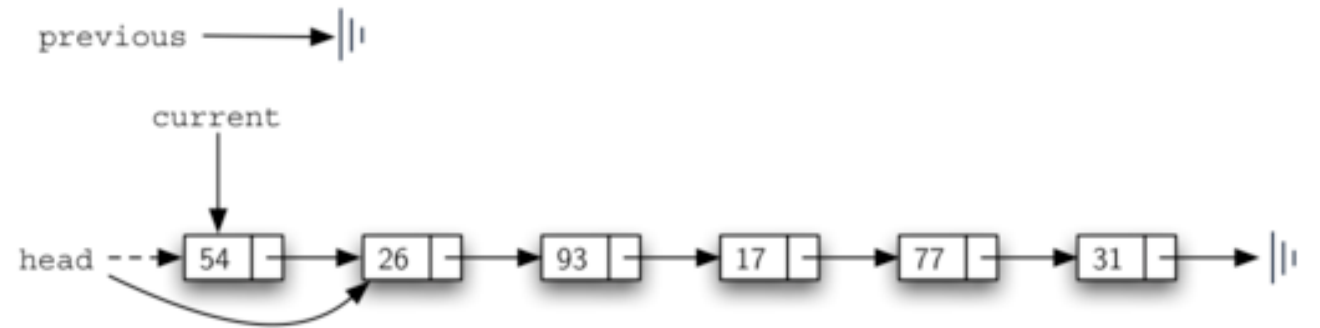
```
            current = current.getNext()
```

```
    if previous == None:
```

```
        self.head = current.getNext()
```

```
    else:
```

```
        previous.setNext(current.getNext())
```



ADT ของ OrderedList

- `OrderedList()` ทำการสร้าง `OrderedList` ใหม่แล้วคืนค่าเป็น `OrderedList` ว่าง
- `add(item)` รับ parameter คือ `item` เพื่อเพิ่มเข้าใน `OrderedList`
- `remove(item)` ค้นหา `item` แล้วเอาออกจาก `OrderedList`
- `search(item)` ค้นหา `item` แล้วคืนค่าว่าเจอหรือไม่ เป็น `True` กับ `False`
- `isEmpty()` ตรวจสอบแล้วคืนค่าว่างหรือไม่
- `size()` คืนค่าขนาดของ `OrderedList`
- `index(item)` หาดัชนีของ `item` ใน `OrderedList`
- `pop()` นำข้อมูลตัวสุดท้ายออกจาก `OrderedList` และคืนค่านั้นด้วย
- `pop(pos)` นำข้อมูล ณ ดัชนีที่ `pos` ออกจาก `OrderedList` และคืนค่านั้นด้วย

ความสับสนเวียนเกิด

Recursion

ความสัมพันธ์เวียนเกิดคืออะไร?

- วิธีการแก้ปัญหด้วยการแตกเป็นปัญหาย่อยๆ จนถึงส่วนที่ย่อยที่สุด
- มักจะเป็นฟังก์ชันที่เรียกตัวเอง

เรียนเพื่ออะไร?

- มองปัญหาที่ซับซ้อนให้ง่ายลงด้วยความสัมพันธ์เวียนเกิด
- ตีปัญหาให้อยู่ในรูปความสัมพันธ์เวียนเกิด
- เข้าใจและเขียนโปรแกรมที่อยู่ในรูปความสัมพันธ์เวียนเกิดได้

ปัญหาการหาผลบวกของ list

- ใช้วิธี loop

```
1 def listsum(numList):  
2     theSum = 0  
3     for i in numList:  
4         theSum = theSum + i  
5     return theSum  
6  
7 print(listsum([1, 3, 5, 7, 9]))  
8
```

$$0 + 1 = 1$$

$$1 + 3 = 4$$

$$4 + 5 = 9$$

$$9 + 7 = 16$$

$$16 + 9 = 25$$

$$((((1+3)+5)+7)+9)$$

จัดรูปใหม่

$$(1+(3+(5+(7+9))))$$

$$\text{total} = (1+(3+(5+(7+9))))$$

$$(3+(5+(7+9)))$$

$$\text{total} = (1+(3+(5+16)))$$

$$(5+(7+9))$$

$$\text{total} = (1+(3+21))$$

$$(7+9)$$

$$\text{total} = (1+24)$$

$$9$$

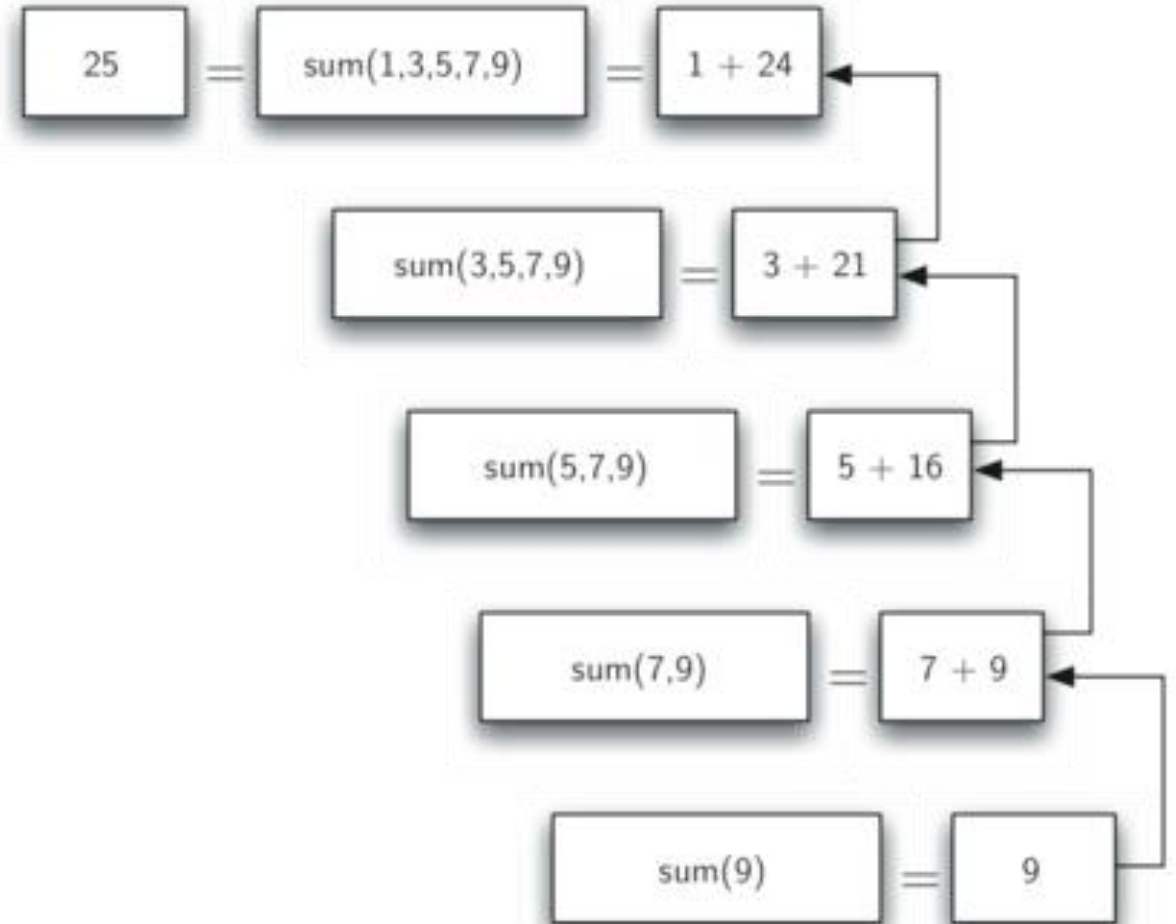
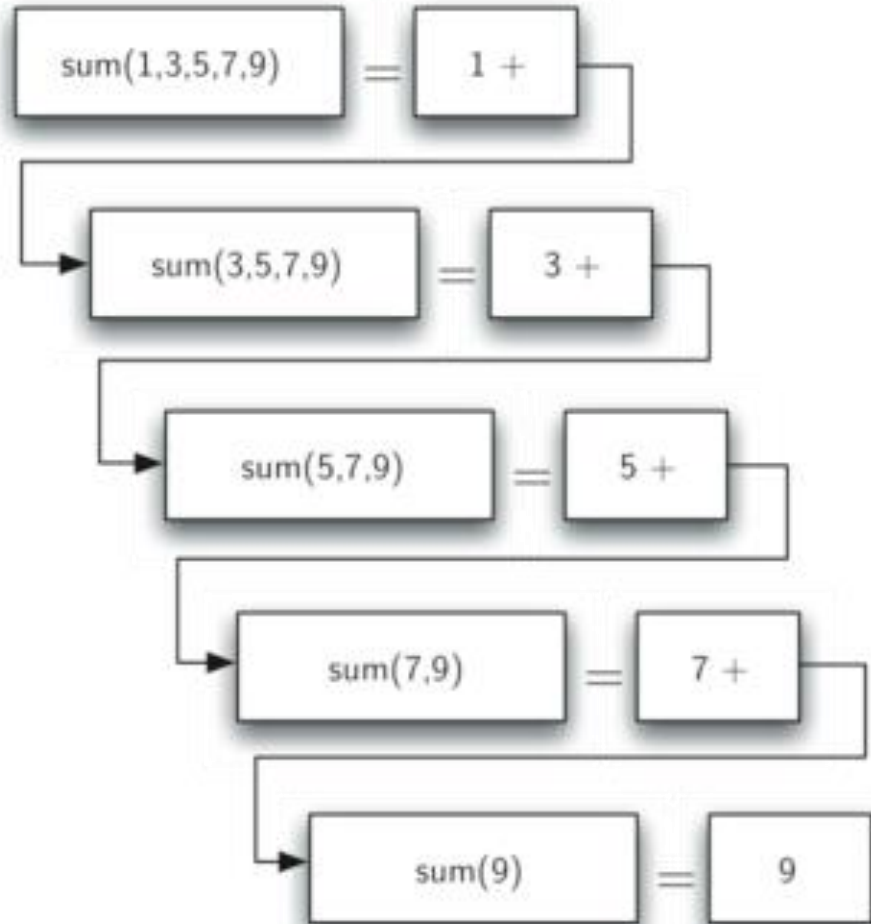
$$\text{total} = 25$$

แปลงเป็นโปรแกรม

- `listSum(numList) = first(numList) + listSum(rest(numList))`

```
1 def listsum(numList):  
2     if len(numList) == 1:  
3         return numList[0]  
4     else:  
5         return numList[0] + listsum(numList[1:])  
6  
7 print(listsum([1, 3, 5, 7, 9]))  
8
```


การทำงาน



กฎของความสัมพันธ์เวียนเกิด

- ต้องมีกรณีฐาน
- ต้องเปลี่ยนสถานะเข้าสู่กรณีพื้นฐาน
- ต้องเรียกตัวเอง

```
1 def listsum(numList):  
2     if len(numList) == 1:  
3         return numList[0]  
4     else:  
5         return numList[0] + listsum(numList[1:])  
6  
7 print(listsum([1, 3, 5, 7, 9]))  
8
```

กรณีฐาน

เปลี่ยนสถานะ

เรียกตัวเอง

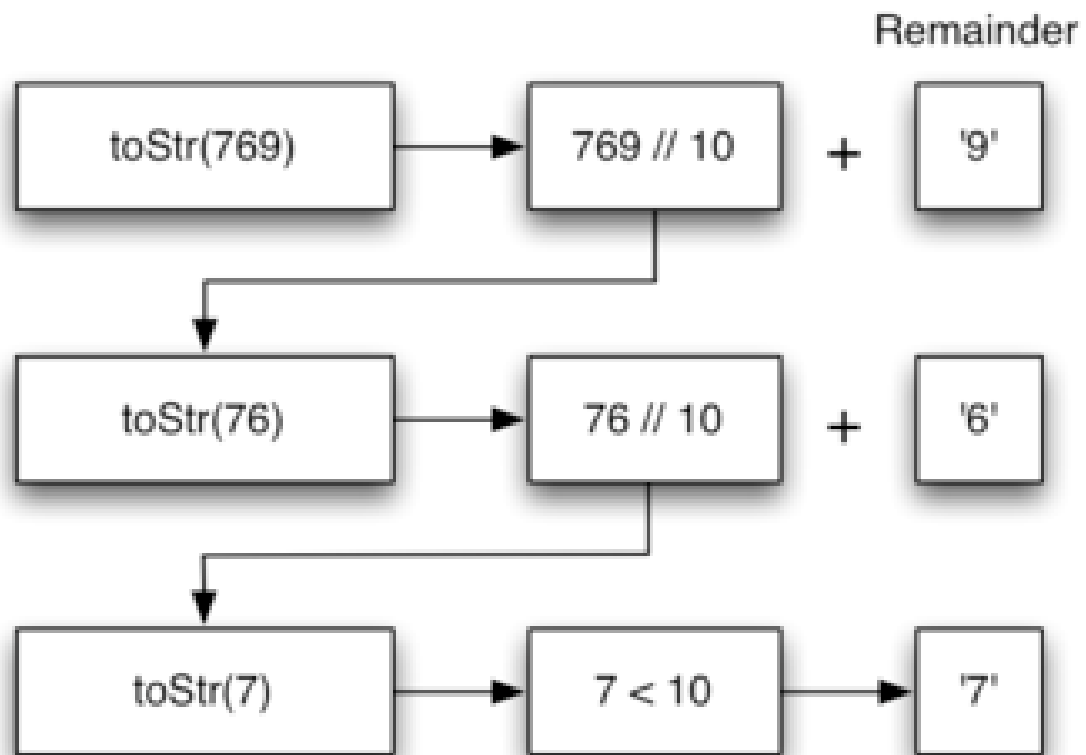
แปลงเลขฐาน (เก็บเป็น string)

- ฐานเลือกได้ 2 – 16
- 10 ฐาน 10 $\rightarrow$ “1010” ฐาน 2
- 769 ฐาน 10 $\rightarrow$ “769” ฐาน 10
- แปลงเลข 1 หลักเป็น string ใช้ array = “0123456789ABCDEF”
- วิธีคิด : หารแบบเลขจำนวนเต็มด้วยฐานแบบเอาผลหาร

เอาเศษแปลงเป็น **string** เก็บไว้ เอาผลหารมาหารด้วยฐานต่อไปเรื่อยๆ
หยุดเมื่อเศษน้อยกว่าฐาน

ตัวอย่าง

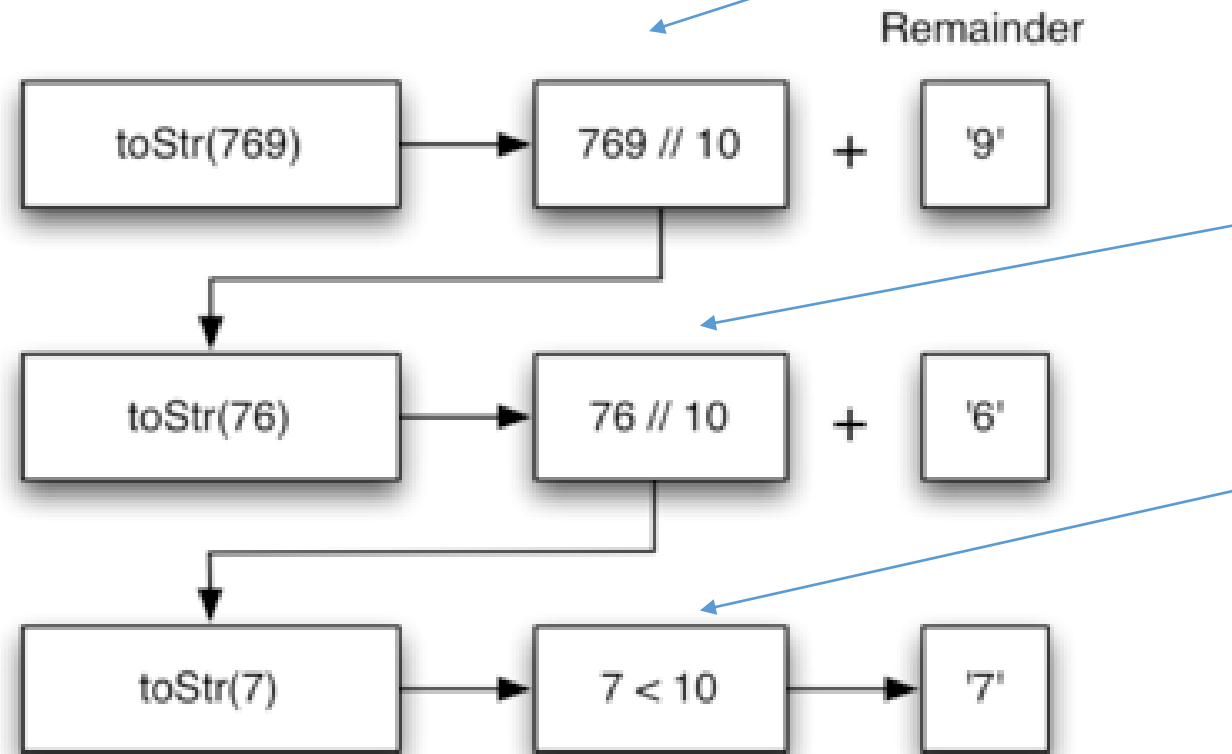
- $769_{\text{ฐาน } 10} \rightarrow "769"_{\text{ฐาน } 10}$



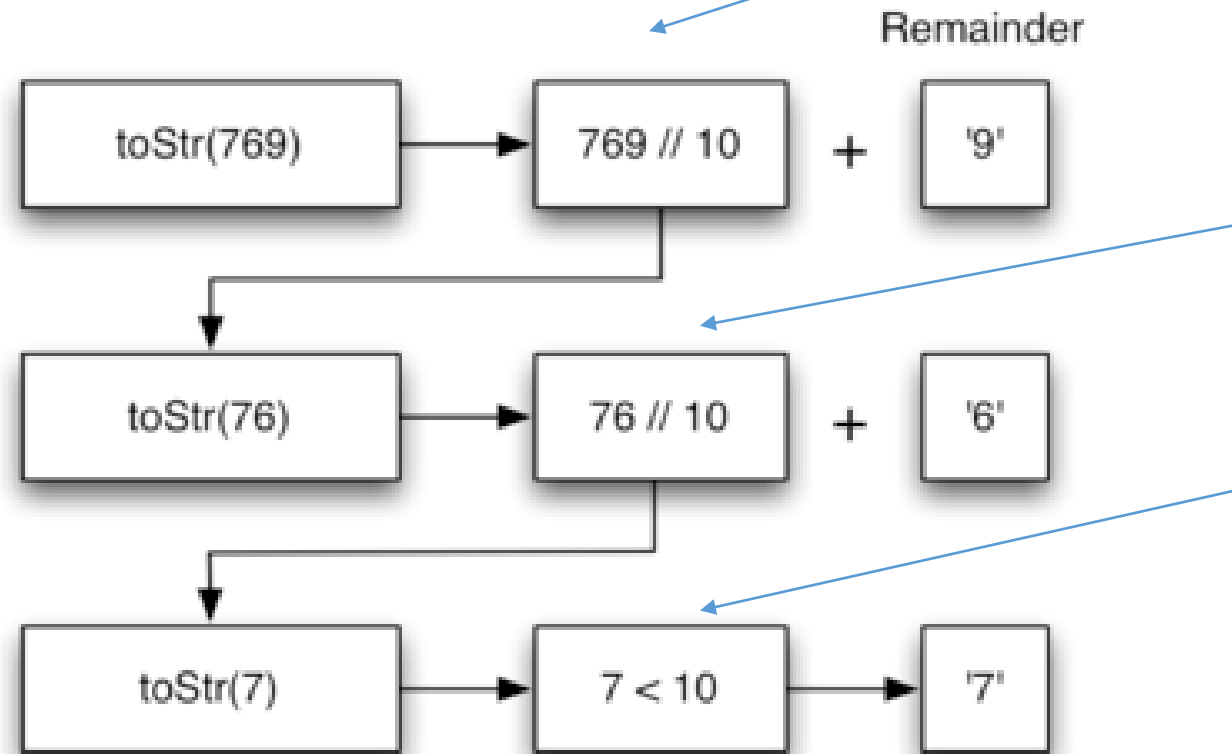
กรณีพื้นฐาน

- สถาน 10
 - 11 หารต่อหรือไม่?
 - 10 หารต่อหรือไม่?
 - 9 หารต่อหรือไม่?
- สถาน n
 - ตัวเศษ $> n$ หารต่อหรือไม่?
 - ตัวเศษ $= n$ หารต่อหรือไม่?
 - ตัวเศษ $< n$ หารต่อหรือไม่?

การเปลี่ยนสถานะ



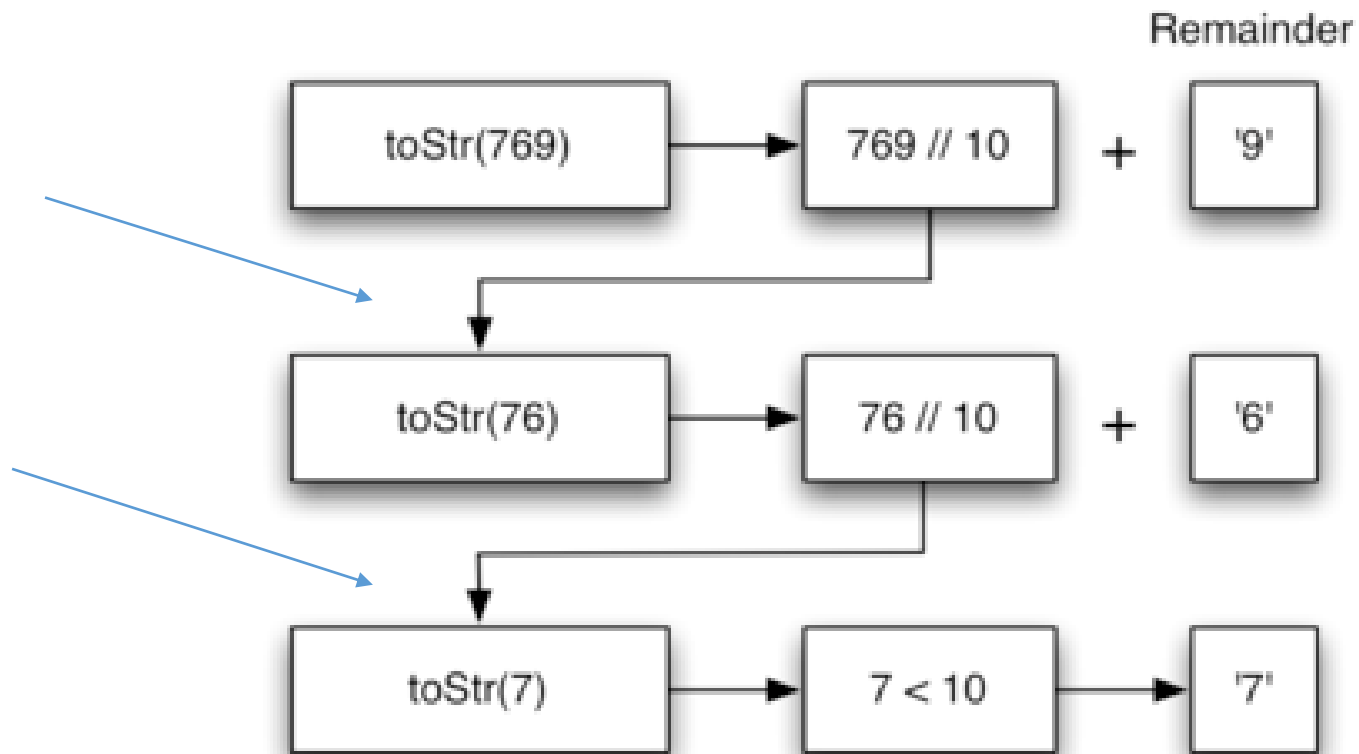
การเปลี่ยนสถานะ



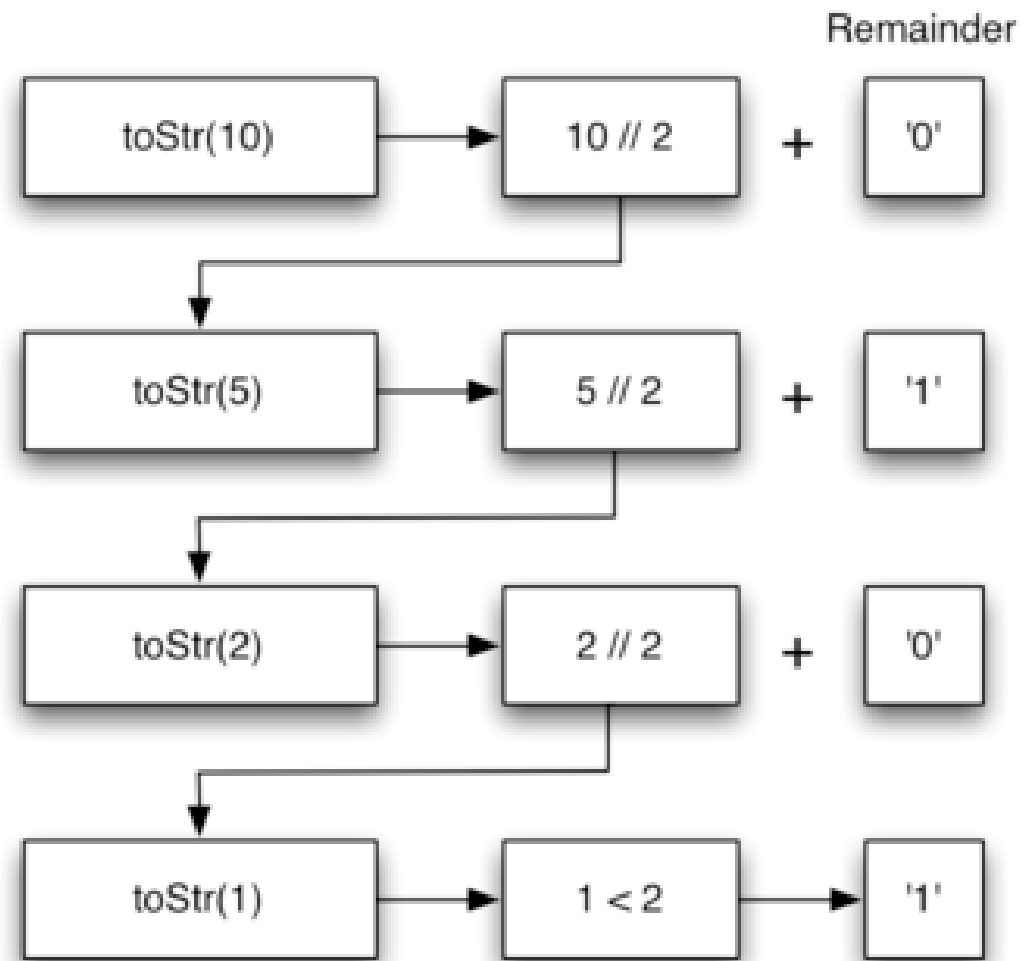
ตัวเลข
ลดลง
เรื่อยๆ

เรียกตัวเอง

เรียกหา
ผลหา



ตัวอย่างฐาน 2



ตัวอย่างโปรแกรม

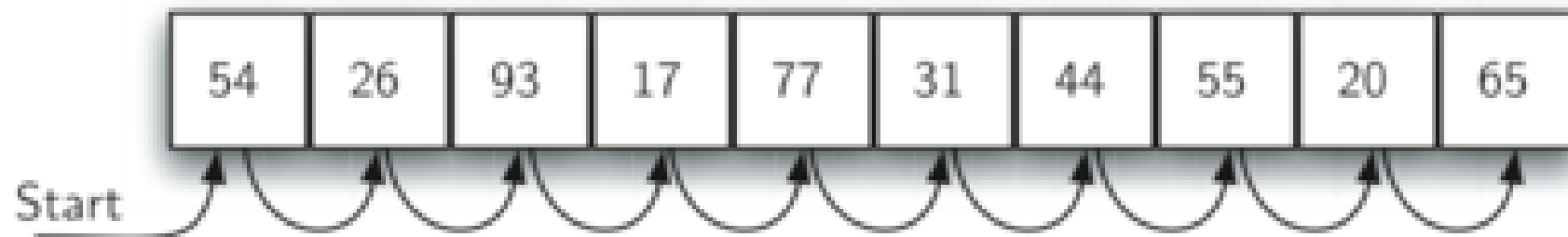
```
1 def toStr(n,base):  
2     convertString = "0123456789ABCDEF"  
3     if n < base:  
4         return convertString[n]  
5     else:  
6         return toStr(n//base,base) + convertString[n%base]  
7  
8 print(toStr(1453,16))  
9
```

โจทย์

- Reverse string
- Palindrome checking

การค้นหา
Searching

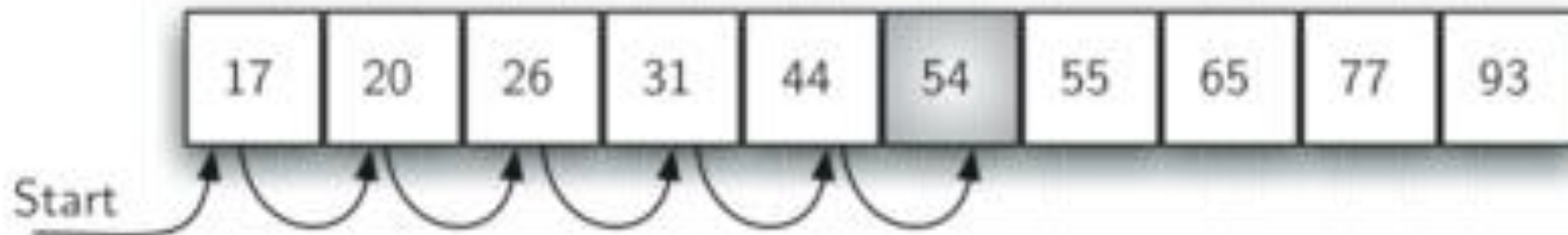
ค้นหาแบบตามลำดับ



ค้นหาแบบตามลำดับ

```
def sequentialSearch(alist, item):  
    pos = 0  
    found = False  
  
    while pos < len(alist) and not found:  
        if alist[pos] == item:  
            found = True  
        else:  
            pos = pos+1  
  
    return found  
  
testlist = [1, 2, 32, 8, 17, 19, 42, 13, 0]  
print(sequentialSearch(testlist, 3))  
print(sequentialSearch(testlist, 13))
```

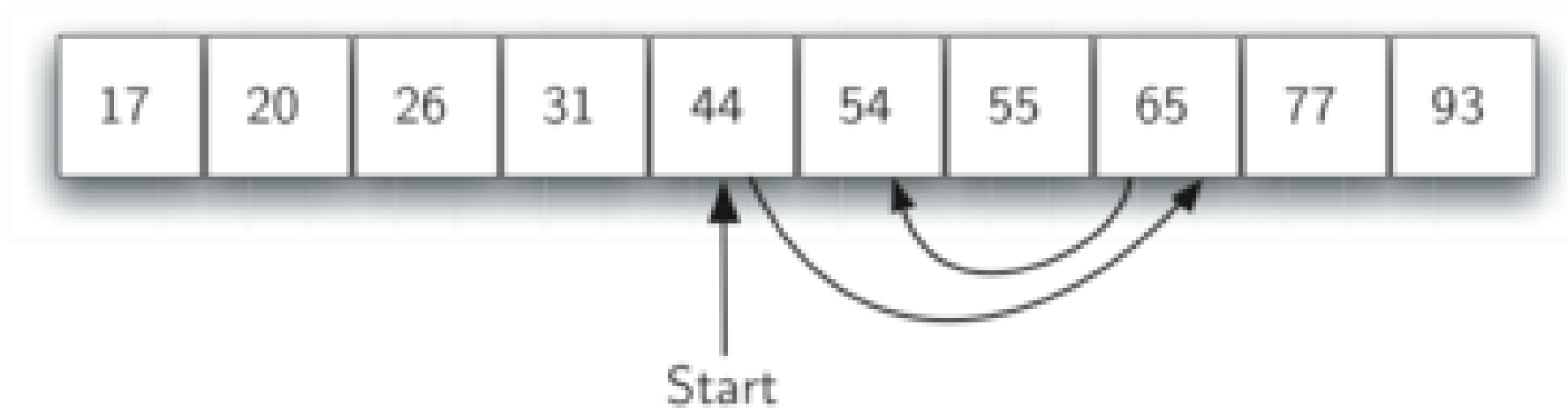
ถ้า list ที่ต้องการค้นหาเรียงลำดับ



ถ้า list ที่ต้องการค้นหาเรียงลำดับ

```
1 def orderedSequentialSearch(alist, item):
2     pos = 0
3     found = False
4     stop = False
5     while pos < len(alist) and not found and not stop:
6         if alist[pos] == item:
7             found = True
8         else:
9             if alist[pos] > item:
10                 stop = True
11             else:
12                 pos = pos+1
13
14     return found
15
```


การค้นหาแบบแบ่งครึ่ง (binary search)



การค้นหาแบบแบ่งครึ่ง

```
1 def binarySearch(alist, item):
2     first = 0
3     last = len(alist)-1
4     found = False
5
6     while first<=last and not found:
7         midpoint = (first + last)//2
8         if alist[midpoint] == item:
9             found = True
10        else:
11            if item < alist[midpoint]:
12                last = midpoint-1
13            else:
14                first = midpoint+1
15
16    return found
```

การค้นหาแบบแบ่งครึ่ง แก้ปัญหาแบบเวียนเกิด

```
1 def binarySearch(alist, item):
2     if len(alist) == 0:
3         return False
4     else:
5         midpoint = len(alist)//2
6         if alist[midpoint]==item:
7             return True
8         else:
9             if item<alist[midpoint]:
10                return binarySearch(alist[:midpoint],item)
11            else:
12                return binarySearch(alist[midpoint+1:],item)
13
```

การเรียงลำดับ

Sorting

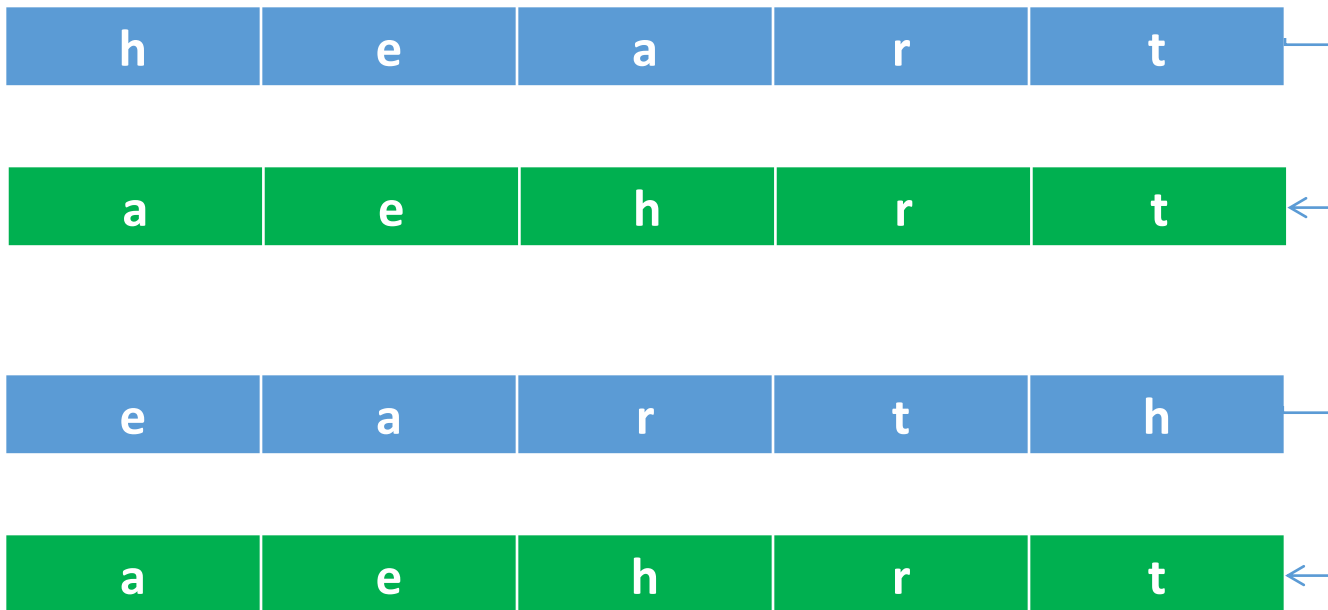
การเรียงลำดับ

54	26	93	17	77	31	44	55	20	65
----	----	----	----	----	----	----	----	----	----

17	20	26	31	44	54	55	65	77	93
----	----	----	----	----	----	----	----	----	----

Anagram detection

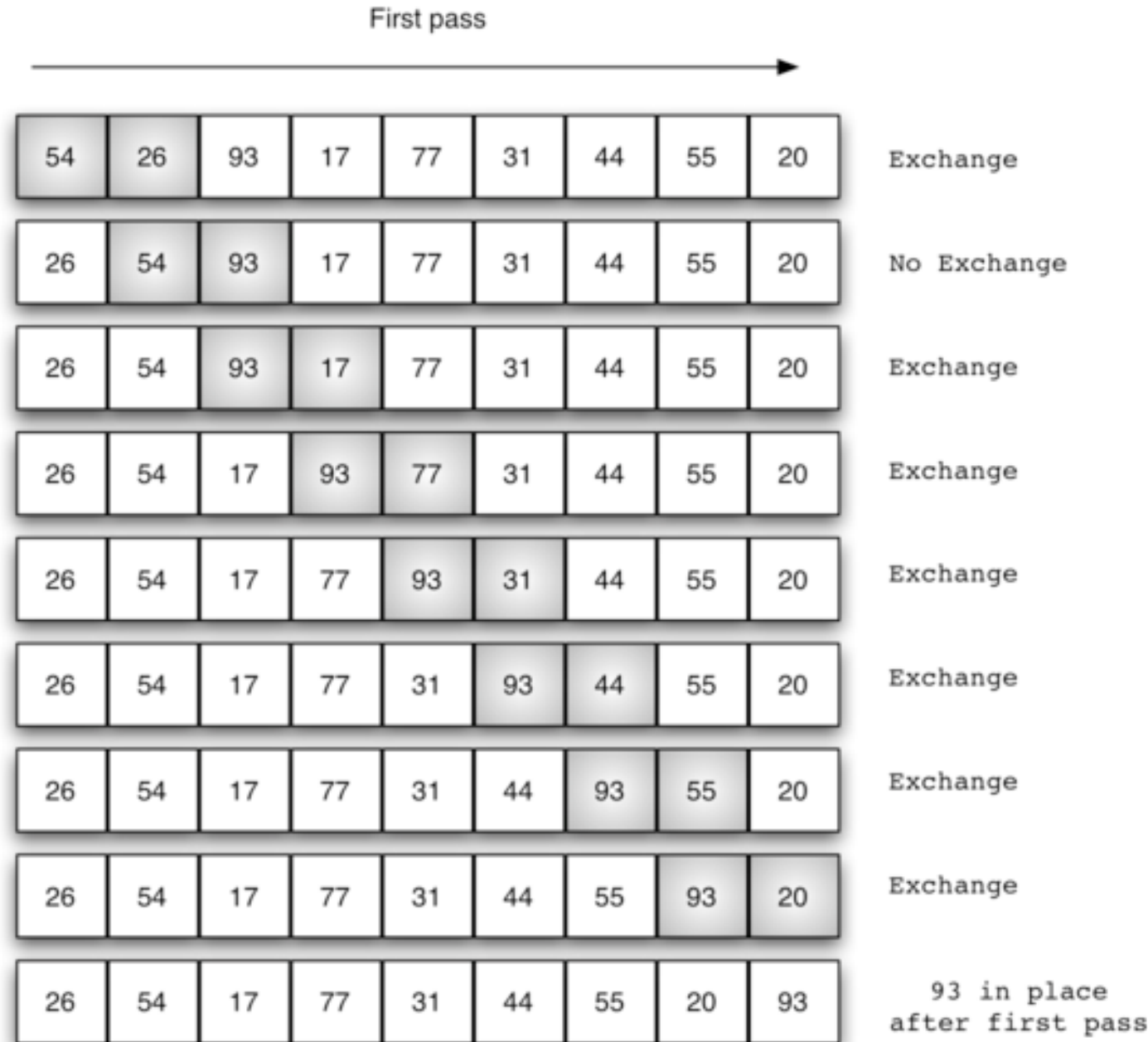
- เรียงลำดับแล้วเปรียบเทียบเป็นคู่ๆ



Bubble sort

- เปรียบเทียบเป็นคู่ๆ
- คู่ไหนไม่ตรงลำดับให้สลับ
- รอบแรก n ตัวเปรียบเทียบ $n - 1$ ครั้ง
- รอบสอง $n - 1$ ตัวเปรียบเทียบ $n - 2$ ครั้ง
- ...
- ทั้งหมด $n - 1$ รอบ

<http://interactivepython.org/runestone/static/pythonds/SortSearch/TheBubbleSort.html>



Bubble sort

```
1 def bubbleSort(alist):
2     for passnum in range(len(alist)-1,0,-1):
3         for i in range(passnum):
4             if alist[i]>alist[i+1]:
5                 temp = alist[i]
6                 alist[i] = alist[i+1]
7                 alist[i+1] = temp
8
9 alist = [54,26,93,17,77,31,44,55,20]
10 bubbleSort(alist)
11 print(alist)
12
```


Bubble sort

เรียงลำดับเสร็จแล้วยังต้องทำต่อ

```
1 def bubbleSort(alist):
2     for passnum in range(len(alist)-1,0,-1):
3         for i in range(passnum):
4             if alist[i]>alist[i+1]:
5                 temp = alist[i]
6                 alist[i] = alist[i+1]
7                 alist[i+1] = temp
8
9 alist = [54,26,93,17,77,31,44,55,20]
10 bubbleSort(alist)
11 print(alist)
12
```

Short bubble sort

```
1 def shortBubbleSort(alist):
2     exchanges = True
3     passnum = len(alist)-1
4     while passnum > 0 and exchanges:
5         exchanges = False
6         for i in range(passnum):
7             if alist[i]>alist[i+1]:
8                 exchanges = True
9                 temp = alist[i]
10                alist[i] = alist[i+1]
11                alist[i+1] = temp
12            passnum = passnum-1
13
14 alist=[20,30,40,90,50,60,70,80,100,110]
15 shortBubbleSort(alist)
16 print(alist)
17
```

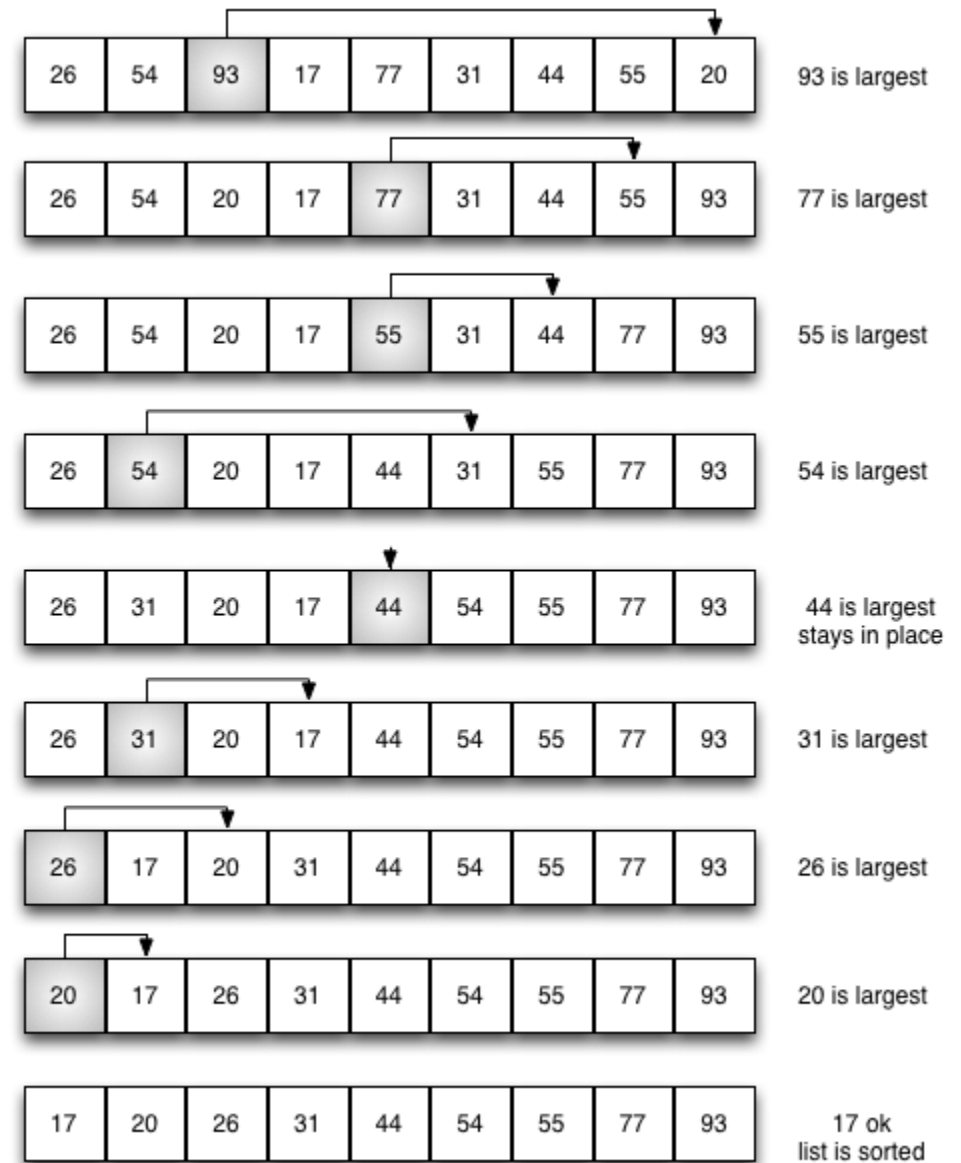
Short bubble sort

มีการสลับที่คือยังมีการเรียงลำดับอยู่

```
1 def shortBubbleSort(alist):
2     exchanges = True
3     passnum = len(alist)-1
4     while passnum > 0 and exchanges:
5         exchanges = False
6         for i in range(passnum):
7             if alist[i]>alist[i+1]:
8                 exchanges = True
9                 temp = alist[i]
10                alist[i] = alist[i+1]
11                alist[i+1] = temp
12        passnum = passnum-1
13
14 alist=[20,30,40,90,50,60,70,80,100,110]
15 shortBubbleSort(alist)
16 print(alist)
17
```

Selection sort

- หาตัวที่มากที่สุดไปวางที่ตำแหน่งสุดท้าย
- ใช้ $n - 1$ รอบในการเรียงลำดับ n ตัว

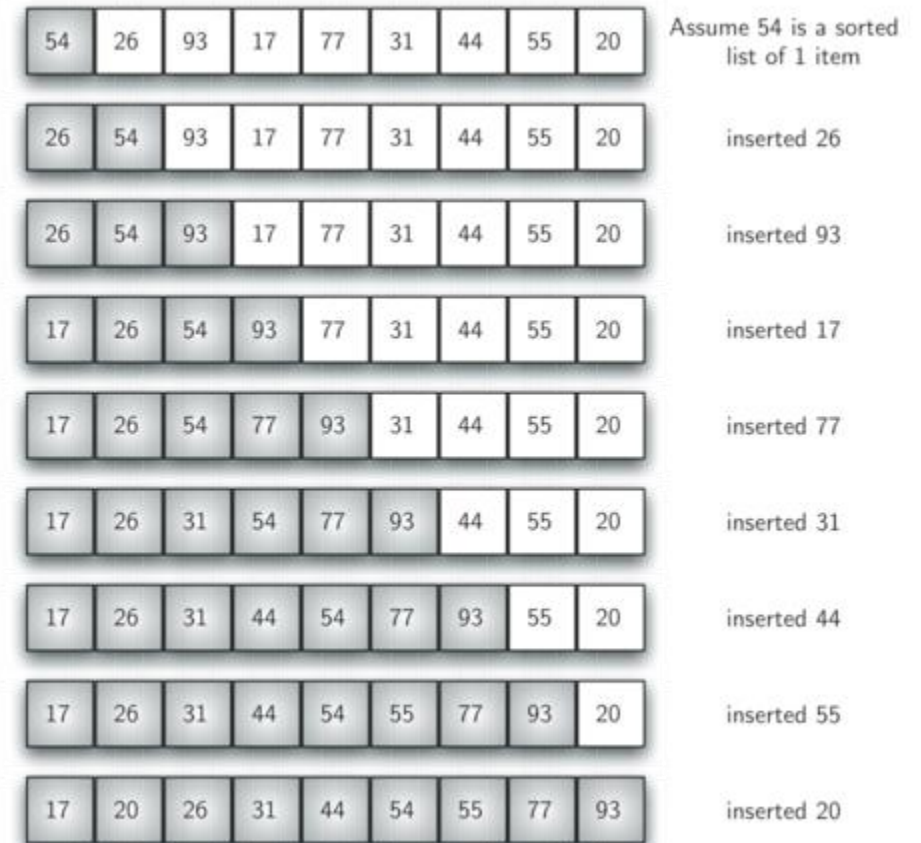


Selection sort

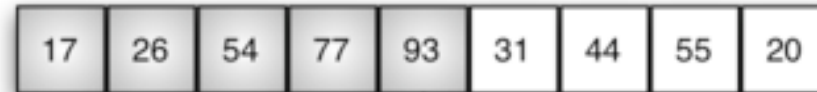
```
1 def selectionSort(alist):
2     for fillslot in range(len(alist)-1,0,-1):
3         positionOfMax=0
4         for location in range(1,fillslot+1):
5             if alist[location]>alist[positionOfMax]:
6                 positionOfMax = location
7
8         temp = alist[fillslot]
9         alist[fillslot] = alist[positionOfMax]
10        alist[positionOfMax] = temp
11
12 alist = [54,26,93,17,77,31,44,55,20]
13 selectionSort(alist)
14 print(alist)
15
```

Insertion sort

- ให้ส่วนที่เรียงลำดับแล้วอยู่ด้านหน้า
 - เริ่มที่ **1** ตัว
- ตัวถัดไปจะถูกใส่ในส่วนด้านหน้าให้ถูกตำแหน่ง



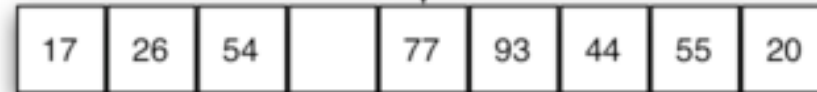
Insertion sort



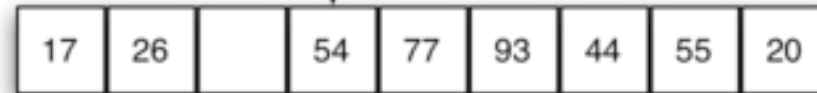
Need to insert 31
back into the sorted list



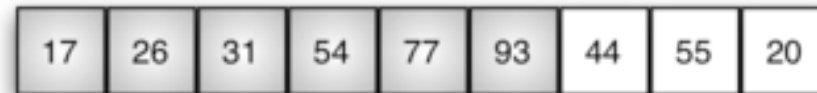
$93 > 31$ so shift it
to the right



$77 > 31$ so shift it
to the right



$54 > 31$ so shift it
to the right

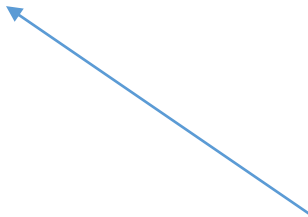


$26 < 31$ so insert 31
in this position

Insertion sort

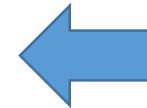
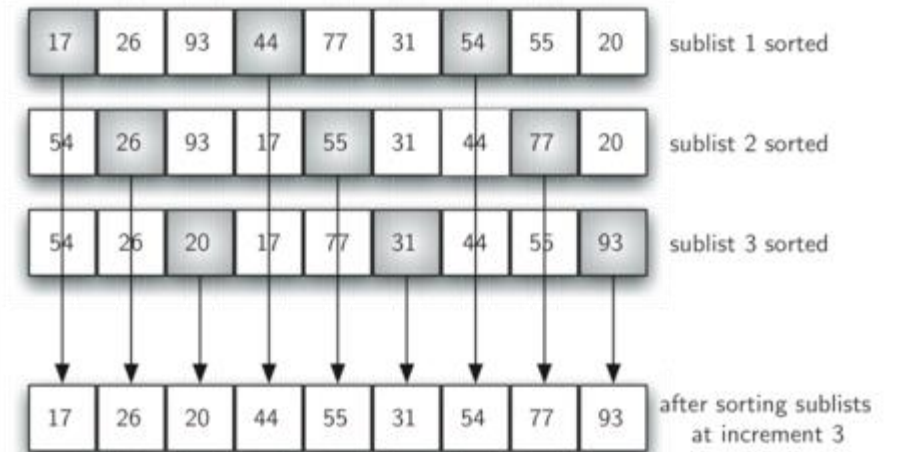
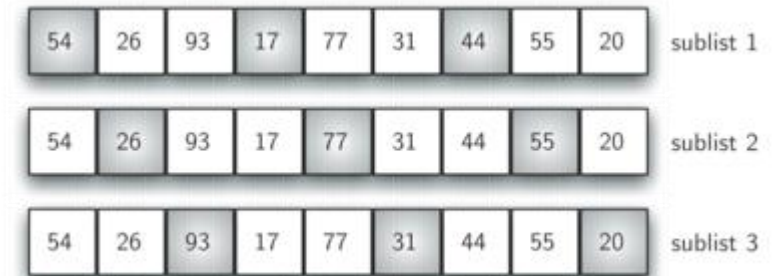
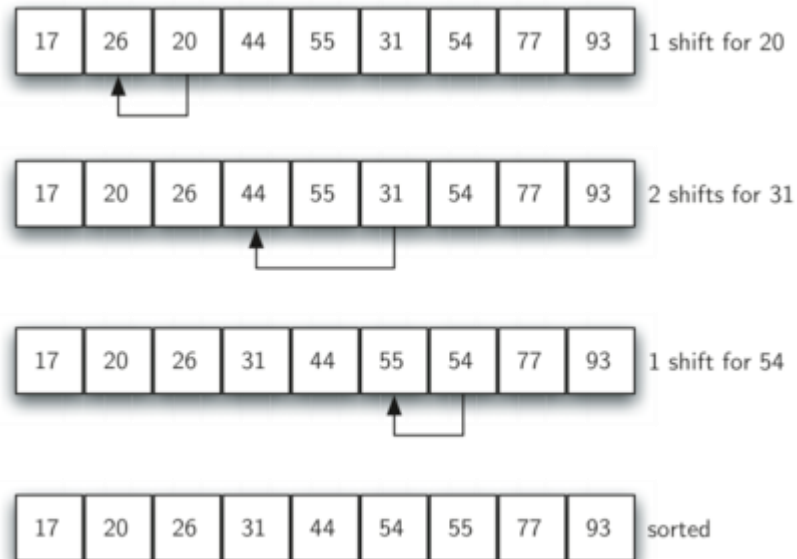
```
1 def insertionSort(alist):
2     for index in range(1,len(alist)):
3
4         currentvalue = alist[index]
5         position = index
6
7         while position>0 and alist[position-1]>currentvalue:
8             alist[position]=alist[position-1]
9             position = position-1
10
11         alist[position]=currentvalue
12
13 alist = [54,26,93,17,77,31,44,55,20]
14 insertionSort(alist)
15 print(alist)
16
```

เลื่อนไปทางขวา



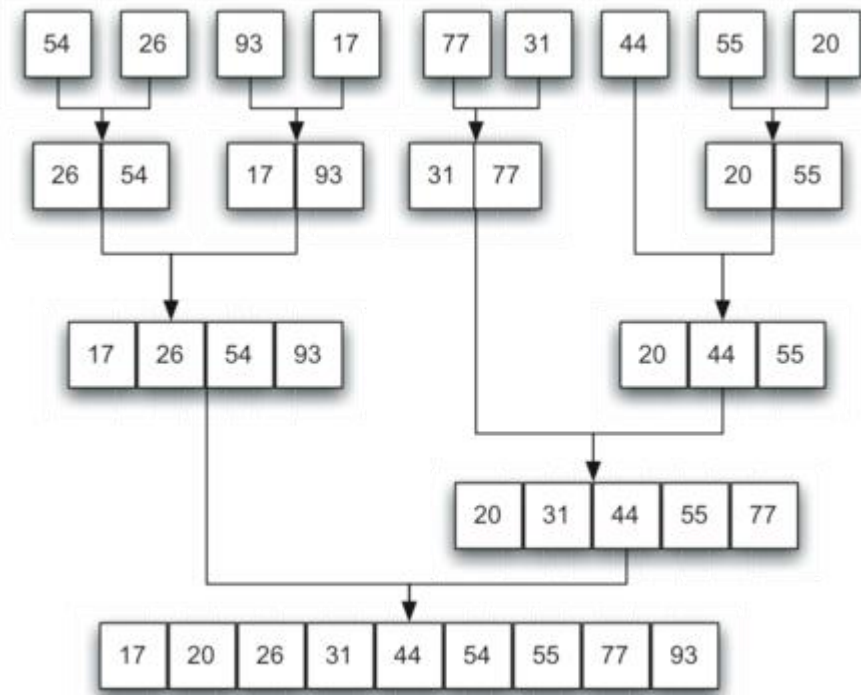
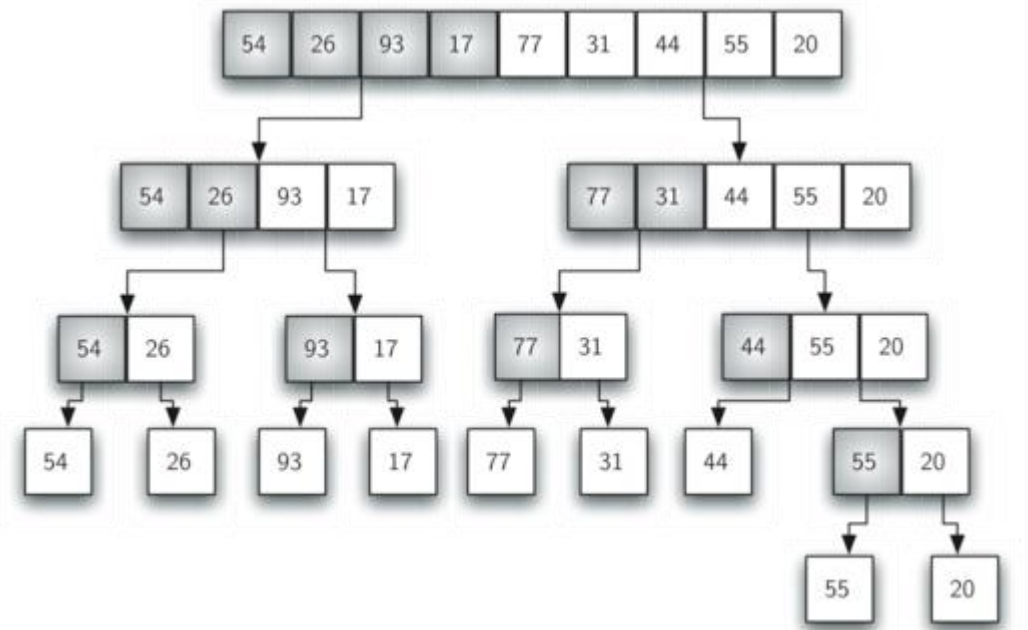
Shell sort

- Insertion sort แบบกระโดดข้าม
- รวมผลจาก insertion sort แบบกระโดดข้าม
- Insertion sort แบบปกติ



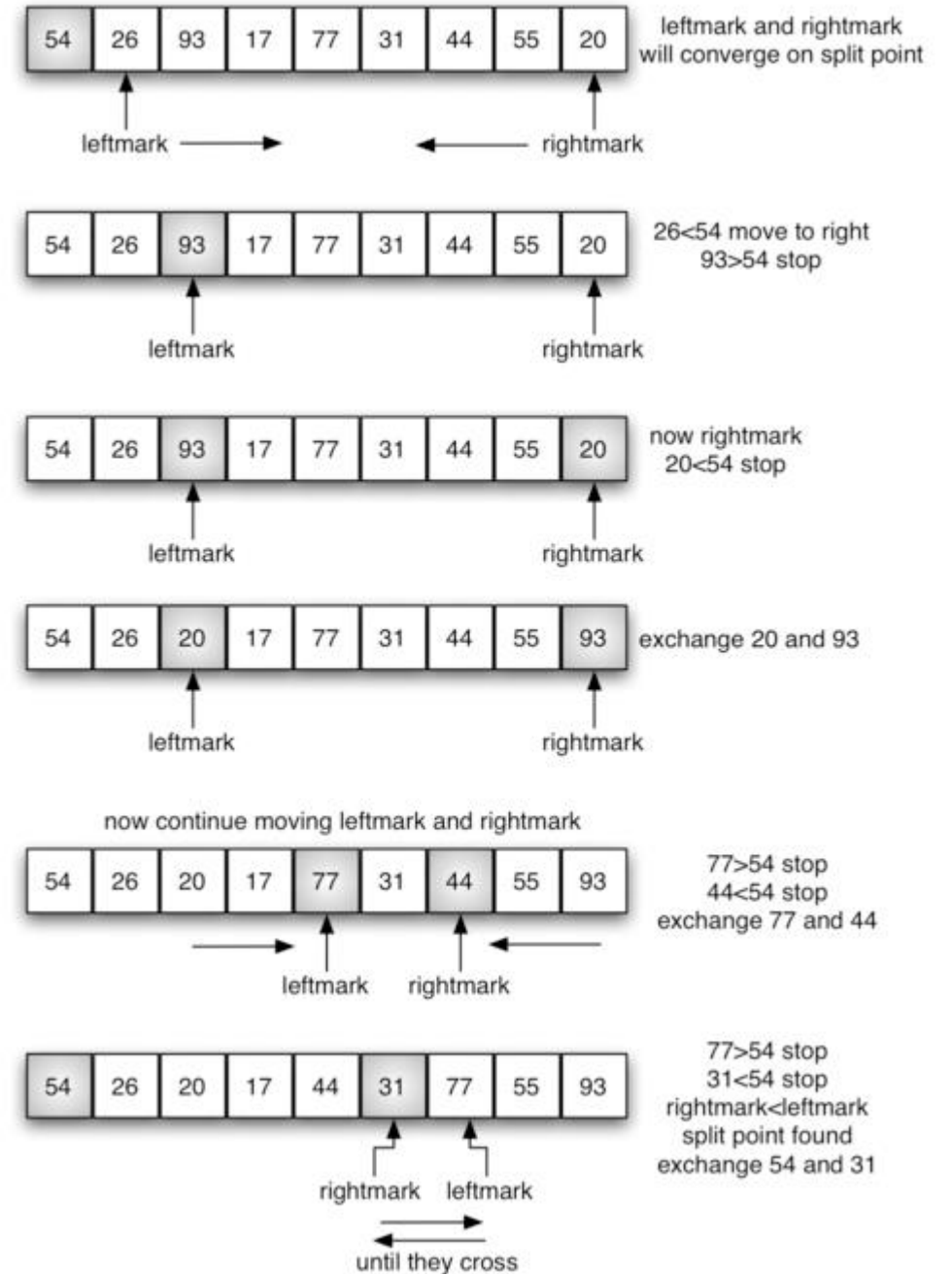
Merge sort

- แบ่งเป็นปัญหาย่อยลงไปเรื่อยๆ
- รวมกลับขึ้นมาเป็นการแก้ปัญหาลใหญ่
 - เปรียบเทียบแล้วเลื่อนหมุน



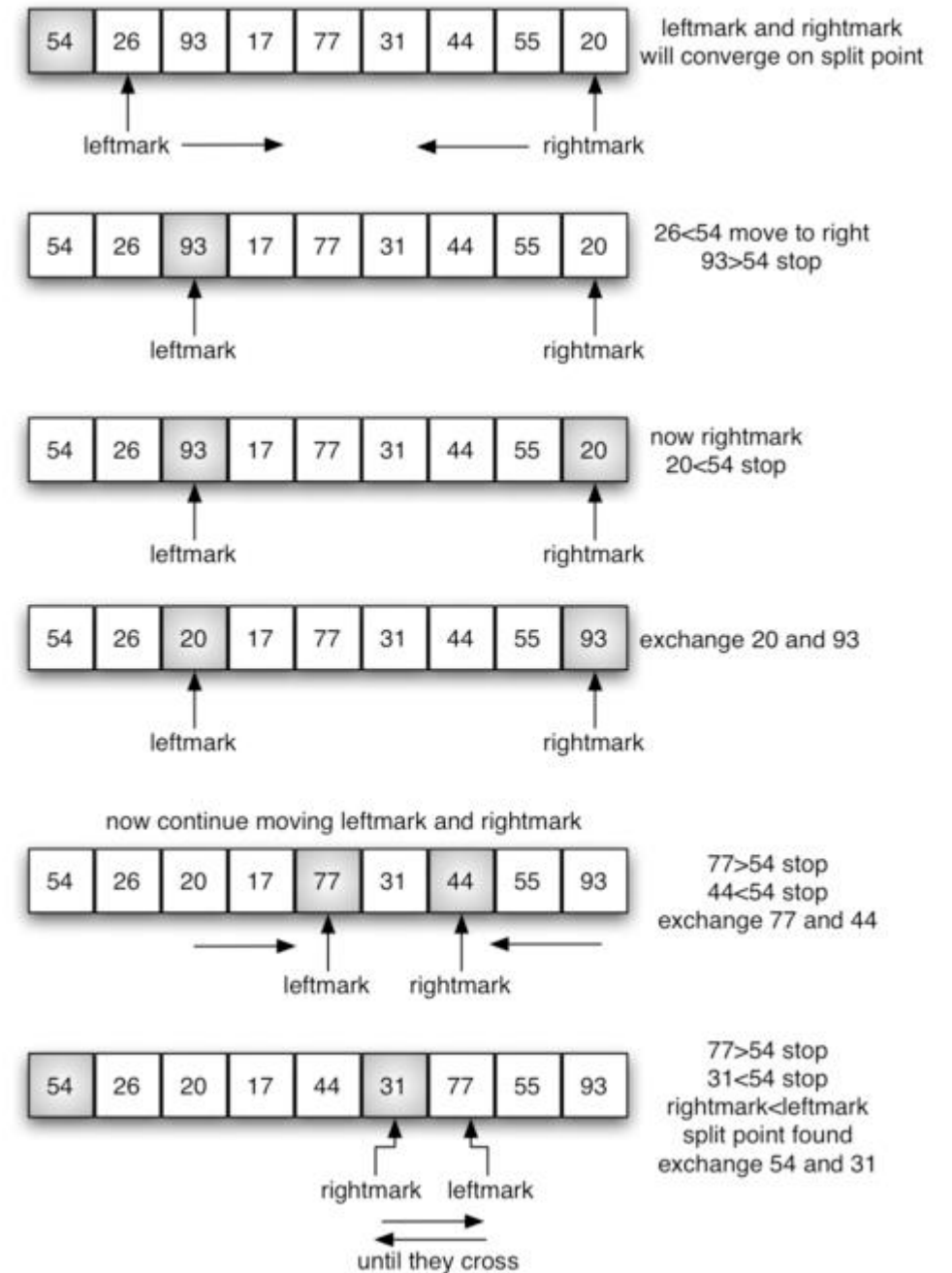
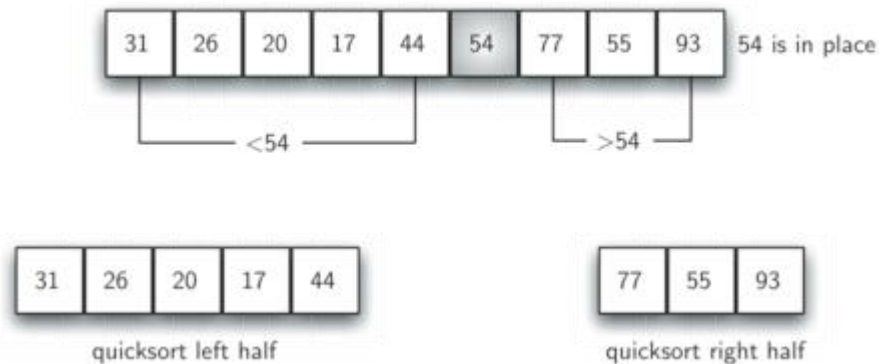
Quick sort

- แบ่งปัญหาย่อยด้วยหมด
 - ใช้ตัวแรกของปัญหาย่อยเป็นหมด
- มี **marker 2** ตัวไว้จัดกลุ่ม
 - กลุ่มที่ $<$ ตัวหมด
 - กลุ่มที่ $\geq$ ตัวหมด
- ทั้ง **2** กลุ่มคือปัญหาย่อยที่นำไปแบ่งต่อ



Quick sort

- แบ่งปัญหาย่อยด้วยหมด
 - ใช้ตัวแรกของปัญหาย่อยเป็นหมด
- มี **marker 2** ตัวไว้จัดกลุ่ม
 - กลุ่มที่ $<$ ตัวหมด
 - กลุ่มที่ $\geq$ ตัวหมด
- ทั้ง **2** กลุ่มคือปัญหาย่อยที่นำไปแบ่งต่อ



แฮช

Hashing

แฮชทำไม?

- เพื่อเข้าถึงข้อมูลได้อย่างรวดเร็ว
- เราเคยใช้มาก่อนคือ **dictionary**
 - บอก **key** ได้ **value**

Key Value	John 0143	Jack 0001	Jim 0020	Jones 0989	Jane 1233	June 1234	July 9087	Joly 8765
--------------	--------------	--------------	-------------	---------------	--------------	--------------	--------------	--------------

ลองหาแบบ **List**

แฮชทำไม?

- เพื่อเข้าถึงข้อมูลได้อย่างรวดเร็ว
- เราเคยใช้มาก่อนคือ **dictionary**
 - บอก **key** ได้ **value**



Key	John	Jack	Jim	Jones	Jane	June	July	Joly
Value	0143	0001	0020	0989	1233	1234	9087	8765

July → Hash function → 6

ตารางเลข

- เก็บอยู่ในรูปของตารางแบ่งเป็น “ช่อง” (slot)

0	1	2	3	4	5	6	7	8	9	10
77	None	None	None	26	93	17	None	None	31	54

- ตารางเริ่มต้น

[illegible]

ตัวอย่าง

- สนใจเฉพาะตัวเลข
 - 54, 26, 93, 17, 77, 31
- ตารางจำนวน 11 ช่อง
- หากการจับคู่ (map) ระหว่างตัวเลขกับตำแหน่งช่อง
 - ง่ายสุดคือ mod 11
 - $h(\text{item}) = \text{item} \% 11$
 - ฟังก์ชันแฮช

Item	Hash Value
54	10
26	4
93	5
17	6
77	0
31	9

0	1	2	3	4	5	6	7	8	9	10
77	None	None	None	26	93	17	None	None	31	54

ค่า load factor

- $\lambda = \frac{\text{number of items}}{\text{table size}}$
- จากตัวอย่างคือ $\frac{6}{11}$

0	1	2	3	4	5	6	7	8	9	10
77	None	None	None	26	93	17	None	None	31	54

การเข้าถึงข้อมูล

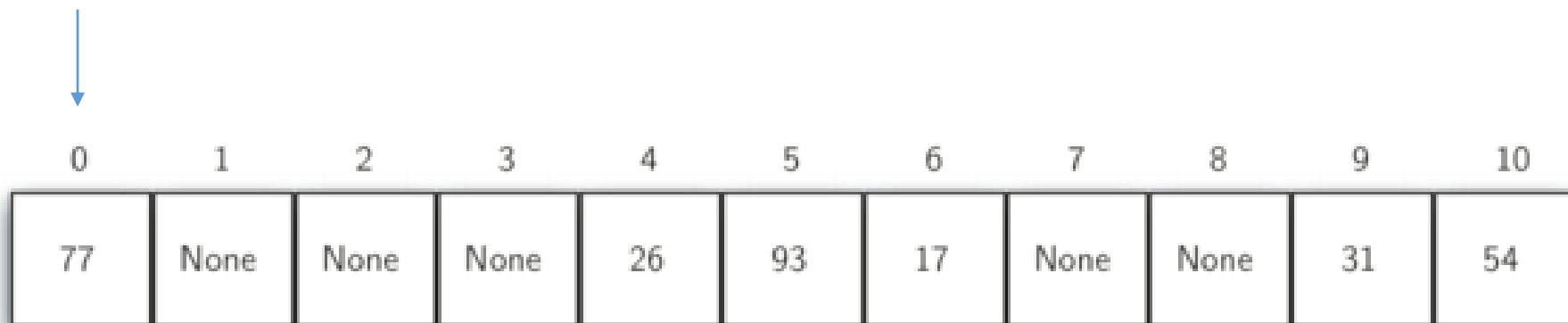
- ใช้ฟังก์ชันแฮช
- เช่น อยากถามว่ามี 54 หรือไม่
 - $h(\text{item}) = \text{item} \% 11$
 - $h(54) = 54 \% 11 = 10$
- $O(1)$



0	1	2	3	4	5	6	7	8	9	10
77	None	None	None	26	93	17	None	None	31	54

การเข้าถึงข้อมูล

- เหมือนจะดีถ้าข้อมูลทั้งหมดลงที่ช่องต่างกันหมด
- ถ้ามี 44 เพิ่มเข้ามา
 - $h(\text{item}) = \text{item} \% 11$
 - $h(44) = 44 \% 11 = 0$
- เกิดการชน (collision)



0	1	2	3	4	5	6	7	8	9	10
77	None	None	None	26	93	17	None	None	31	54

ฟังก์ชันแฮช

- เราอยากได้ฟังก์ชันแฮชที่ไม่มีการชน
 - 1 ช่อง 1 ข้อมูล
- ตัวอย่าง ID 3 หลัก
 - ความเป็นไปได้ทั้งหมดคือ 10^3
 - สร้างตารางขนาด 10^3 จะไม่มีการชนเกิดขึ้น
 - แต่ถ้ามีการใช้ ID จริงๆ แค่ 10 ?
 - Load factor ควรจะเยอะด้วย

ฟังก์ชันแฮชที่ดี

- ชนน้อย
- คำนวณง่าย
- กระจายตัว

วิธีการพับ (folding)

- แบ่งเลขเป็นช่วงๆ บวกกันแล้ว mod
- ตัวอย่างหมายเลขโทรศัพท์ 436-555-4601
- แบ่งช่วงละ 2 ได้ (43,65,55,46,01)
- บวกกัน $43+65+55+46+01 = 210$
- mod จำนวนช่อง $210 \% 11 = 1$
- จับใส่ช่อง 1

ตัวกลางยกกำลัง (mid square method)

- $44^2 = 1936$
- เอาสองตัวกลางมา **mod** จำนวนช่อง $93 \% 11 = 5$

Item	Remainder	Mid-Square
54	10	3
26	4	7
93	5	9
17	6	8
77	0	4
31	9	6

ถ้าเป็นตัวอักษร?

- แปลงตัวอักษรเป็นตัวเลข
- cat
- `ord('c') = 99`
- `ord('a') = 97`
- `ord('t') = 116`
- บวกกัน $99 + 97 + 116 = 312$
- mod ด้วยจำนวนช่อง $312 \% 11$

```
def hash(astring, tablesize):  
    sum = 0  
    for pos in range(len(astring)):  
        sum = sum + ord(astring[pos])  
  
    return sum%tablesize
```

Anagram

- ถ้าเป็น anagram จะได้ค่าแฮชเดียวกัน
- $\text{hash}(\text{'cat'}) = \text{hash}(\text{'act'})$ ขน
- ใส่ค่าถ่วงน้ำหนักตามตำแหน่ง
- $\text{hash}(\text{'cat'}) = 99 * 1 + 97 * 2 + 116 * 3 = 641 \% 11 = 3$
- $\text{hast}(\text{'act'}) = 97 * 1 + 99 * 2 + 116 * 3 = 643 \% 11 = 5$

mod ด้วยจำนวนช่องเสมอ !!!!

แก้ปัญหาการชน

ไล่หาช่องว่างช่องแรกที่เราเจอ (linear probing)

- เพิ่ม 44
- $\text{hash}(44) = 44 \% 11 = 0$ ชน

0	1	2	3	4	5	6	7	8	9	10
77	None	None	None	26	93	17	None	None	31	54

- ช่องว่างถัดไปคือ 1

0	1	2	3	4	5	6	7	8	9	10
77	44	None	None	26	93	17	None	None	31	54

ไต่หาช่องว่างช่องแรกที่เราเจอ (linear probing)

- เพิ่ม 55
- $\text{hash}(55) = 55 \% 11 = 0$ ชน

0	1	2	3	4	5	6	7	8	9	10
77	44	None	None	26	93	17	None	None	31	54

- ช่องว่างถัดไปคือ 2

0	1	2	3	4	5	6	7	8	9	10
77	44	55	None	26	93	17	None	None	31	54

ไต่หาช่องว่างช่องแรกที่เจอ (linear probing)

- เพิ่ม 20
- $\text{hash}(20) = 20 \% 11 = 9$ ชน

0	1	2	3	4	5	6	7	8	9	10
77	44	55	None	26	93	17	None	None	31	54

- ช่องว่างถัดไปคือ 3 (หมดตารางวนกลับมาเริ่มที่ 0)

0	1	2	3	4	5	6	7	8	9	10
77	44	55	20	26	93	17	None	None	31	54

การเข้าถึงข้อมูลแบบ linear probing

- หาเจอตอบเลยว่า **True**
 - หา 93
 - $\text{hash}(93) = 93 \% 11 = 5$ เจอ
- หาไม่เจอยังตอบไม่ได้
 - หา 20
 - $\text{hash}(20) = 20 \% 11 = 9$ ชน
 - วิ่งหา 20 เจอตอบ **True** วิ่งเจอช่องว่าง (**None**) ถึงจะตอบ **False**

0	1	2	3	4	5	6	7	8	9	10
77	44	55	20	26	93	17	None	None	31	54

ปัญหาของ linear probing

- ข้อมูลที่ชนกันด้วยค่าแฮชเดียวกันจะกระจุกตัว
- ตัวที่ $\text{mod } 11 = 0$ กระจุกตัวกัน
- เต็ม 20 ต้องไล่หลายช่องกว่าจะเจอช่องว่าง

0	1	2	3	4	5	6	7	8	9	10
77	44	55	None	26	93	17	None	None	31	54

0	1	2	3	4	5	6	7	8	9	10
77	44	55	20	26	93	17	None	None	31	54

กระโดดข้าม

- กระโดดหาช่องว่างทีละ 3 ช่อง
- เต็ม 44 คนที่ช่อง 0 กระโดดไป 3 เจอช่อง 3 ว่าง
- เต็ม 55 คนที่ช่อง 0 กระโดดไป 3 เจอช่อง 3 คนอีก กระโดดไป 3 ช่องเจอช่อง 6 คนอีก กระโดดไป 3 ช่องเจอช่อง 9 คนอีก กระโดดไป 3 เจอช่อง 1 ว่าง
- เต็ม 20 ?

0	1	2	3	4	5	6	7	8	9	10
77	None	None	None	26	93	17	None	None	31	54

0	1	2	3	4	5	6	7	8	9	10
77	55	None	44	26	93	17	20	None	31	54

การแก้ไข (rehash)

- การหาช่องว่างเมื่อเกิดการชน
- ฟังก์ชันการแก้ไข
- $\text{rehash}(\text{pos}) = (\text{pos} + 1) \% \text{sizeof table}$
- $\text{rehash}(\text{pos}) = (\text{pos} + 3) \% \text{sizeof table}$
- $\text{rehash}(\text{pos}) = (\text{pos} + \text{skip}) \% \text{sizeof table}$
- วิธีการกระโดดข้ามควรใช้กับขนาดตารางที่เป็นจำนวนเฉพาะ

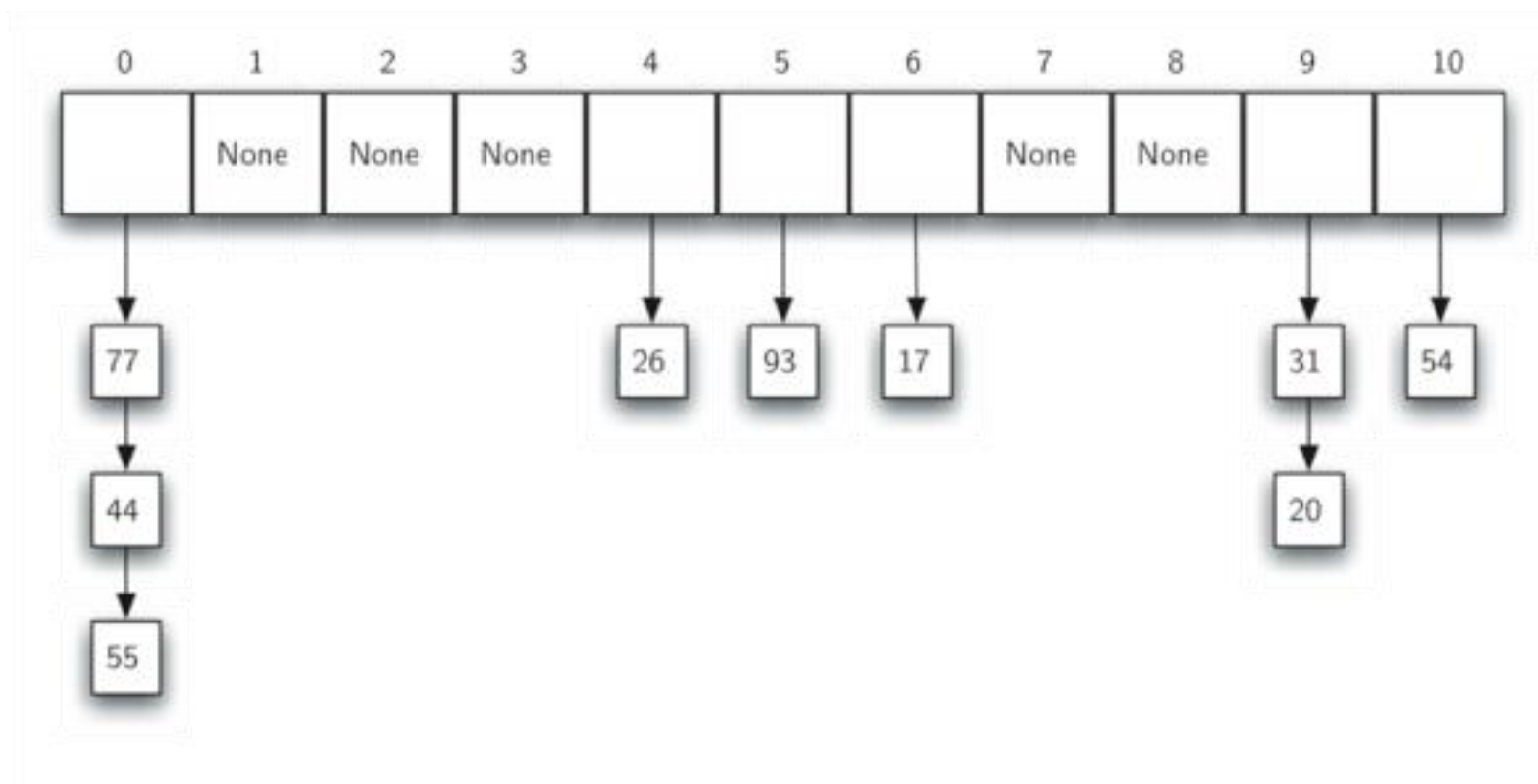
กระโดดแบบยกกำลังสอง

- ชนครั้งแรกกระโดดจากค่าแฮชเดิม $1^2 = 1$
- ชนครั้งที่สองกระโดดจากค่าแฮชเดิม $2^2 = 4$
- ชนครั้งที่สองกระโดดจากค่าแฮชเดิม $3^2 = 9$
- $\text{rehash}(h) = h + 1$
- $\text{rehash}(h) = h + 4$
- $\text{rehash}(h) = h + 9$

0	1	2	3	4	5	6	7	8	9	10
77	None	None	None	26	93	17	None	None	31	54

0	1	2	3	4	5	6	7	8	9	10
77	44	20	55	26	93	17	None	None	31	54

การเก็บเป็นลูกโซ่



Map ADT

- หลักการเดียวกับ **dictionary**
 - เรามองว่า **key** คือค่าที่จะนำไปคำนวณแฮชฟังก์ชันแล้วเก็บ **val** คู่ไปด้วย เรียกอีกอย่างว่า **Map**
- **Map()** สร้าง **map** ว่างแล้วคืนค่า **map** ว่างนั้น
- **put(key, val)** เพิ่มคู่ **key-val** เข้าใน **map** ถ้า **key** มีอยู่แล้วใน **map** ให้แทนที่ค่าเก่าด้วย **val** ใหม่
- **get(key)** คืนค่าที่ **key** นั้น หรือ **None** ถ้าไม่มี **key** นั้น
- **del map[key]** ลบค่าที่ **key**
- **len()** คืนค่าจำนวนที่เก็บอยู่ใน **map**.
- **key in map** คืนค่า **True** เมื่อมี **key** ใน **map** หรือ **False** เมื่อไม่มี **key** นั้น

สร้าง class HashTable

```
class HashTable:  
    def __init__(self):  
        self.size = 11  
        self.slots = [None] * self.size  
        self.data = [None] * self.size
```

```

def put(self, key, data):
    hashvalue = self.hashfunction(key, len(self.slots))

    if self.slots[hashvalue] == None:
        self.slots[hashvalue] = key
        self.data[hashvalue] = data
    else:
        if self.slots[hashvalue] == key:
            self.data[hashvalue] = data #replace
        else:
            nextslot = self.rehash(hashvalue, len(self.slots))
            while self.slots[nextslot] != None and \
                  self.slots[nextslot] != key:
                nextslot = self.rehash(nextslot, len(self.slots))

            if self.slots[nextslot] == None:
                self.slots[nextslot] = key
                self.data[nextslot] = data
            else:
                self.data[nextslot] = data #replace

def hashfunction(self, key, size):
    return key % size

def rehash(self, oldhash, size):
    return (oldhash + 1) % size

```

```

def get(self, key):
    startslot = self.hashfunction(key, len(self.slots))

    data = None
    stop = False
    found = False
    position = startslot
    while self.slots[position] != None and \
           not found and not stop:
        if self.slots[position] == key:
            found = True
            data = self.data[position]
        else:
            position = self.rehash(position, len(self.slots))
            if position == startslot:
                stop = True
    return data

def __getitem__(self, key):
    return self.get(key)

def __setitem__(self, key, data):
    self.put(key, data)

```

ใช้งาน HashTable

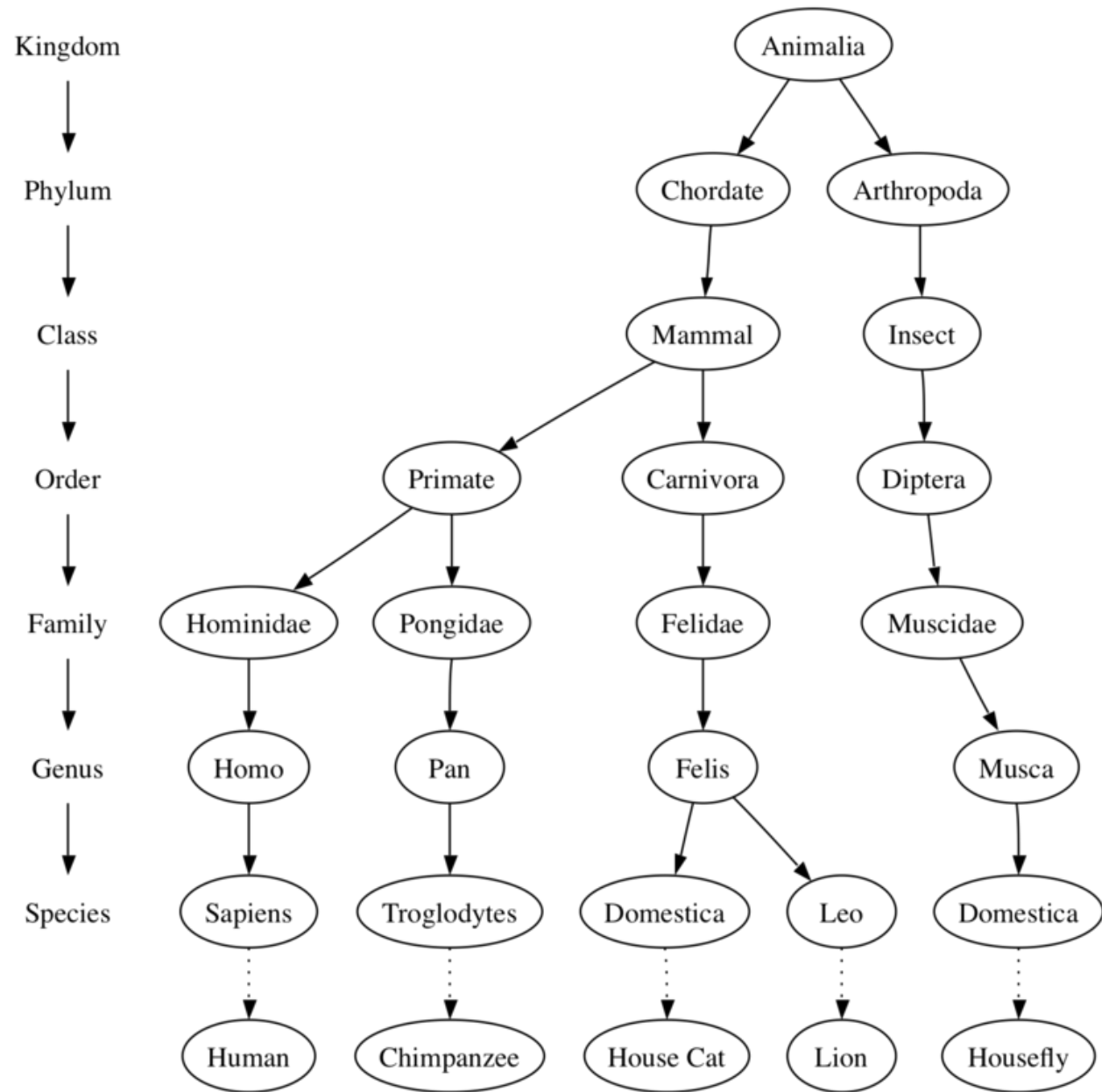
```
>>> H=HashTable()
>>> H[54]="cat"
>>> H[26]="dog"
>>> H[93]="lion"
>>> H[17]="tiger"
>>> H[77]="bird"
>>> H[31]="cow"
>>> H[44]="goat"
>>> H[55]="pig"
>>> H[20]="chicken"
>>> H.slots
[77, 44, 55, 20, 26, 93, 17, None, None, 31, 54]
>>> H.data
['bird', 'goat', 'pig', 'chicken', 'dog', 'lion',
 'tiger', None, None, 'cow', 'cat']
```

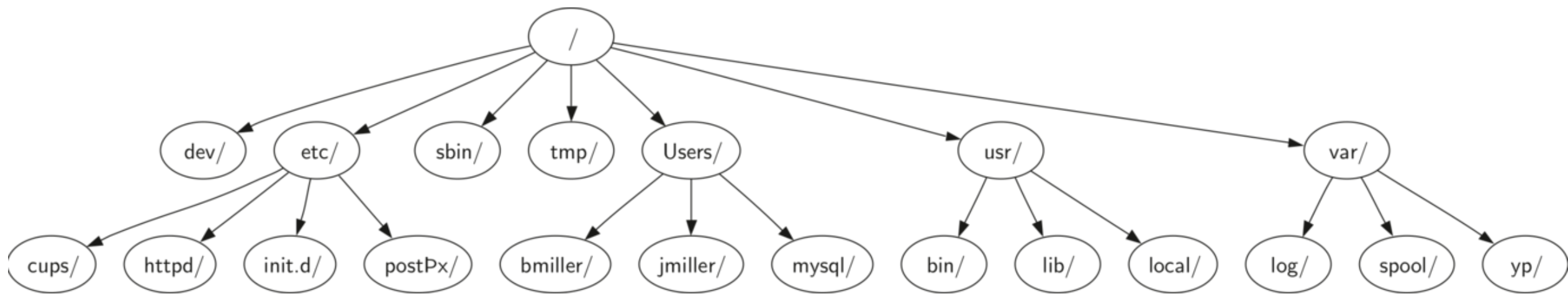
```
>>> H[20]
'chicken'
>>> H[17]
'tiger'
>>> H[20]='duck'
>>> H[20]
'duck'
>>> H.data
['bird', 'goat', 'pig', 'duck', 'dog', 'lion',
 'tiger', None, None, 'cow', 'cat']
>> print(H[99])
None
```


ต้นไม้ (tree)

เรียนเพื่ออะไร?

- เข้าใจหลักการของโครงสร้างข้อมูลต้นไม้
- เพื่อให้เห็นว่าต้นไม้สามารถใช้ในการสร้างโครงสร้างข้อมูลแมพได้
- เพื่อสร้างต้นไม้โดยใช้ **list**
- เพื่อสร้าง **class** ของต้นไม้
- เพื่อสร้างต้นไม้ในรูปแบบความสัมพันธ์เวียนเกิด
- เพื่อสร้างคิวแบบมีความสำคัญ (**priority queue**) ด้วยฮีพ (**heap**)





```
<html xmlns="http://www.w3.org/1999/xhtml"
```

```
  xml:lang="en" lang="en">
```

```
<head>
```

```
  <meta http-equiv="Content-Type"
```

```
    content="text/html; charset=utf-8" />
```

```
  <title>simple</title>
```

```
</head>
```

```
<body>
```

```
<h1>A simple web page</h1>
```

```
<ul>
```

```
  <li>List item one</li>
```

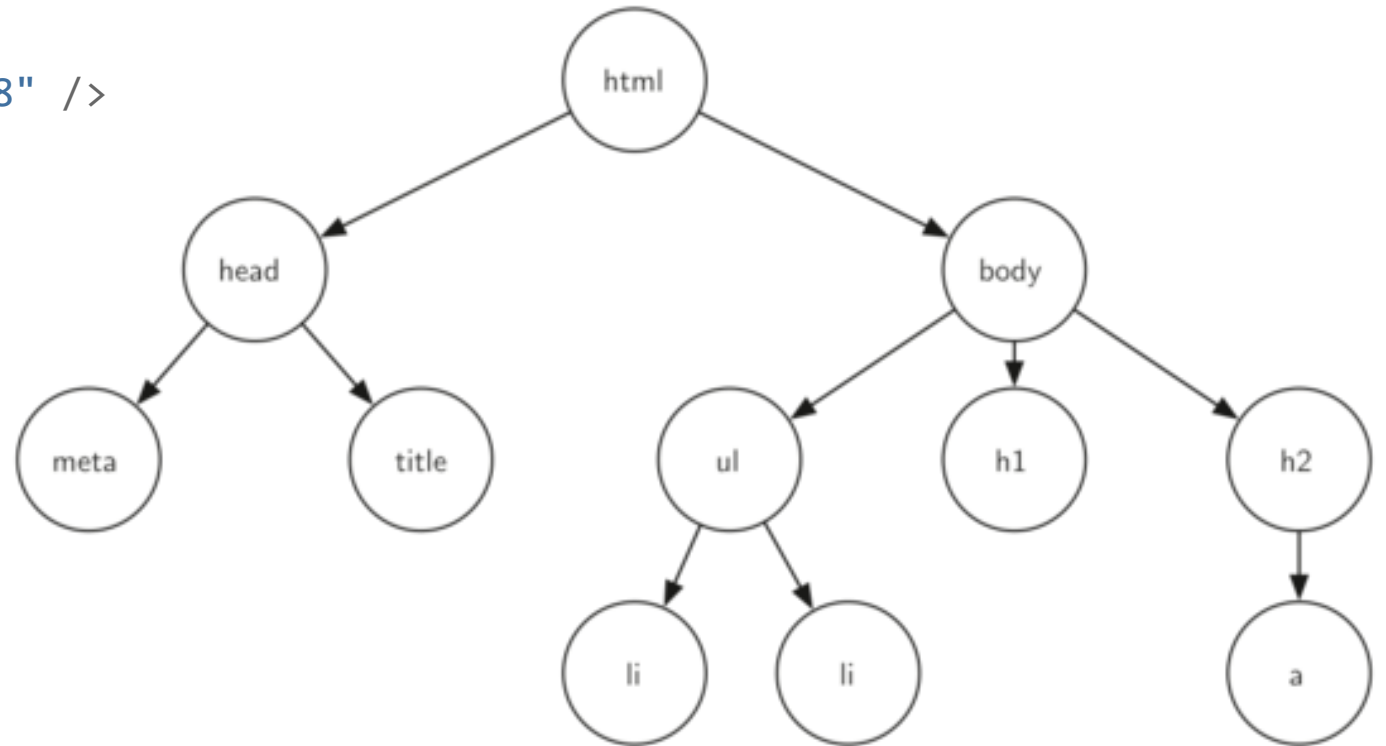
```
  <li>List item two</li>
```

```
</ul>
```

```
<h2><a href="http://www.cs.luther.edu">Luther CS </a></h2>
```

```
</body>
```

```
</html>
```

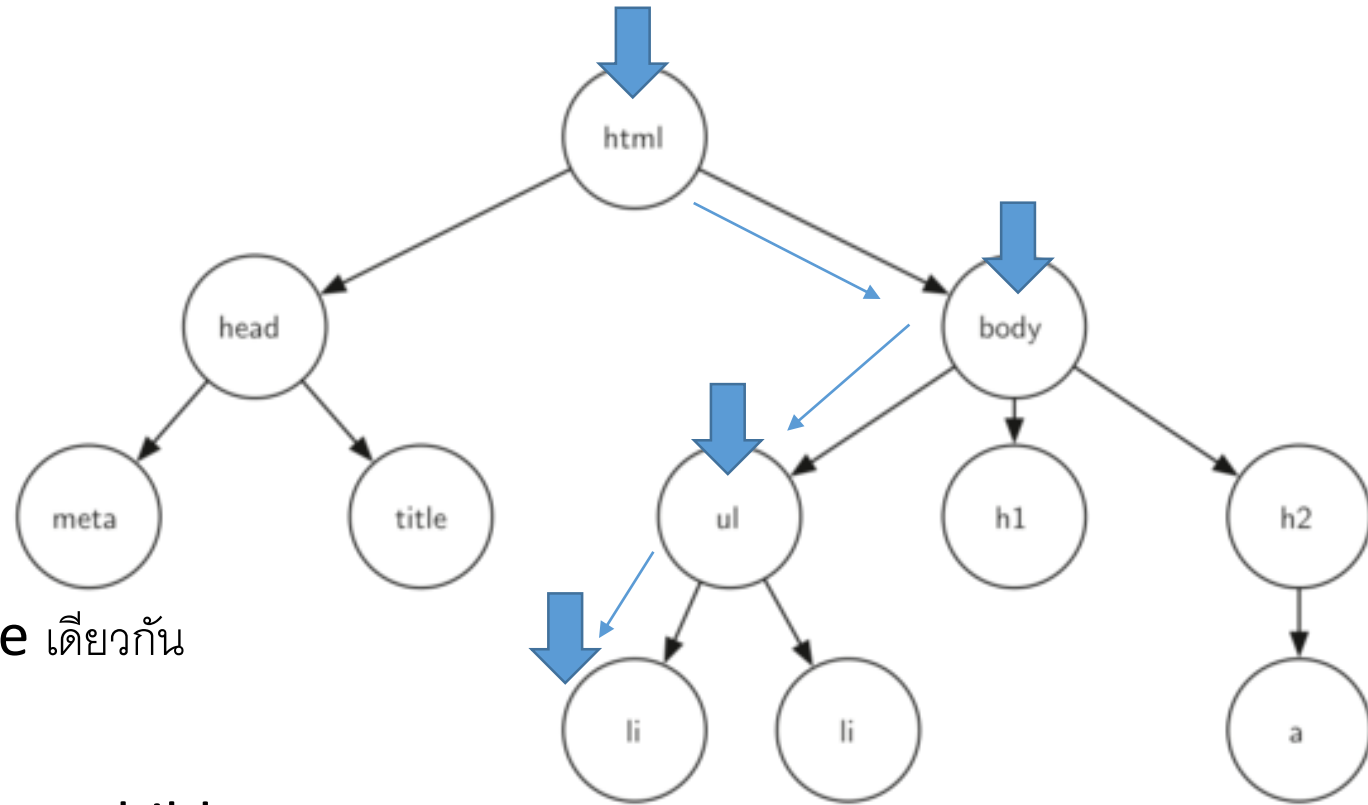


คำศัพท์

- ปม (node)
 - มีชื่อได้ เรียกว่า **key**
 - มีข้อมูลอื่นๆ เพิ่มได้ เรียกว่า **payload**
- เส้นเชื่อม (edge)
 - เชื่อมระหว่าง **2 node** เพื่อแสดงความสัมพันธ์ระหว่างกัน
 - ทุก **node** ต้องมีอย่างน้อย **1 edge** เข้าหามัน (ยกเว้น **root**)
 - **1 node** สามารถมีได้มากกว่า **1 edge** ออกจากตัวมัน
- ราก (root)
 - เป็น **node** ที่มีแต่ขาออก

คำศัพท์

- เส้นทาง (path)
 - list ของลำดับของ node ที่เชื่อมด้วย edge
- ลูก (children)
 - set ของ node ที่มี edge ขาเข้าจาก node เดียวกัน
- พ่อ (parent)
 - node ที่เป็นขาออกของ edge เพื่อไปเชื่อมต่อกับ children
- พี่น้อง (sibling)
 - node ที่มี parent เดียวกัน

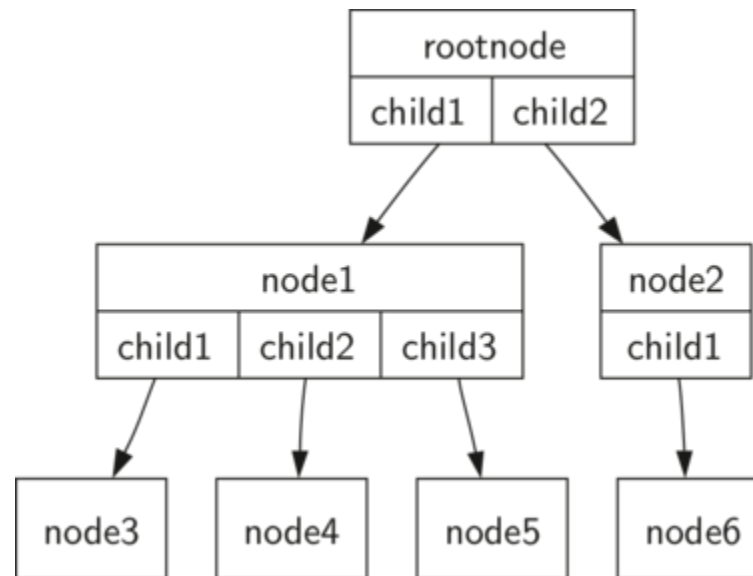


คำศัพท์

- ต้นไม้ย่อย (subtree)
 - set ของ node และ edge ประกอบด้วย parent และผู้สืบสายพันธุ์
- ปมใบไม้ (leaf node)
 - node ที่ไม่มี children
- ระดับ (level)
 - จำนวนของ edge จาก root ถึงตัวมัน
- ความสูง (height)
 - level ที่มากที่สุดของ node ในต้นไม้

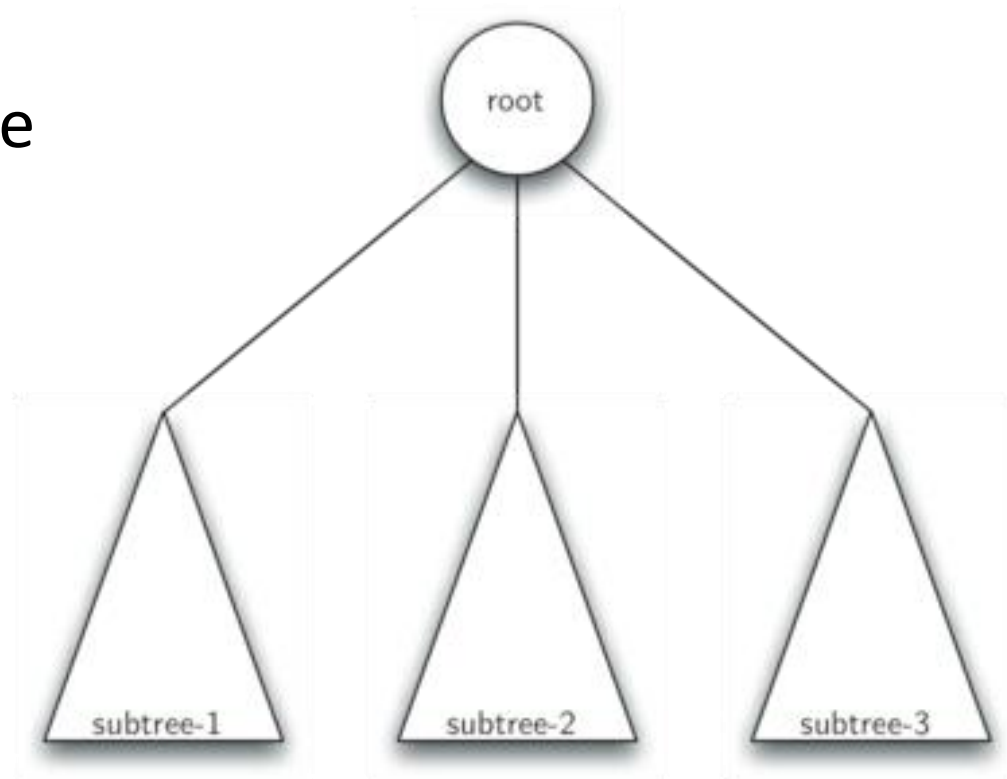
นิยาม 1

- **tree** ประกอบด้วย **set** ของ **node** และ **edge** ซึ่งเชื่อมคู่ของ **node** ซึ่ง **tree** มีคุณสมบัติดังนี้
 - ต้องมี **1 node** เป็น **root**
 - ทุก **node n** ยกเว้น **root** ต้องมี **edge** เชื่อมกับ **node p** ที่เป็น **parent** ของมัน
 - มีแค่ **path** เดียวจาก **root** ถึงแต่ละ **node**
 - ถ้าทุก **node** มีจำนวน **children** ไม่เกิน **2** จะเรียกว่า ต้นไม้ทวิภาค (**binary tree**)



นิยาม 2

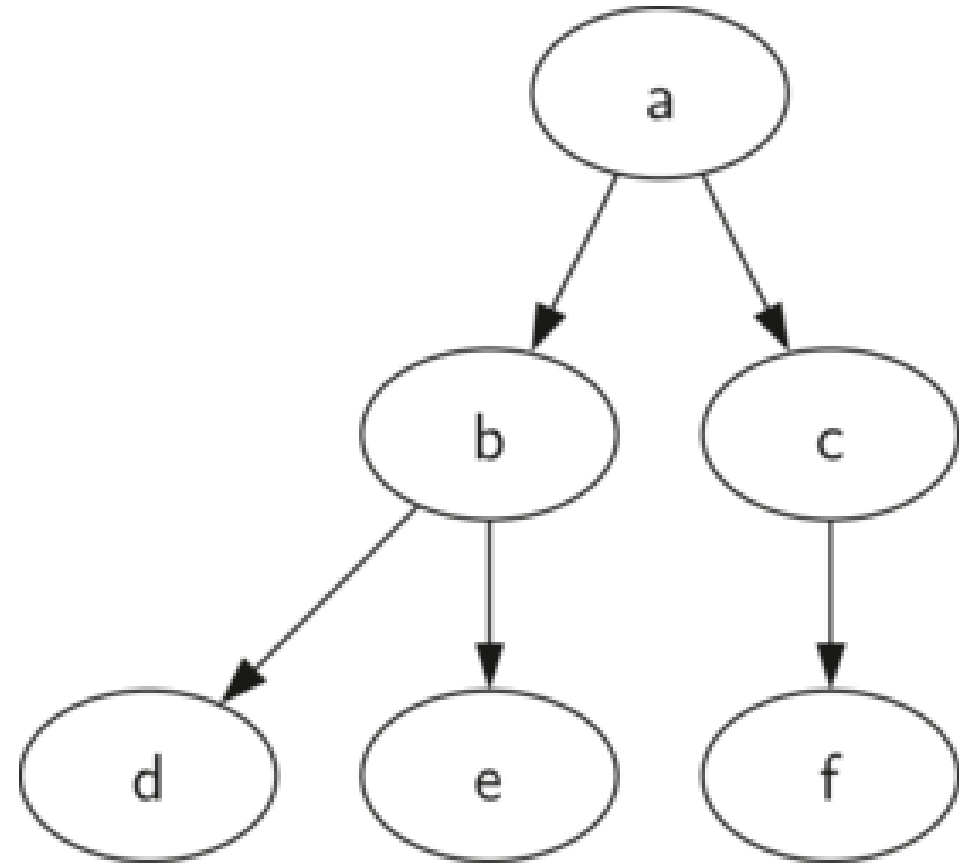
- **tree** จะว่างหรือประกอบด้วย **root** ที่มีหรือไม่มี **subtree** ก็ถือว่าเป็น **tree**
- **root** ของแต่ละ **subtree** ที่เชื่อมต่อกับ **parent** ของ **subtree** ด้วย **edge**
 - เป็นความสัมพันธ์แบบเวียนเกิด
- **tree** ในรูปมีอย่างน้อย 4 node



สร้าง tree ด้วย list ของ list

- แต่ละ node คือ list
 - เก็บค่าไว้ที่ item แรกของ list
 - item ที่ 2 เก็บ list ของ subtree ซ้าย
 - item ที่ 3 เก็บ list ของ subtree ขวา

```
myTree = ['a',  #root
          ['b',  #left subtree
            ['d', [], []],
            ['e', [], []] ],
          ['c',  #right subtree
            ['f', [], []],
            [] ]
        ]
```



การใช้งาน

- `myTree[0]` คือ ค่าที่ **root**
- `myTree[1]` คือ **subtree** ซ้าย
- `myTree[2]` คือ **subtree** ขวา
- เป็นโครงสร้างแบบความสัมพันธ์เวียนเกิด
- ไม่จำกัดว่าจะต้องเป็น **binary tree**

```
myTree = ['a', ['b', ['d',[],[]], ['e',[],[]] ], ['c', ['f',[],[]], [] ]  
print(myTree)  
print('left subtree = ', myTree[1])  
print('root = ', myTree[0])  
print('right subtree = ', myTree[2])
```

```
def BinaryTree(r):  
    return [r, [], []]  
  
def getRootVal(root):  
    return root[0]  
  
def setRootVal(root,newVal):  
    root[0] = newVal  
  
def getLeftChild(root):  
    return root[1]  
  
def getRightChild(root):  
    return root[2]
```

```
def insertLeft(root,newBranch):  
    t = root.pop(1)  
    if len(t) > 1:  
        root.insert(1,[newBranch,t,[]])  
    else:  
        root.insert(1,[newBranch, [], []])  
    return root  
  
def insertRight(root,newBranch):  
    t = root.pop(2)  
    if len(t) > 1:  
        root.insert(2,[newBranch,[],t])  
    else:  
        root.insert(2,[newBranch,[],[]])  
    return root
```

```
r = BinaryTree(3)
insertLeft(r,4)
insertLeft(r,5)
insertRight(r,6)
insertRight(r,7)
l = getLeftChild(r)
print(l)
```

```
setRootVal(l,9)
print(r)
insertLeft(l,11)
print(r)
print(getRightChild(getRightChild(r)))
```

```
[5, [4, [], []], []]
[3, [9, [4, [], []], []], [7, [], [6, [], []]]]
[3, [9, [11, [4, [], []], []], []], [7, [], [6,
[], []]]]
[6, [], []]
```

ฮีพทวิภาค

Binary heap

คิวแบบมีลำดับความสำคัญ (priority queue)

- queue ปกติจะมีลำดับตามการเข้า
- priority queue มีการปรับลำดับตามความสำคัญ
 - ความสำคัญสูงอยู่ที่หัว queue
 - ความสำคัญต่ำอยู่ที่ปลาย queue
- เมื่อมีการ enqueue item จะถูกจัดไปอยู่ในลำดับตามความสำคัญ
- ง่ายสุดในการสร้างคือ insert ด้วย $O(n)$ แล้วจัดลำดับความสำคัญด้วยการ sort ใช้เวลา $O(n \log n)$
- ดีกว่านั้นคือใช้ ฮีพทวิภาค
 - enqueue ใช้ $O(\log n)$
 - dequeue ใช้ $O(\log n)$

ฮีพทวิภาค

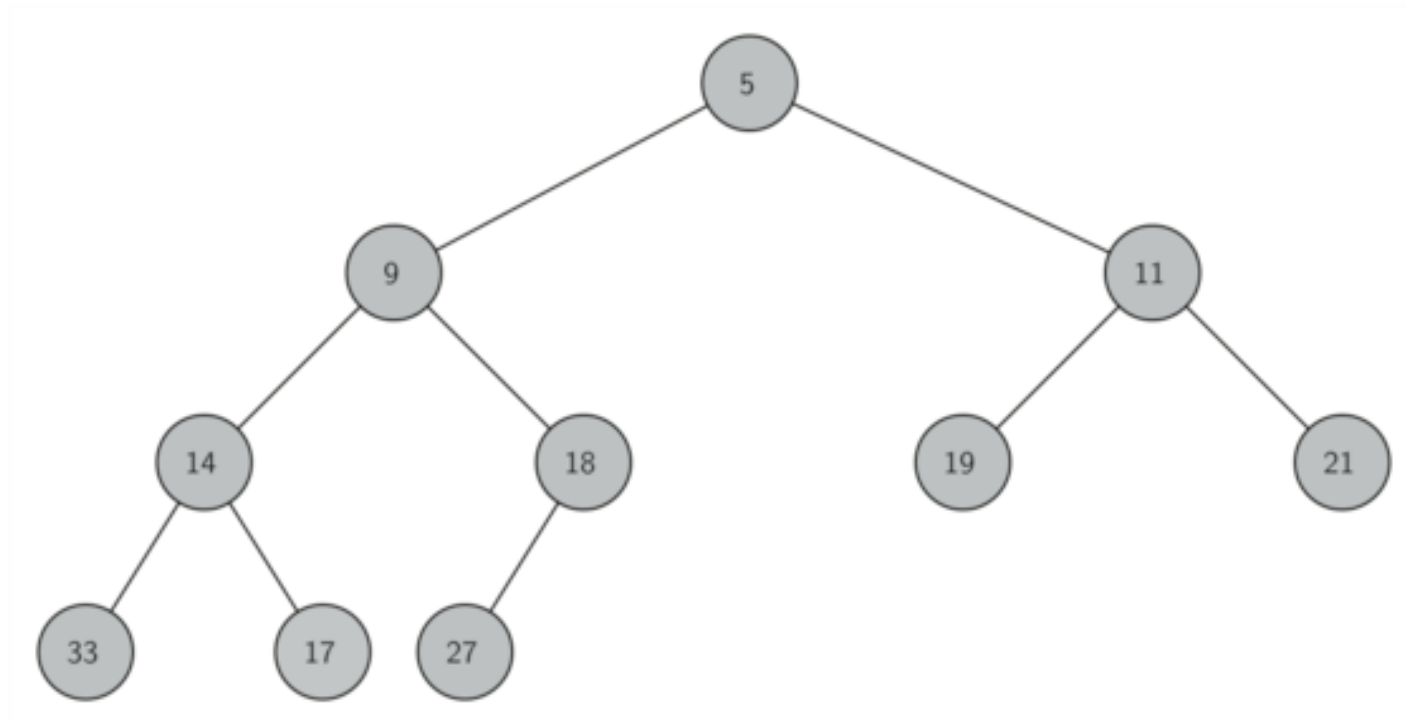
- มีความคล้ายกับต้นไม้
- การสร้างจริงๆ ใช้แค่ **list**
- มี 2 ประเภทคือ
 - **max heap** ความสำคัญสูงสุดคือมีค่า **key** มากสุด
 - **min heap** ความสำคัญสูงสุดคือมีค่า **key** น้อยสุด

method ของฮีพทวิภาค

- **BinaryHeap()** สร้างฮีพว่าง
- **insert(k)** เพิ่ม **item** ใน ฮีพ
- **findMin()** คืนค่า **item** ที่มีค่า **key** น้อยที่สุด
- **delMin()** คืนค่า **item** ที่มีค่า **key** น้อยที่สุดแล้วลบออกด้วย
- **isEmpty()** ถามว่าฮีพว่างหรือไม่
- **size()** คืนค่าจำนวน **item** ในฮีพ
- **buildHeap(list)** สร้างฮีพใหม่จาก **list**

โครงสร้าง

- คล้ายต้นไม้ที่รักษาความสมดุล
 - ปม (**node**) ฝั่งซ้ายและฝั่งขวามีจำนวนเท่ากัน
 - ใช้ต้นไม้สมบูรณ์
 - แต่ละลำดับชั้นมี **node** เต็ม
 - ชั้นสุดท้ายเรียกว่าใบเรียงวางจากซ้ายไปขวา

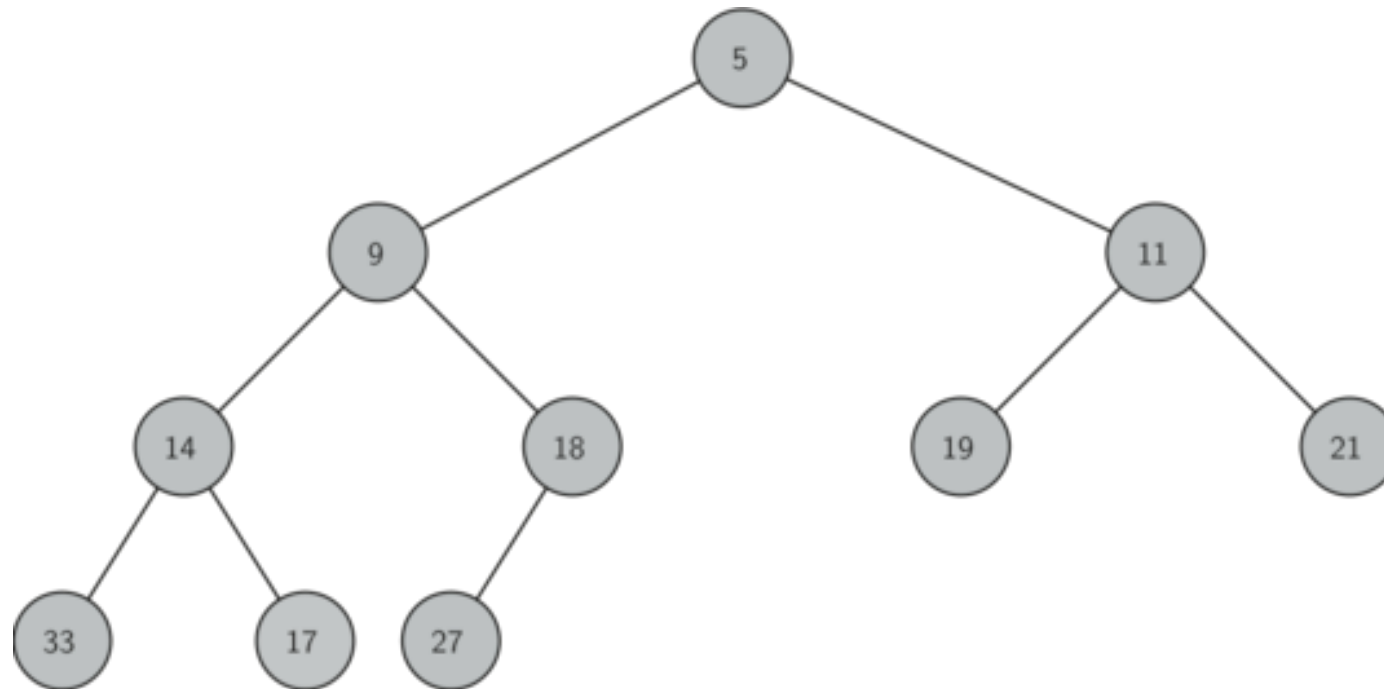


ต้นไม้สมบูรณ์

- สร้างได้ด้วย list
- ที่ parent node p
 - left child อยู่ที่ตำแหน่ง $2p$
 - right child อยู่ที่ตำแหน่ง $2p + 1$
- ที่ node p ใดๆ parent อยู่ที่ $p // 2$

คุณสมบัติของฮีพ

- key ที่ parent มีค่าน้อยกว่าหรือเท่ากับ children เสมอ



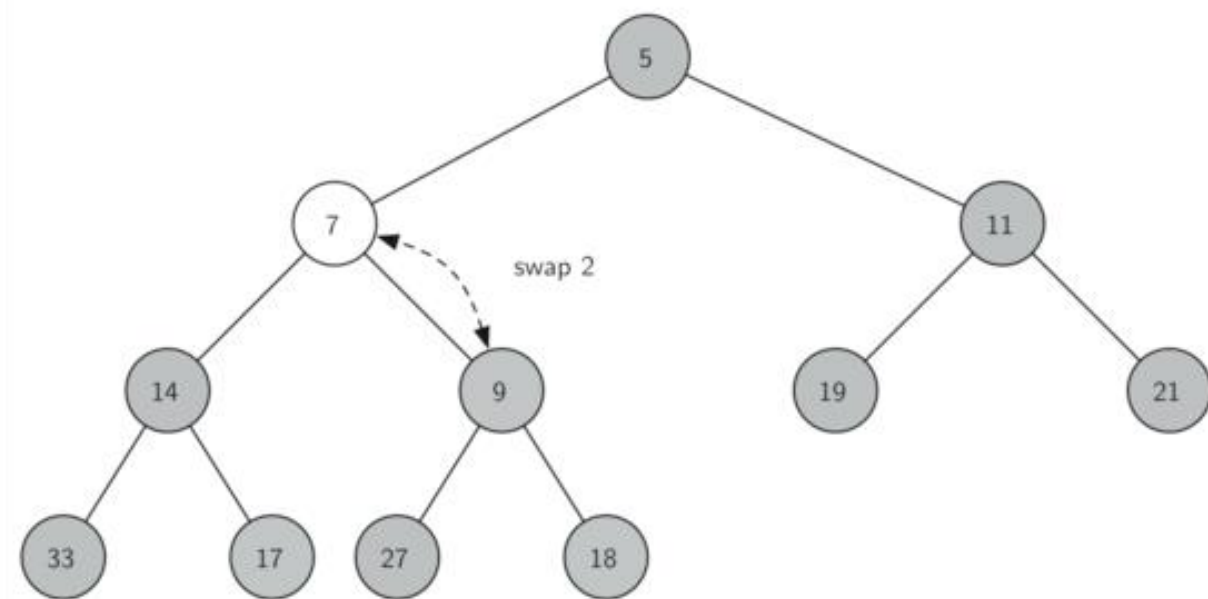
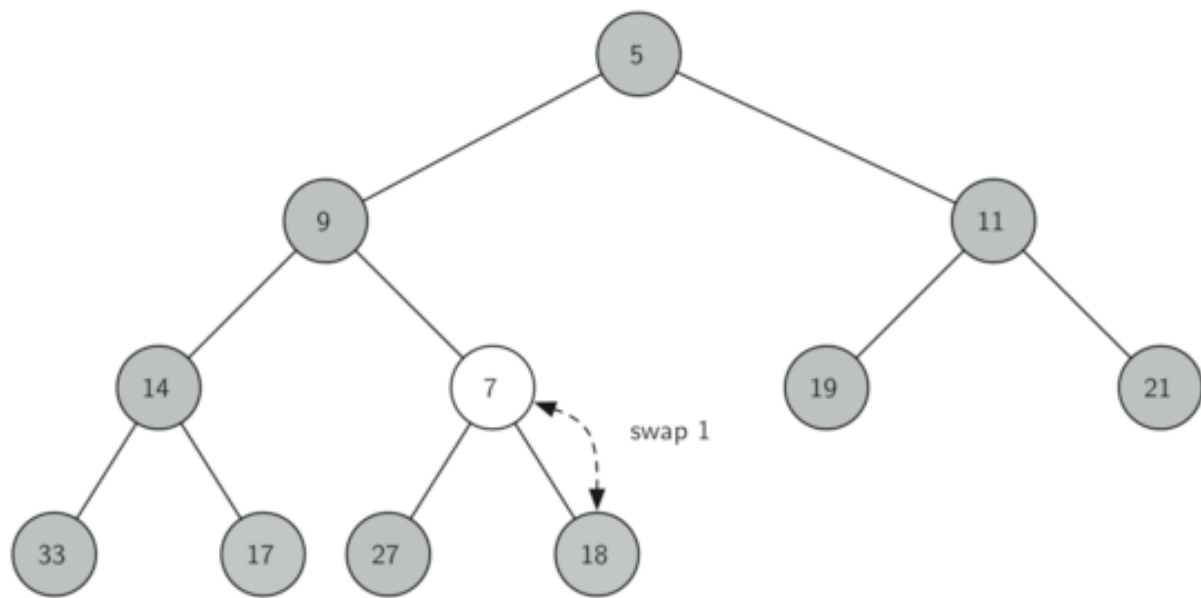
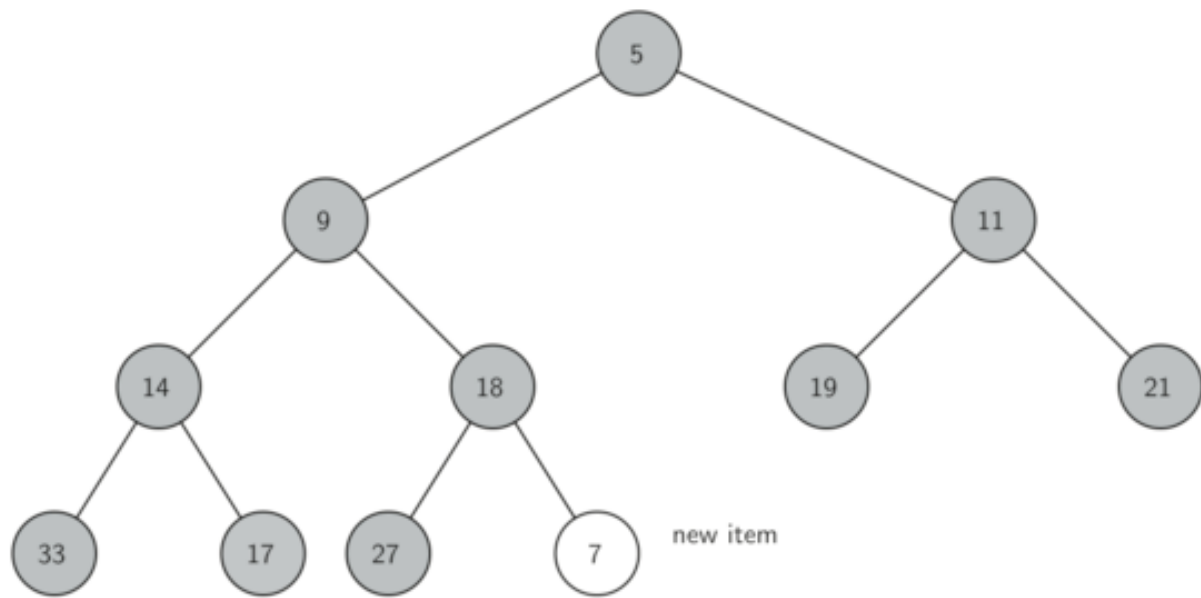
0	5	9	11	14	18	19	21	33	17	27	
0	1	2	3	4	5	6	7	8	9	10	11

การสร้าง class ของฮีพ

```
class BinHeap:  
    def __init__(self):  
        self.heapList = [0]  
        self.currentSize = 0
```

insert

- ทำโดย **append item** เข้าไปใน **list**
- รักษาสภาพความสมบูรณ์เสมอ
- ดัน **item** กลับขึ้นด้านบนเพื่อให้อยู่ในลำดับที่ถูกต้อง (**percolate up**)
 - **parent** มี **key** น้อยกว่าหรือเท่ากับ **children** เสมอ

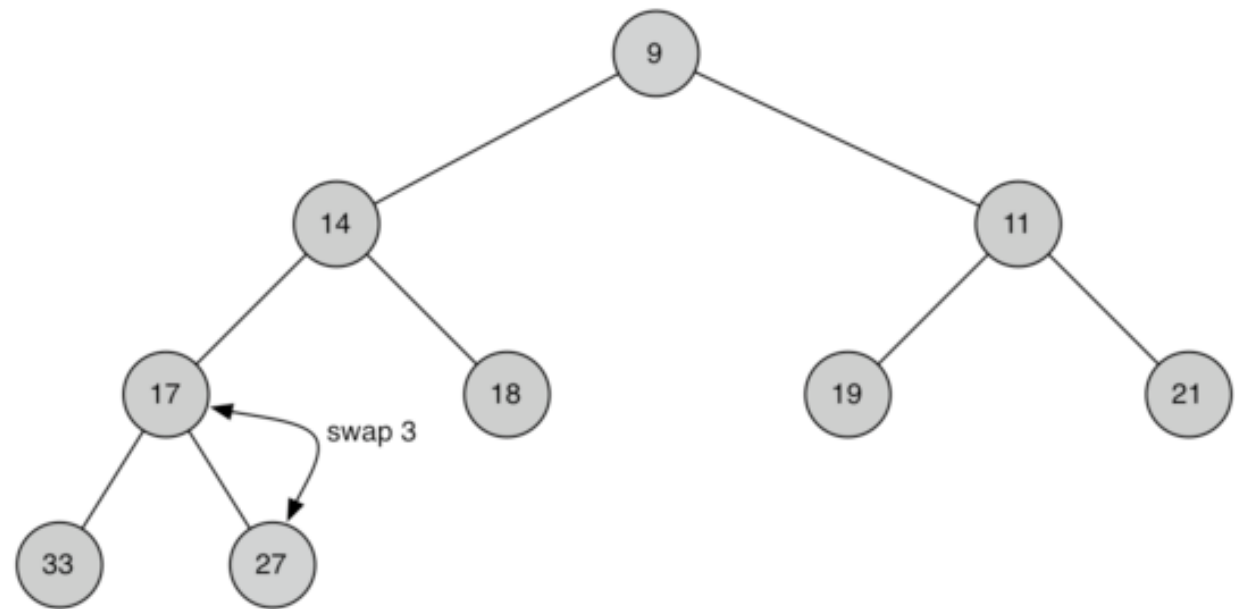
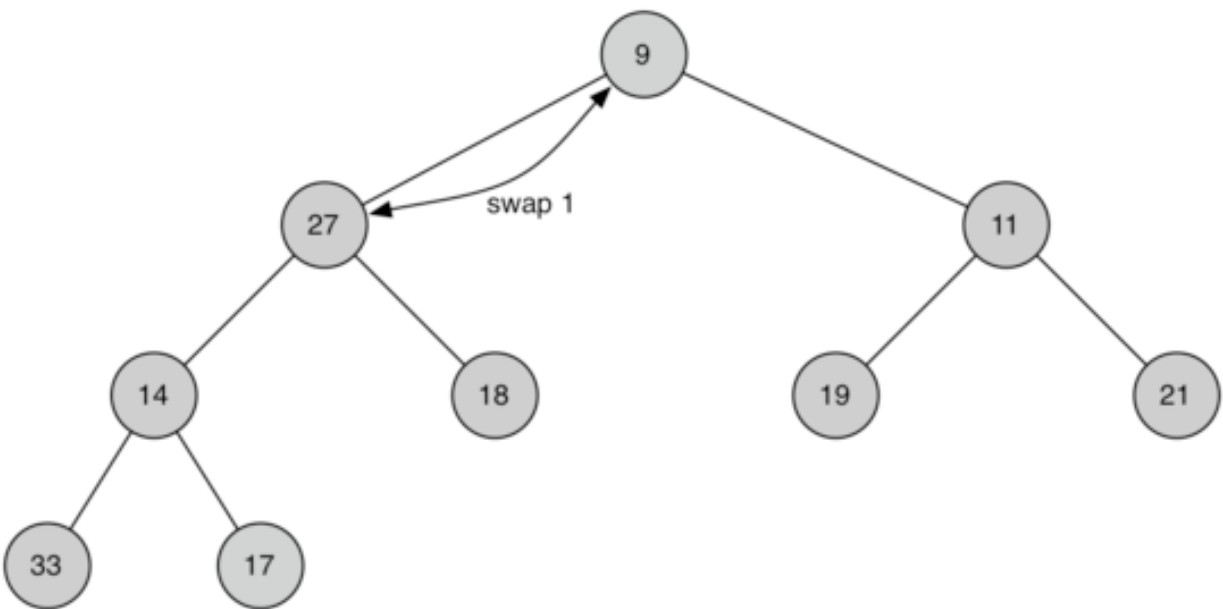
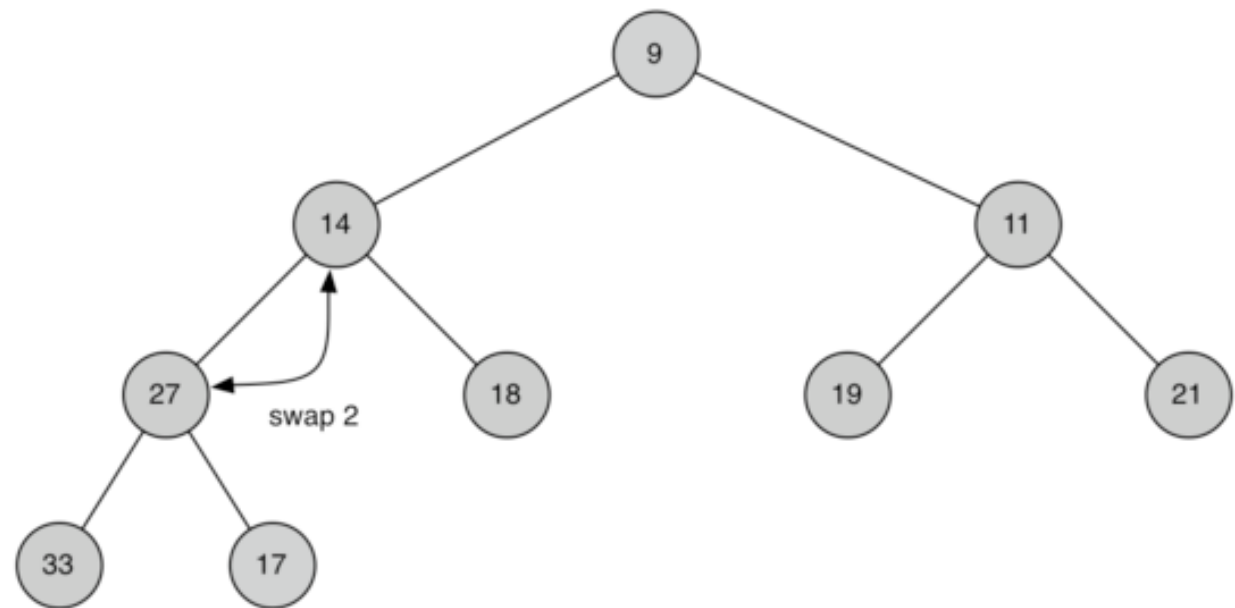
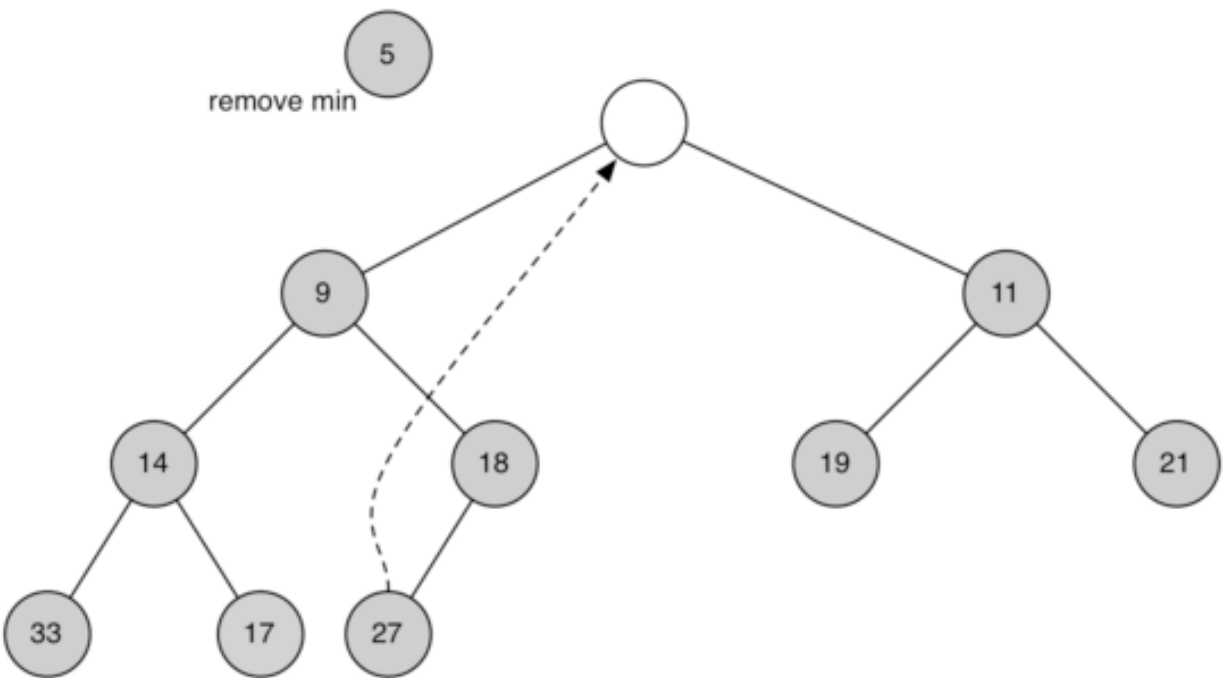


```
def percUp(self,i):  
    while i // 2 > 0:  
        if self.heapList[i] < self.heapList[i // 2]:  
            tmp = self.heapList[i // 2]  
            self.heapList[i // 2] = self.heapList[i]  
            self.heapList[i] = tmp  
        i = i // 2
```

```
def insert(self,k):  
    self.heapList.append(k)  
    self.currentSize = self.currentSize + 1  
    self.percUp(self.currentSize)
```

delMin

- ต้องหา **node** ที่มี **key** น้อยสุดขึ้นมาแทนที่
- เอา **node** สุดท้ายขึ้นมาก่อน
- ไต่กลับลงไปตำแหน่งที่ (**percolate down**)



```
def percDown(self,i):
```

```
    while (i * 2) <= self.currentSize:
```

```
        mc = self.minChild(i)
```

```
        if self.heapList[i] > self.heapList[mc]:
```

```
            tmp = self.heapList[i]
```

```
            self.heapList[i] = self.heapList[mc]
```

```
            self.heapList[mc] = tmp
```

```
    i = mc
```

```
def minChild(self,i):
```

```
    if i * 2 + 1 > self.currentSize:
```

```
        return i * 2
```

```
    else:
```

```
        if self.heapList[i*2] < self.heapList[i*2+1]:
```

```
            return i * 2
```

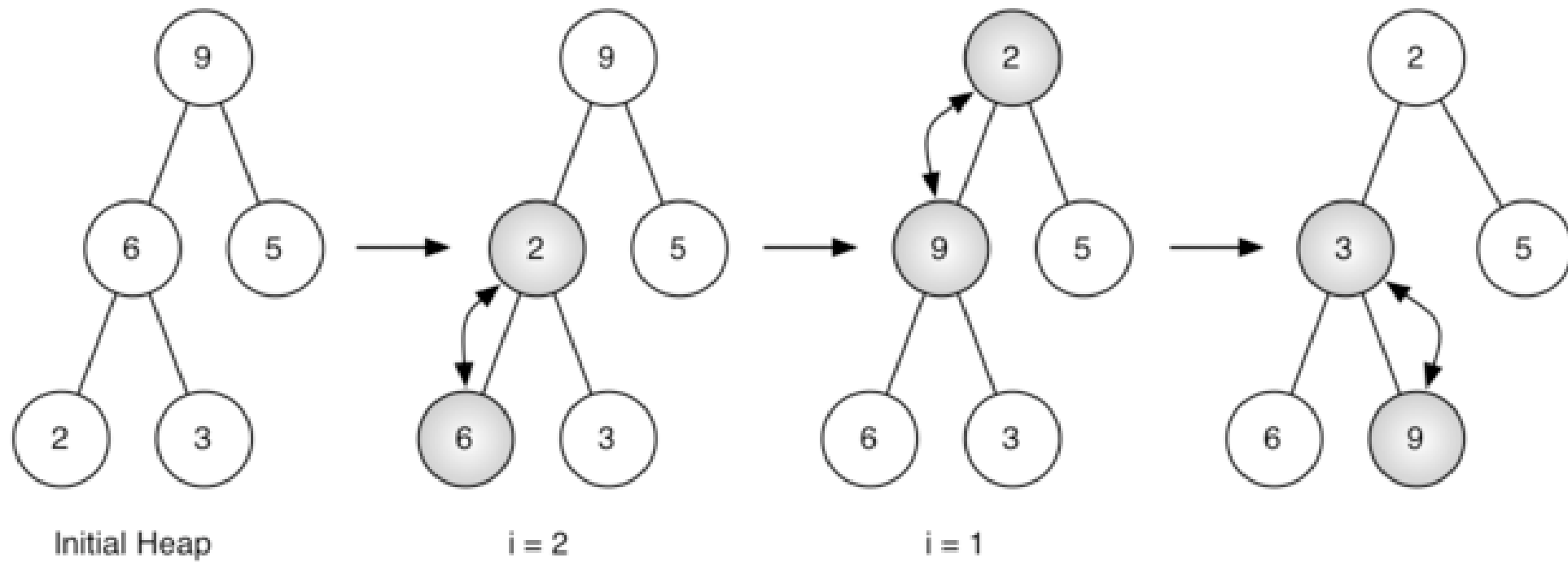
```
        else:
```

```
            return i * 2 + 1
```

```
def delMin(self):  
    retval = self.heapList[1]  
    self.heapList[1] = self.heapList[self.currentSize]  
    self.currentSize = self.currentSize - 1  
    self.heapList.pop()  
    self.percDown(1)  
    return retval
```

สร้างฮีพจาก list

- ง่ายสุดคือค่อยๆ insert ทีละค่าใช้ $O(n \log n)$
- สร้างจาก list ทั้งหมดใช้ $O(n)$
- เริ่มจากครึ่งของ list
 - อีกครึ่งที่เหลือเป็น leaf node ที่ไม่มี child
- ขยับไปข้างหน้า list เรื่อยๆ (วิ่งขึ้นบนของ tree) แล้วไถ่กลับลงมา



$i = 2$ $[0, 9, 5, 6, 2, 3]$

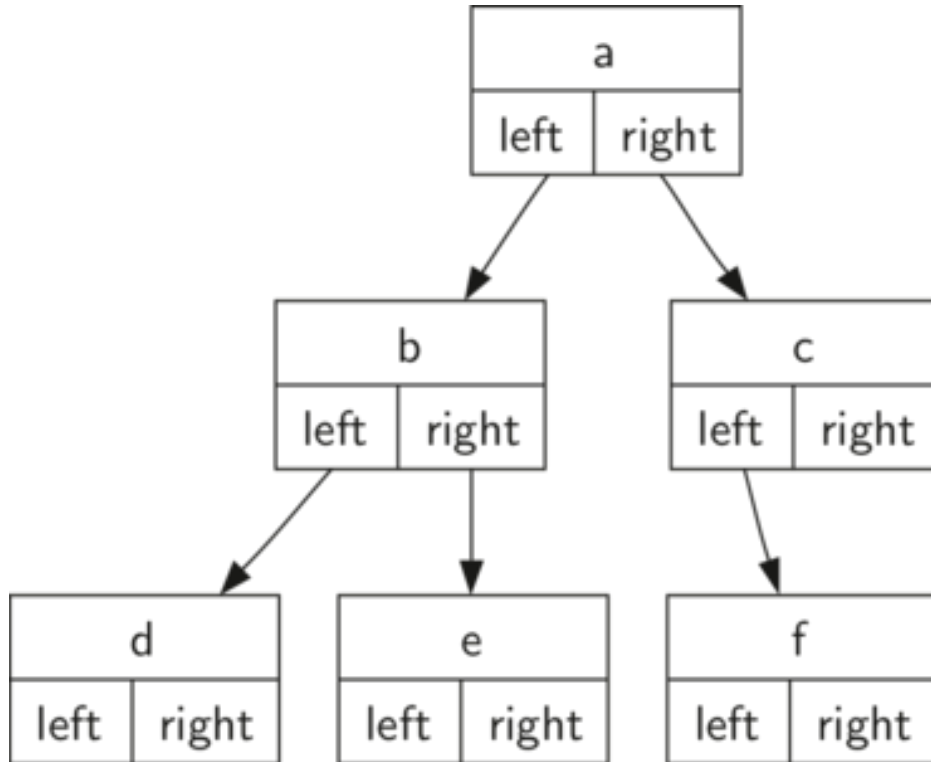
$i = 1$ $[0, 9, 2, 6, 5, 3]$

$i = 0$ $[0, 2, 3, 6, 5, 9]$


```
def buildHeap(self,alist):  
    i = len(alist) // 2  
    self.currentSize = len(alist)  
    self.heapList = [0] + alist[:]  
    while (i > 0):  
        self.percDown(i)  
        i = i - 1
```

ปมและการอ้างอิง
(nodes and references)

root และ subtree ซ้าย ขวา



```
class BinaryTree:
```

```
    def __init__(self, rootObj):
```

```
        self.key = rootObj
```

```
        self.leftChild = None
```

```
        self.rightChild = None
```

insertLeft และ insertRight

```
def insertLeft(self, newNode):  
    if self.leftChild == None:  
        self.leftChild = BinaryTree(newNode)  
    else:  
        t = BinaryTree(newNode)  
        t.leftChild = self.leftChild  
        self.leftChild = t
```

```
def insertRight(self, newNode):  
    if self.rightChild == None:  
        self.rightChild = BinaryTree(newNode)  
    else:  
        t = BinaryTree(newNode)  
        t.rightChild = self.rightChild  
        self.rightChild = t
```

อธิบายเป็นภาพการทำงานได้อย่างไร?

method อื่นๆ

```
def getRightChild(self):  
    return self.rightChild
```

```
def getLeftChild(self):  
    return self.leftChild
```

```
def setRootVal(self,obj):  
    self.key = obj
```

```
def getRootVal(self):  
    return self.key
```

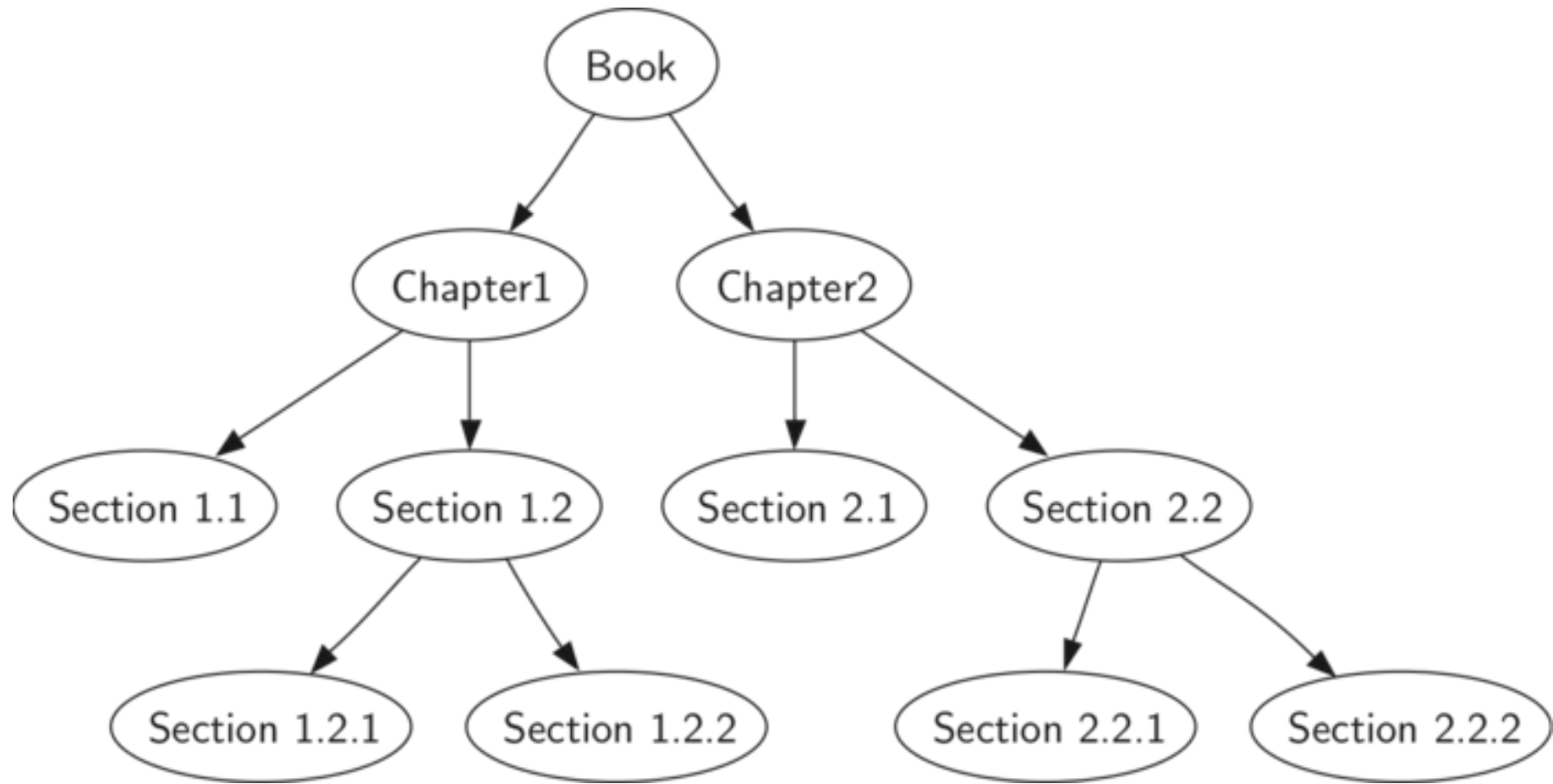
```
1 class BinaryTree:
2     def __init__(self, rootObj):
3         self.key = rootObj
4         self.leftChild = None
5         self.rightChild = None
6
7     def insertLeft(self, newNode):
8         if self.leftChild == None:
9             self.leftChild = BinaryTree(newNode)
10        else:
11            t = BinaryTree(newNode)
12            t.leftChild = self.leftChild
13            self.leftChild = t
14
15    def insertRight(self, newNode):
16        if self.rightChild == None:
17            self.rightChild = BinaryTree(newNode)
18        else:
19            t = BinaryTree(newNode)
20            t.rightChild = self.rightChild
21            self.rightChild = t
22
23
```

```
24 def getRightChild(self):
25     return self.rightChild
26
27 def getLeftChild(self):
28     return self.leftChild
29
30 def setRootVal(self, obj):
31     self.key = obj
32
33 def getRootVal(self):
34     return self.key
35
36
37 r = BinaryTree('a')
38 print(r.getRootVal())
39 print(r.getLeftChild())
40 r.insertLeft('b')
41 print(r.getLeftChild())
42 print(r.getLeftChild().getRootVal())
43 r.insertRight('c')
44 print(r.getRightChild())
45 print(r.getRightChild().getRootVal())
46 r.getRightChild().setRootVal('hello')
47 print(r.getRightChild().getRootVal())
48
```

การเดินทางในต้นไม้ (tree traversal)

การเดินทาง 3 แบบ

- preorder ลำดับก่อน
 - root $\rightarrow$ left subtree $\rightarrow$ right subtree
- postorder ลำดับหลัง
 - left subtree $\rightarrow$ right subtree $\rightarrow$ root
- inorder ในลำดับ
 - left subtree $\rightarrow$ root $\rightarrow$ right subtree



ควรจะเดินทางแบบไหน?

preorder

```
def preorder(tree):  
    if tree:  
        print(tree.getRootVal())  
        preorder(tree.getLeftChild())  
        preorder(tree.getRightChild())
```

แบบ function แยก

```
def preorder(self):  
    print(self.key)  
    if self.leftChild:  
        self.leftChild.preorder()  
    if self.rightChild:  
        self.rightChild.preorder()
```

แบบ method

```
def postorder(tree):
```

```
    if tree != None:
```

```
        postorder(tree.getLeftChild())
```

```
        postorder(tree.getRightChild())
```

```
    print(tree.getRootVal())
```

postorder

```
def postordereval(tree):
```

```
   opers = {'+':operator.add, '-':operator.sub, '*':operator.mul, '/':operator.truediv}
```

```
    res1 = None
```

```
    res2 = None
```

```
    if tree:
```

```
        res1 = postordereval(tree.getLeftChild())
```

```
        res2 = postordereval(tree.getRightChild())
```

```
        if res1 and res2:
```

```
            return opers[tree.getRootVal()](res1,res2)
```

```
        else:
```

```
            return tree.getRootVal()
```

inorder

```
def inorder(tree):  
    if tree != None:  
        inorder(tree.getLeftChild())  
        print(tree.getRootVal())  
        inorder(tree.getRightChild())
```

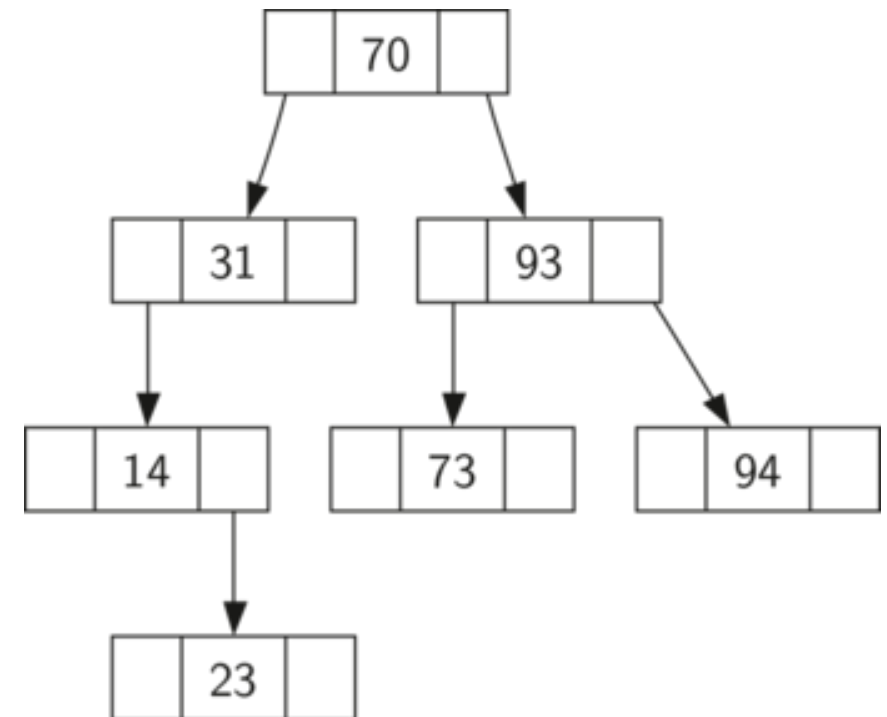
```
def printexp(tree):  
    sVal = ""  
    if tree:  
        sVal = '(' + printexp(tree.getLeftChild())  
        sVal = sVal + str(tree.getRootVal())  
        sVal = sVal + printexp(tree.getRightChild())+')'  
    return sVal
```

ต้นไม้ค้นหาแบบทวิภาค
(binary search tree)

คุณสมบัติ

- ทุก **key** ของ **left subtree** จะต้องค่าน้อยกว่า **key** ของ **root** เสมอ
- ทุก **key** ของ **right subtree** จะต้องค่ามากกว่า **key** ของ **root** เสมอ
- เราจะสร้าง **search tree** สำหรับ **map**

70,31,93,94,14,23,73



```
class BinarySearchTree:
```

```
    def __init__(self):  
        self.root = None  
        self.size = 0
```

```
    def length(self):  
        return self.size
```

```
    def __len__(self):  
        return self.size
```

```
class TreeNode:
```

```
    def __init__(self, key, val, left=None, right=None,  
                  parent=None):
```

```
        self.key = key  
        self.payload = val  
        self.leftChild = left  
        self.rightChild = right  
        self.parent = parent
```

```
    def hasLeftChild(self):  
        return self.leftChild
```

```
    def hasRightChild(self):  
        return self.rightChild
```

```
    def isLeftChild(self):  
        return self.parent and self.parent.leftChild == self
```

```
    def isRightChild(self):  
        return self.parent and self.parent.rightChild == self
```

```
    def isRoot(self):
```

```
        return not self.parent
```

```
    def isLeaf(self):
```

```
        return not (self.rightChild or self.leftChild)
```

```
    def hasAnyChildren(self):
```

```
        return self.rightChild or self.leftChild
```

```
    def hasBothChildren(self):
```

```
        return self.rightChild and self.leftChild
```

```
    def replaceNodeData(self, key, value, lc, rc):
```

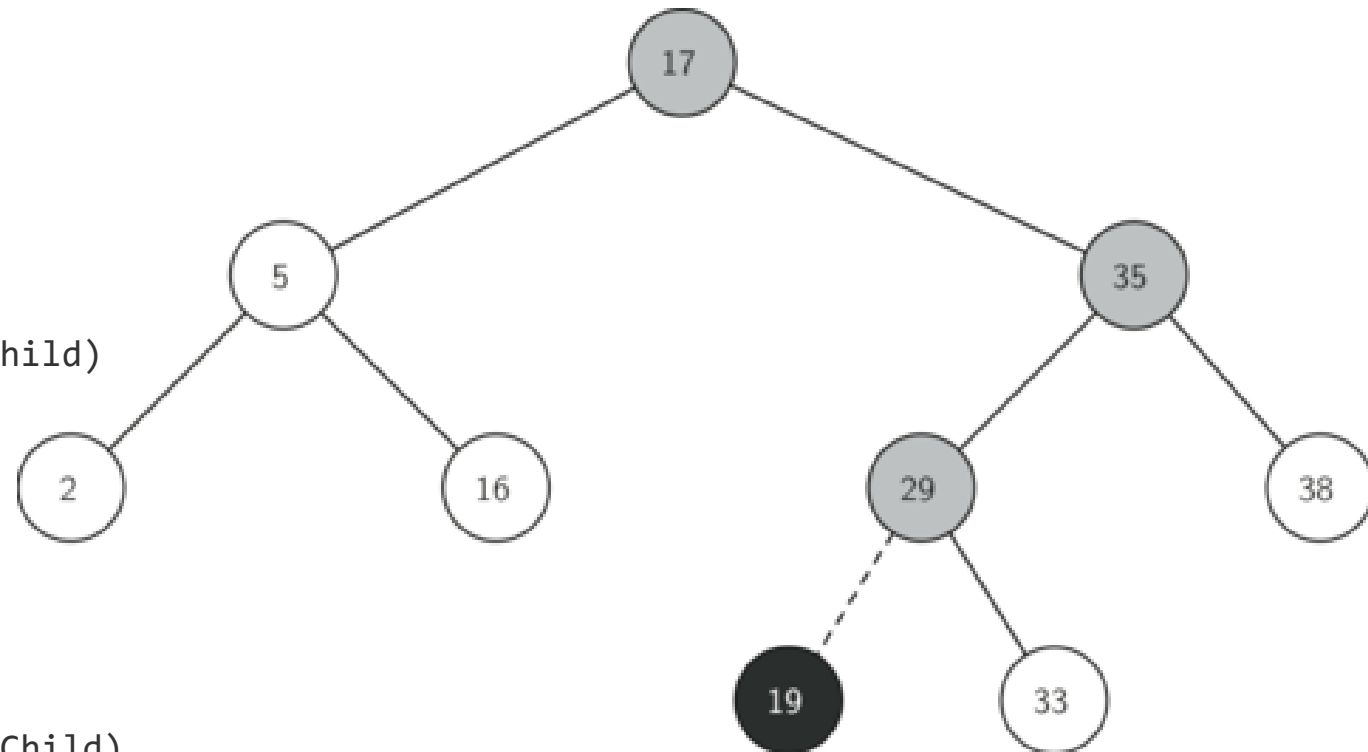
```
        self.key = key  
        self.payload = value  
        self.leftChild = lc  
        self.rightChild = rc  
        if self.hasLeftChild():  
            self.leftChild.parent = self  
        if self.hasRightChild():  
            self.rightChild.parent = self
```



```
def put(self, key, val):
    if self.root:
        self._put(key, val, self.root)
    else:
        self.root = TreeNode(key, val)
    self.size = self.size + 1
```

```
def _put(self, key, val, currentNode):
    if key < currentNode.key:
        if currentNode.hasLeftChild():
            self._put(key, val, currentNode.leftChild)
        else:
            currentNode.leftChild =
TreeNode(key, val, parent=currentNode)
    else:
        if currentNode.hasRightChild():
            self._put(key, val, currentNode.rightChild)
        else:
            currentNode.rightChild =
TreeNode(key, val, parent=currentNode)
```

```
def __setitem__(self, k, v):
    self.put(k, v)
```



```
def get(self, key):
    if self.root:
        res = self._get(key, self.root)
        if res:
            return res.payload
        else:
            return None
    else:
        return None

def _get(self, key, currentNode):
    if not currentNode:
        return None
    elif currentNode.key == key:
        return currentNode
    elif key < currentNode.key:
        return self._get(key, currentNode.leftChild)
    else:
        return self._get(key, currentNode.rightChild)
```

```
def __getitem__(self, key):
    return self.get(key)

def __contains__(self, key):
    if self._get(key, self.root):
        return True
    else:
        return False

if 'Northfield' in myZipTree:
    print("oom ya ya")
```

```

def delete(self, key):
    if self.size > 1:
        nodeToRemove = self._get(key, self.root)
        if nodeToRemove:
            self.remove(nodeToRemove)
            self.size = self.size - 1
        else:
            raise KeyError('Error, key not in tree')
    elif self.size == 1 and self.root.key == key:
        self.root = None
        self.size = self.size - 1
    else:
        raise KeyError('Error, key not in tree')

def __delitem__(self, key):
    self.delete(key)

```

- remove มี 3 กรณี
 - node ที่ต้องการ remove ไม่มีลูก
 - node ที่ต้องการ remove มีลูก 1 node
 - node ที่ต้องการ remove มีลูก 2 nodes

```
if currentNode.isLeaf():
```

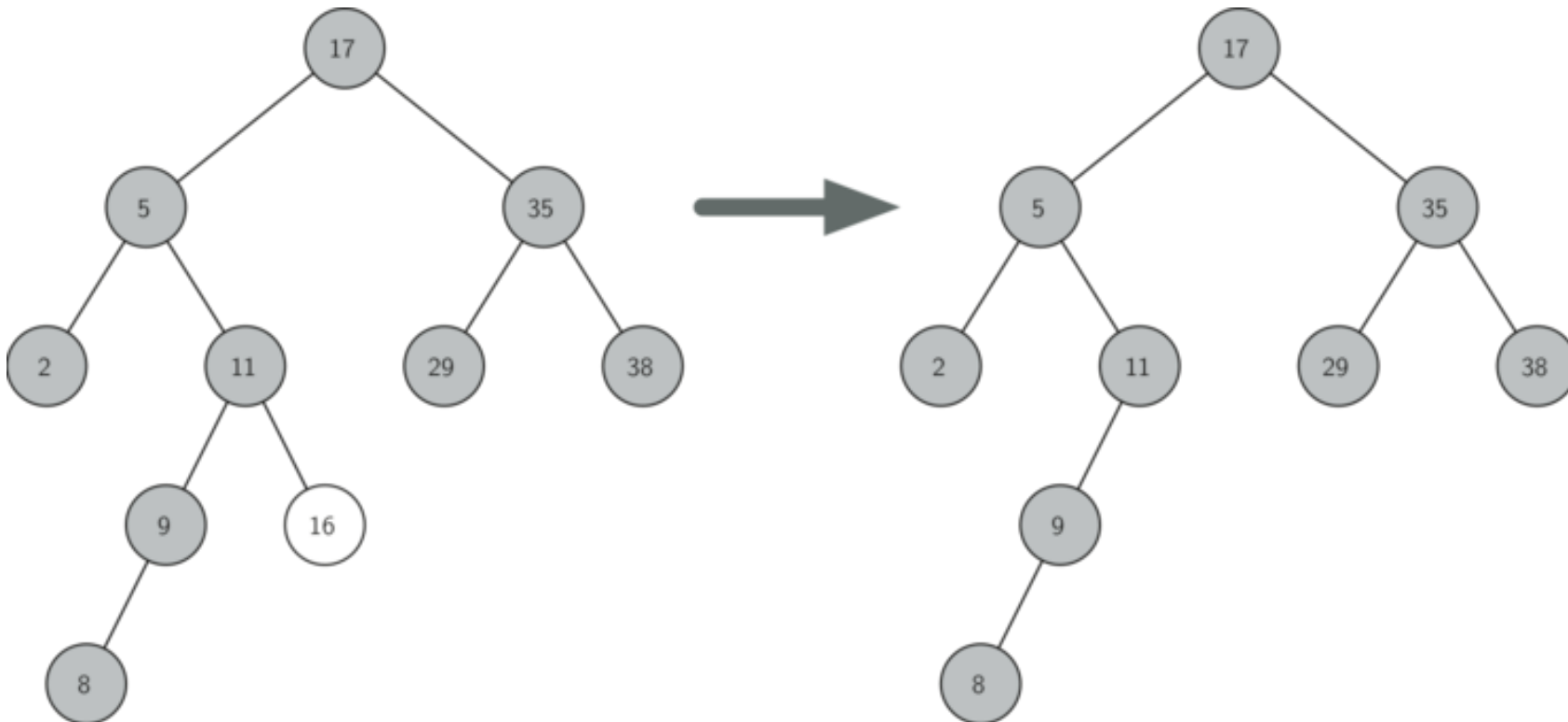
```
    if currentNode == currentNode.parent.leftChild:
```

```
        currentNode.parent.leftChild = None
```

```
    else:
```

```
        currentNode.parent.rightChild = None
```

กรณีเป็น leaf node



else: *# this node has one child*

if currentNode.hasLeftChild():

if currentNode.isLeftChild():

currentNode.leftChild.parent = currentNode.parent

currentNode.parent.leftChild = currentNode.leftChild

elif currentNode.isRightChild():

currentNode.leftChild.parent = currentNode.parent

currentNode.parent.rightChild = currentNode.leftChild

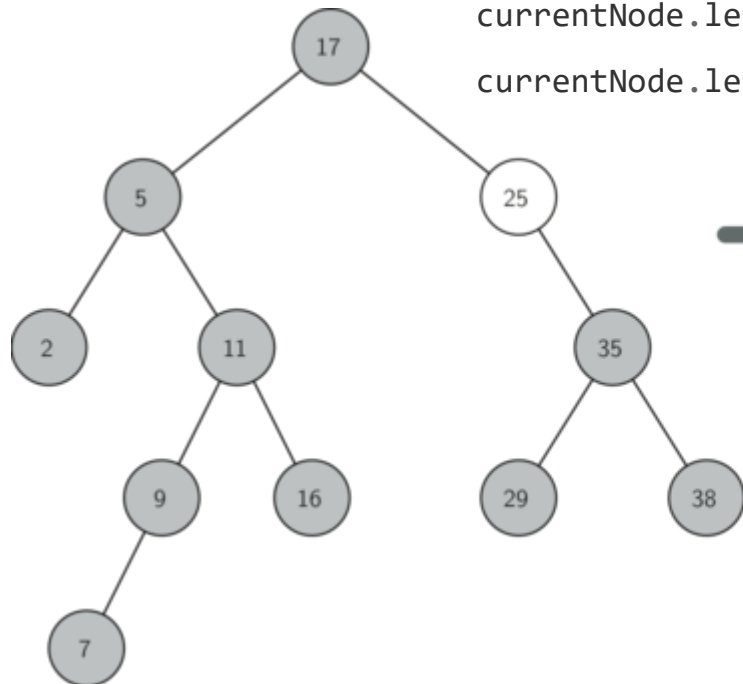
else:

currentNode.replaceNodeData(currentNode.leftChild.key,

currentNode.leftChild.payload,

currentNode.leftChild.leftChild,

currentNode.leftChild.rightChild)



else:

if currentNode.isLeftChild():

currentNode.rightChild.parent = currentNode.parent

currentNode.parent.leftChild = currentNode.rightChild

elif currentNode.isRightChild():

currentNode.rightChild.parent = currentNode.parent

currentNode.parent.rightChild = currentNode.rightChild

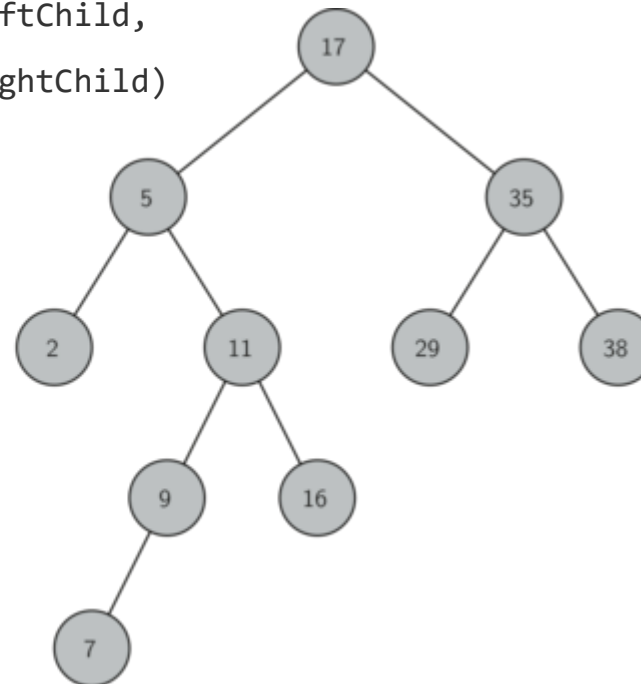
else:

currentNode.replaceNodeData(currentNode.rightChild.key,

currentNode.rightChild.payload,

currentNode.rightChild.leftChild,

currentNode.rightChild.rightChild)



กรณีมีลูก **1** ให้เอาลูกมาแทน
ตำแหน่งตัวเอง

กรณีมีลูก 2

- หาตัวแทนที่ยังรักษาคุณสมบัติของ **search tree**
 - ทุก **key** ของ **left subtree** จะต้องน้อยกว่า **key** ของ **root** เสมอ
 - ทุก **key** ของ **right subtree** จะต้องมากกว่า **key** ของ **root** เสมอ
- ตัวแทนคือ **node** ที่มี **key** มากเป็นลำดับถัดไปจาก **key** ของ **node** ที่ต้องการ **remove** (**successor**)
- ตัวแทนจะมีลูกไม่เกิน **1** ตัว
 - เก็บตัวแทนไว้
 - **remove** ตัวแทนที่ตำแหน่งเก่า
 - ใส่ตัวแทนกลับเข้าไปที่ตำแหน่ง **node** ที่ต้องการ **remove**

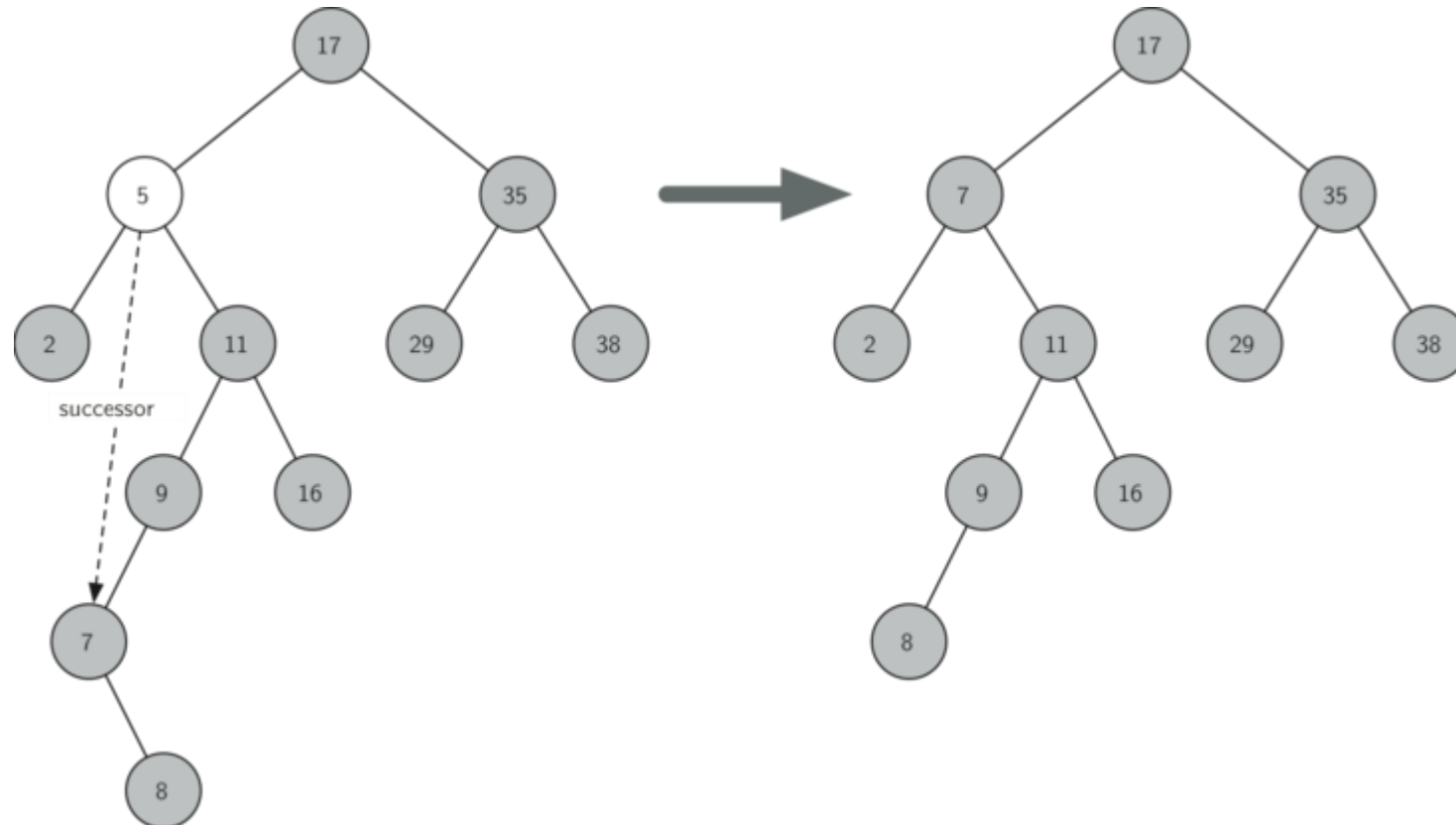
```
elif currentNode.hasBothChildren(): #interior
```

```
    succ = currentNode.findSuccessor()
```

```
    succ.spliceOut()
```

```
    currentNode.key = succ.key
```

```
    currentNode.payload = succ.payload
```



findSuccessor()

- พิจารณาในกรณีทั่วไป
- ถ้ามี **right child** แล้ว **successor** คือ **node** ที่มีค่า **key** น้อยสุดของ **right subtree**
- ถ้าไม่มี **right child** และตัวมันเองเป็น **left child** แล้ว **successor** คือ **parent**
- ถ้าไม่มี **right child** และตัวมันเองเป็น **right child** แล้ว **successor** คือ **successor** ของ **parent** ที่ไม่รวมตัวมันเอง


```
def findSuccessor(self):
    succ = None
    if self.hasRightChild():
        succ = self.rightChild.findMin()
    else:
        if self.parent:
            if self.isLeftChild():
                succ = self.parent
            else:
                self.parent.rightChild = None
                succ = self.parent.findSuccessor()
                self.parent.rightChild = self
    return succ

def findMin(self):
    current = self
    while current.hasLeftChild():
        current = current.leftChild
    return current
```

```
def spliceOut(self):
    if self.isLeaf():
        if self.isLeftChild():
            self.parent.leftChild = None
        else:
            self.parent.rightChild = None
    elif self.hasAnyChildren():
        if self.hasLeftChild():
            if self.isLeftChild():
                self.parent.leftChild = self.leftChild
            else:
                self.parent.rightChild = self.leftChild
                self.leftChild.parent = self.parent
        else:
            if self.isLeftChild():
                self.parent.leftChild = self.rightChild
            else:
                self.parent.rightChild = self.rightChild
            self.rightChild.parent = self.parent
```

ต้นไม้เอวีแอล

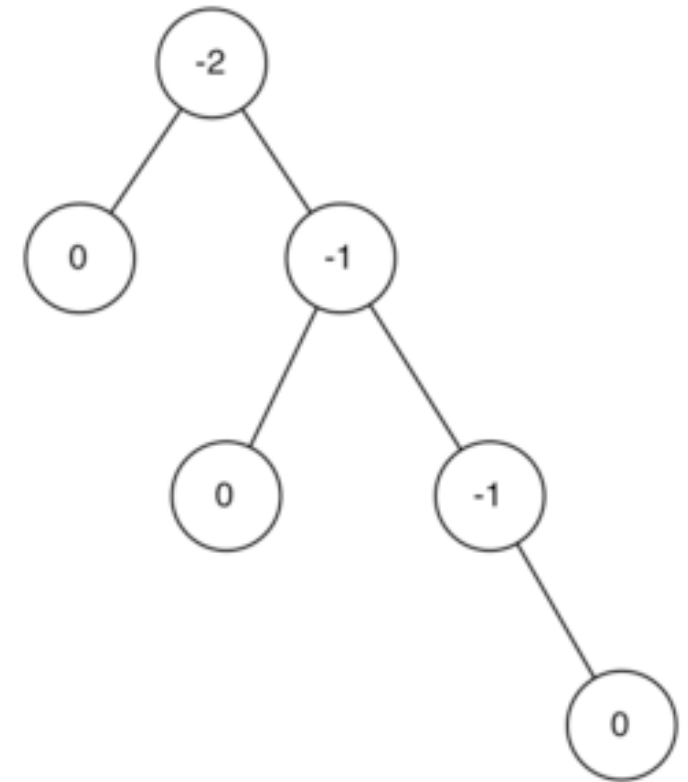
AVL Tree

ADT

- มี **method** เหมือนกับ **binary search tree**
- ต่างกันที่การทำงานข้างใน **method**
 - คำนึงถึงค่าความสมดุล (**balance factor**)

Balance factor

- $\text{balanceFactor} = \text{height}(\text{leftSubTree}) - \text{height}(\text{rightSubTree})$
- balance factor > 0 ต้นไม้หนักไปทางฝั่งซ้าย
- balance factor < 0 ต้นไม้หนักไปทางฝั่งขวา
- balance factor $= 0$ ต้นไม้สมดุลแบบสมบูรณ์
- AVL tree
 - balance factor = -1, 0, 1
 - ถ้าหลุดออกจากค่าพวกนี้ต้องทำการแก้ไข
 - ทำงานได้เร็วขึ้น



เพิ่ม node

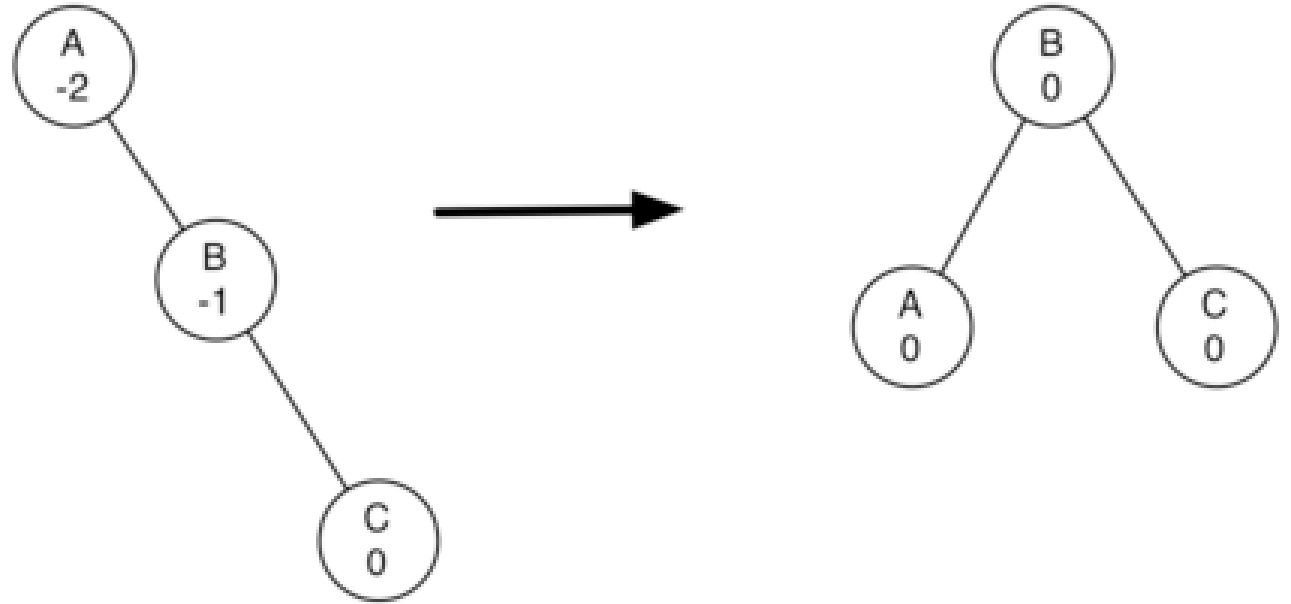
- เพิ่ม left node
 - parent และ ต้นตระกูลมีค่า balance factor เพิ่ม 1
- เพิ่ม right node
 - parent และ ต้นตระกูลมีค่า balance factor ลด 1

```
def _put(self, key, val, currentNode):  
    if key < currentNode.key:  
        if currentNode.hasLeftChild():  
            self._put(key, val, currentNode.leftChild)  
        else:  
            currentNode.leftChild = TreeNode(key, val, parent=currentNode)  
            self.updateBalance(currentNode.leftChild)  
    else:  
        if currentNode.hasRightChild():  
            self._put(key, val, currentNode.rightChild)  
        else:  
            currentNode.rightChild = TreeNode(key, val, parent=currentNode)  
            self.updateBalance(currentNode.rightChild)
```

```
def updateBalance(self,node):
    if node.balanceFactor > 1 or node.balanceFactor < -1:
        self.rebalance(node)
        return
    if node.parent != None:
        if node.isLeftChild():
            node.parent.balanceFactor += 1
        elif node.isRightChild():
            node.parent.balanceFactor -= 1

        if node.parent.balanceFactor != 0:
            self.updateBalance(node.parent)
```

rebalance



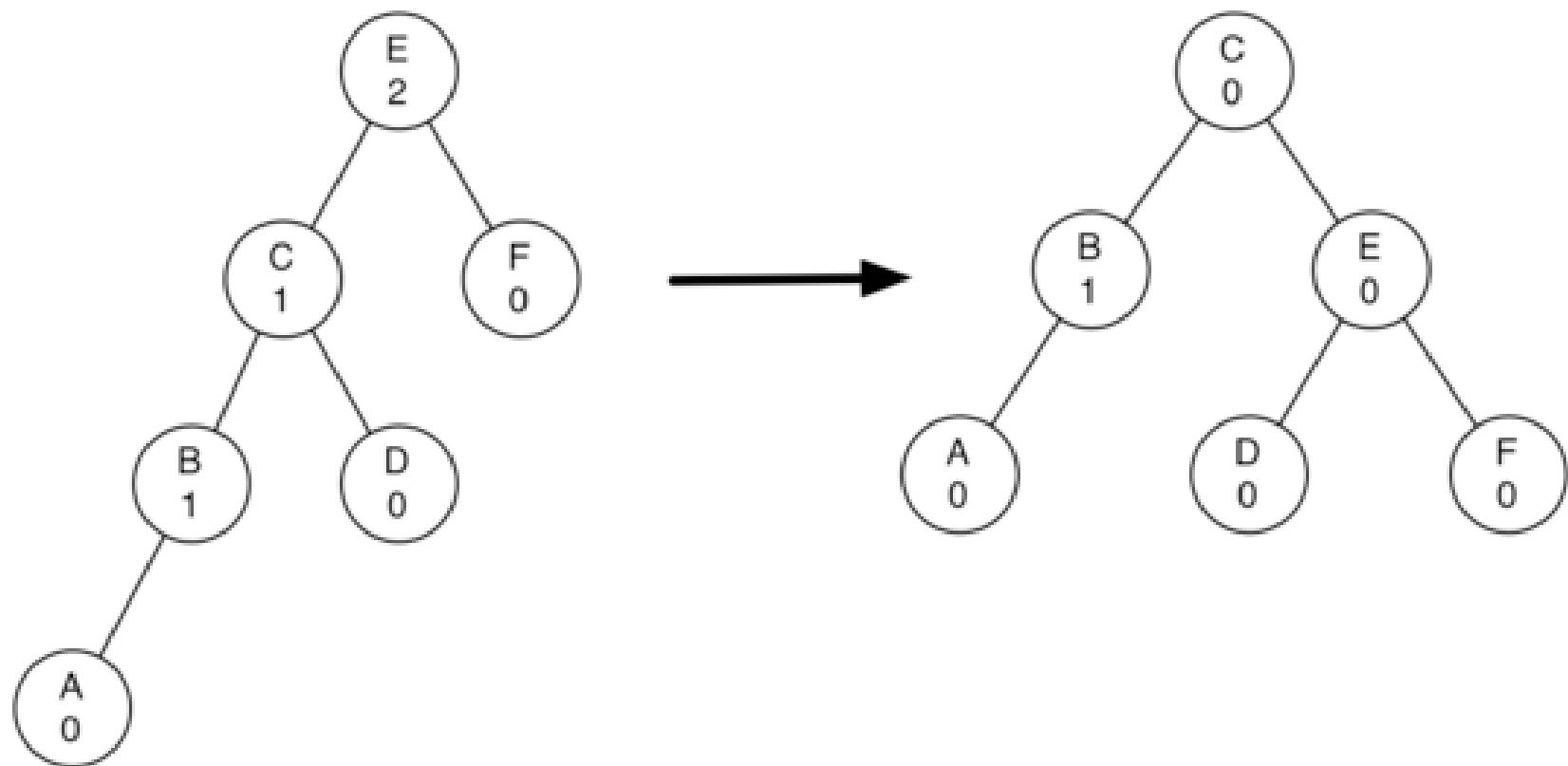
- rotation

- left rotation

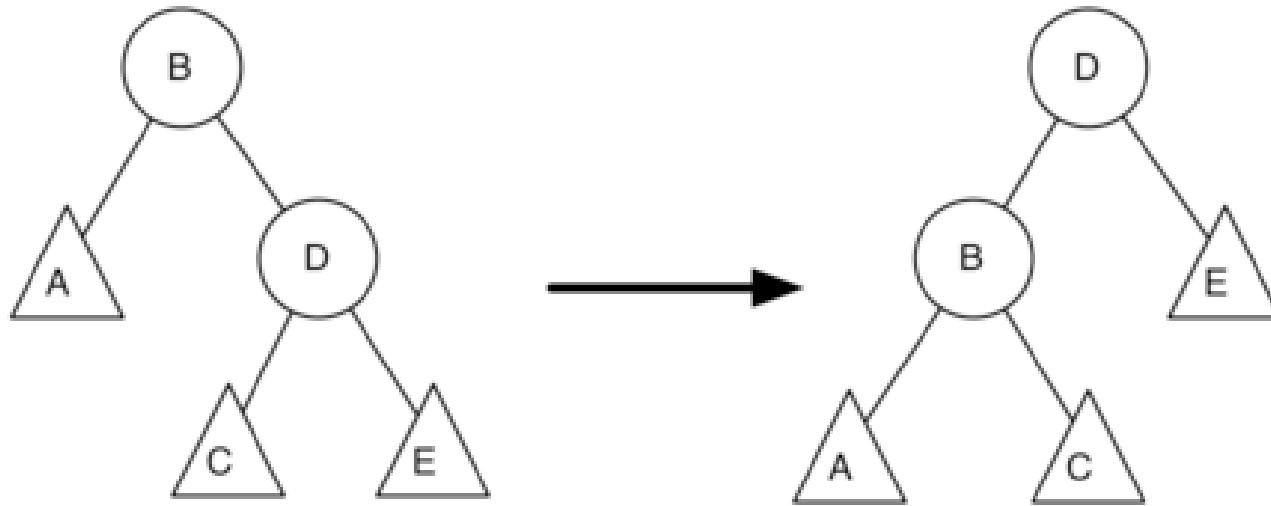
- ปรับ right child มาเป็น root ของ subtree
 - เอา root เดิมมาเป็น left child
 - ถ้า root ใหม่มี left child อยู่แล้วให้ปรับมาเป็น right child ของ left child ใหม่

- right rotation

- ปรับ left child มาเป็น root ของ subtree
 - เอา root เดิมมาเป็น right child
 - ถ้า root ใหม่มี right child อยู่แล้วให้ปรับมาเป็น left child ของ right child ใหม่



```
def rotateLeft(self, rotRoot):
    newRoot = rotRoot.rightChild
    rotRoot.rightChild = newRoot.leftChild
    if newRoot.leftChild != None:
        newRoot.leftChild.parent = rotRoot
    newRoot.parent = rotRoot.parent
    if rotRoot.isRoot():
        self.root = newRoot
    else:
        if rotRoot.isLeftChild():
            rotRoot.parent.leftChild = newRoot
        else:
            rotRoot.parent.rightChild = newRoot
    newRoot.leftChild = rotRoot
    rotRoot.parent = newRoot
    rotRoot.balanceFactor = rotRoot.balanceFactor + 1 - min(newRoot.balanceFactor, 0)
    newRoot.balanceFactor = newRoot.balanceFactor + 1 + max(rotRoot.balanceFactor, 0)
```



$$newBal(B) = h_A - h_C$$

$$oldBal(B) = h_A - h_D$$

$$oldBal(B) = h_A - (1 + \max(h_C, h_E))$$



$$newBal(B) - oldBal(B) = h_A - h_C - (h_A - (1 + \max(h_C, h_E)))$$

$$newBal(B) - oldBal(B) = h_A - h_C - h_A + (1 + \max(h_C, h_E))$$

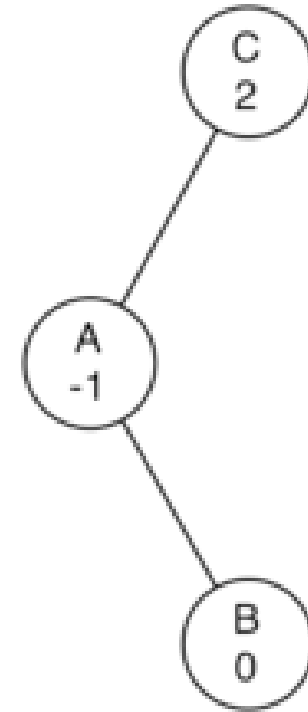
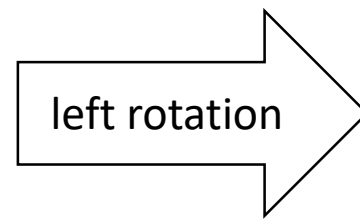
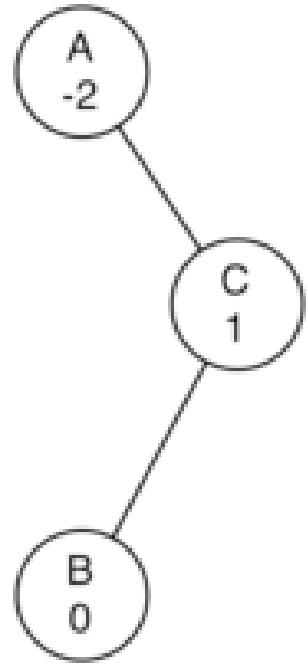
$$newBal(B) - oldBal(B) = h_A - h_A + 1 + \max(h_C, h_E) - h_C$$

$$newBal(B) - oldBal(B) = 1 + \max(h_C, h_E) - h_C$$

$$newBal(B) = oldBal(B) + 1 + \max(h_C - h_C, h_E - h_C)$$

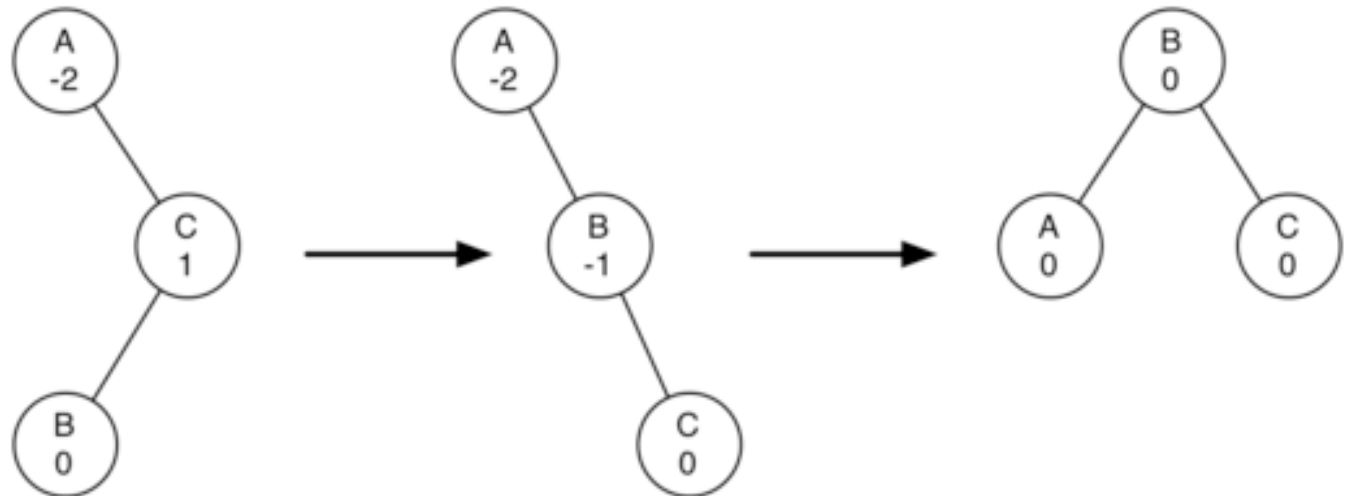
$$newBal(B) = oldBal(B) + 1 + \max(0, -oldBal(D))$$

$$newBal(B) = oldBal(B) + 1 - \min(0, oldBal(D))$$



double rotation

- ทำ left rotation
 - ถ้า right child มี balance factor > 0 (หนักไปทางฝั่งซ้าย) ให้ทำ right rotation ที่ right child ก่อน แล้วจึงทำ left rotation ปกติ
- ทำ right rotation
 - ถ้า left child มี balance factor < 0 (หนักไปทางฝั่งขวา) ให้ทำ left rotation ที่ left child ก่อน แล้วจึงทำ right rotation ปกติ

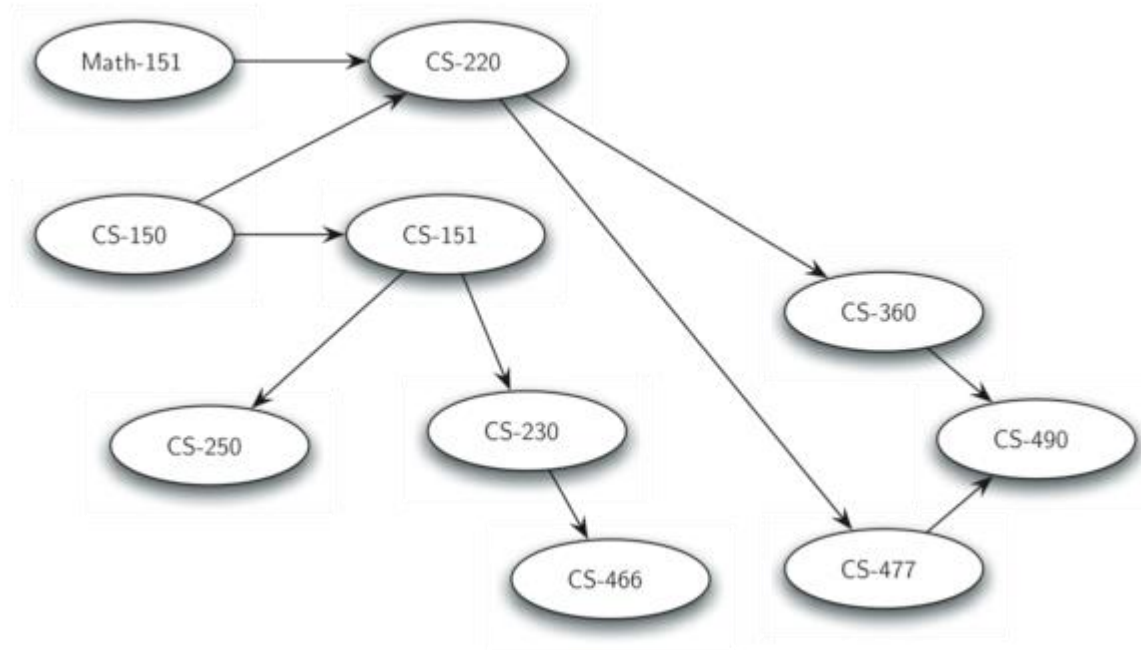


```
def rebalance(self,node):
    if node.balanceFactor < 0:
        if node.rightChild.balanceFactor > 0:
            self.rotateRight(node.rightChild)
            self.rotateLeft(node)
        else:
            self.rotateLeft(node)
    elif node.balanceFactor > 0:
        if node.leftChild.balanceFactor < 0:
            self.rotateLeft(node.leftChild)
            self.rotateRight(node)
        else:
            self.rotateRight(node)
```

กราฟ
graph

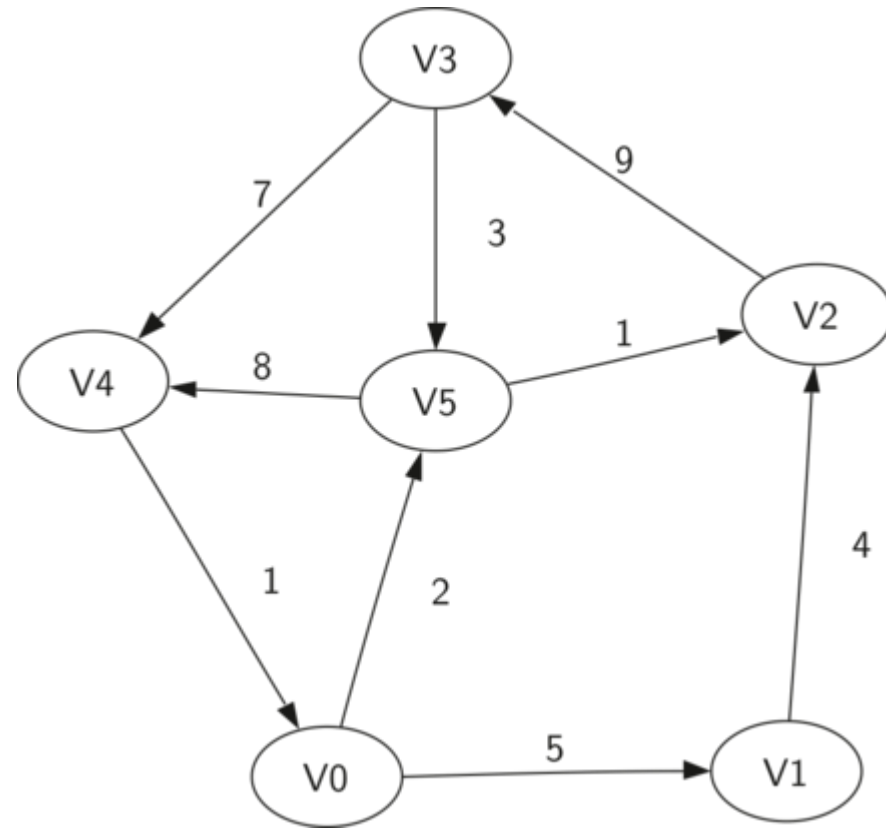
ใช้ทำอะไร?

- แสดงความสัมพันธ์ของข้อมูล
- อธิบายได้กว้างกว่าต้นไม้



คำศัพท์

- จุดยอด (vertex, node)
 - key
 - payload
- เส้นเชื่อม (edge, arc)
 - มีทิศทาง
 - ไม่มีทิศทาง
- น้ำหนัก (weight)
 - ระยะทาง
- $G = (V, E)$

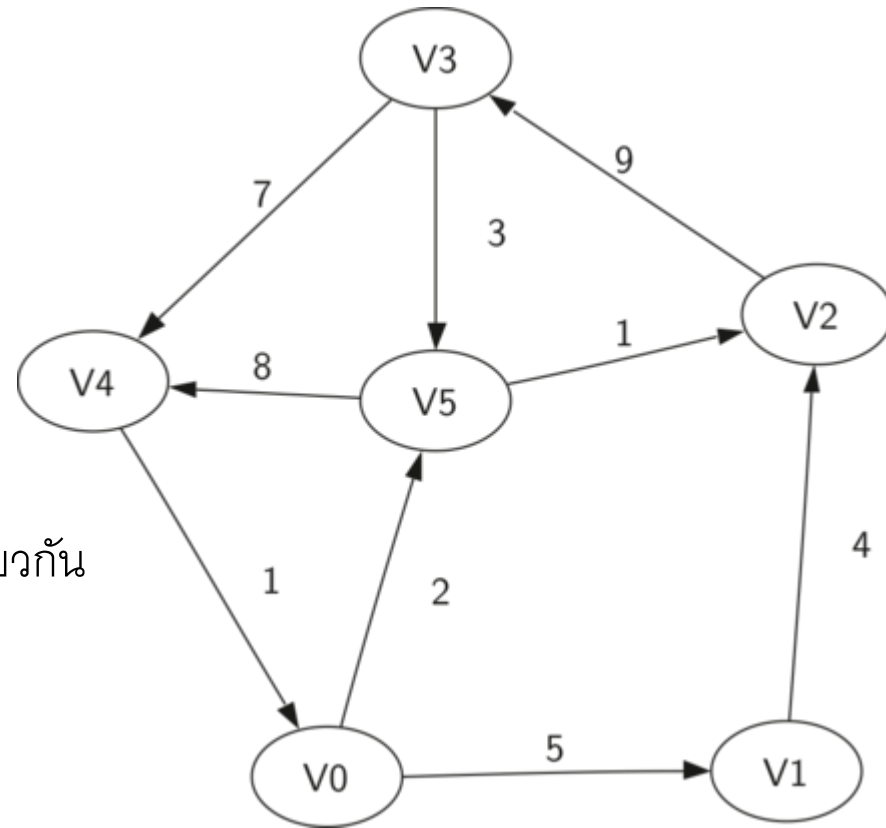


$V = \{V0, V1, V2, V3, V4, V5\}$

$E = \{(v0, v1, 5), (v1, v2, 4), (v2, v3, 9), (v3, v4, 7), (v4, v0, 1), (v0, v5, 2), (v5, v4, 8), (v3, v5, 3), (v5, v2, 1)\}$

คำศัพท์

- เส้นทาง (path)
 - ลำดับของจุดยอดที่เชื่อมโยงกัน
 - ความยาวของเส้นทางคำนวณจากน้ำหนัก
- วงวน (cycle)
 - จุดยอดเริ่มต้นและจุดยอดสุดท้ายของเส้นทางเป็นจุดเดียวกัน



(V3,V4,V0,V1)

{(v3,v4,7),(v4,v0,1),(v0,v1,5)}

graph ADT

- Graph() สร้าง graph ว่าง.
- addVertex(vert) เพิ่ม vertex ให้กับ graph.
- addEdge(fromVert, toVert) กำหนด edge ที่มีทิศทางให้กับ 2 vertex
- addEdge(fromVert, toVert, weight) กำหนด edge ที่มีทิศทางและน้ำหนักให้กับ 2 vertex
- getVertex(vertKey) ค้นหา vertex ด้วย key
- getVertices() คืนค่า list ของ vertex ทั้งหมดใน graph
- in คืนค่า True เมื่อมี vertex ที่ถามอยู่ใน graph คืนค่า False เมื่อไม่มี

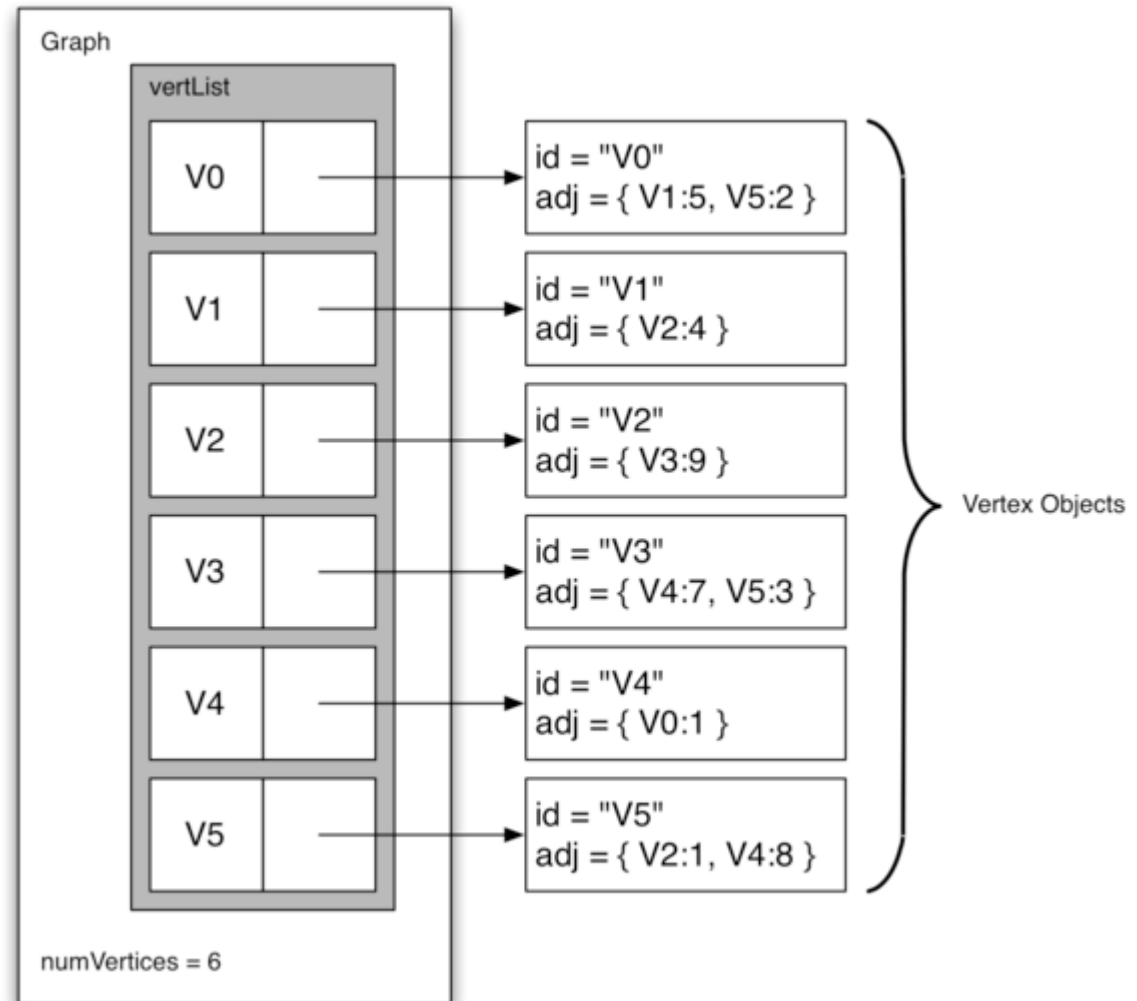
เมตริกซ์การเชื่อมโยง (adjacency matrix)

- ใช้ตารางแสดงความเชื่อมโยง
 - array 2 มิติ
- เห็นความเชื่อมโยงได้ง่าย
- เปลืองเนื้อที่
- ดีเมื่อมี **edge** เยอะ

	V0	V1	V2	V3	V4	V5
V0		5				2
V1			4			
V2				9		
V3					7	3
V4	1					
V5			1		8	

ลิสต์การเชื่อมโยง (adjacency list)

- กะทัดรัด
- หาความเชื่อมโยงได้ง่ายกว่า



```
class Vertex:
    def __init__(self, key):
        self.id = key
        self.connectedTo = {}

    def addNeighbor(self, nbr, weight=0):
        self.connectedTo[nbr] = weight

    def __str__(self):
        return str(self.id) + ' connectedTo: ' + str([x.id for x in self.connectedTo])

    def getConnections(self):
        return self.connectedTo.keys()

    def getId(self):
        return self.id

    def getWeight(self, nbr):
        return self.connectedTo[nbr]
```

```
class Graph:
```

```
    def __init__(self):
```

```
        self.vertList = {}
```

```
        self.numVertices = 0
```

```
    def addVertex(self, key):
```

```
        self.numVertices = self.numVertices + 1
```

```
        newVertex = Vertex(key)
```

```
        self.vertList[key] = newVertex
```

```
        return newVertex
```

```
    def getVertex(self, n):
```

```
        if n in self.vertList:
```

```
            return self.vertList[n]
```

```
        else:
```

```
            return None
```

```
    def __contains__(self, n):
```

```
        return n in self.vertList
```

```
    def addEdge(self, f, t, cost=0):
```

```
        if f not in self.vertList:
```

```
            nv = self.addVertex(f)
```

```
        if t not in self.vertList:
```

```
            nv = self.addVertex(t)
```

```
        self.vertList[f].addNeighbor(self.vertList[t], cost)
```

```
    def getVertices(self):
```

```
        return self.vertList.keys()
```

```
    def __iter__(self):
```

```
        return iter(self.vertList.values())
```

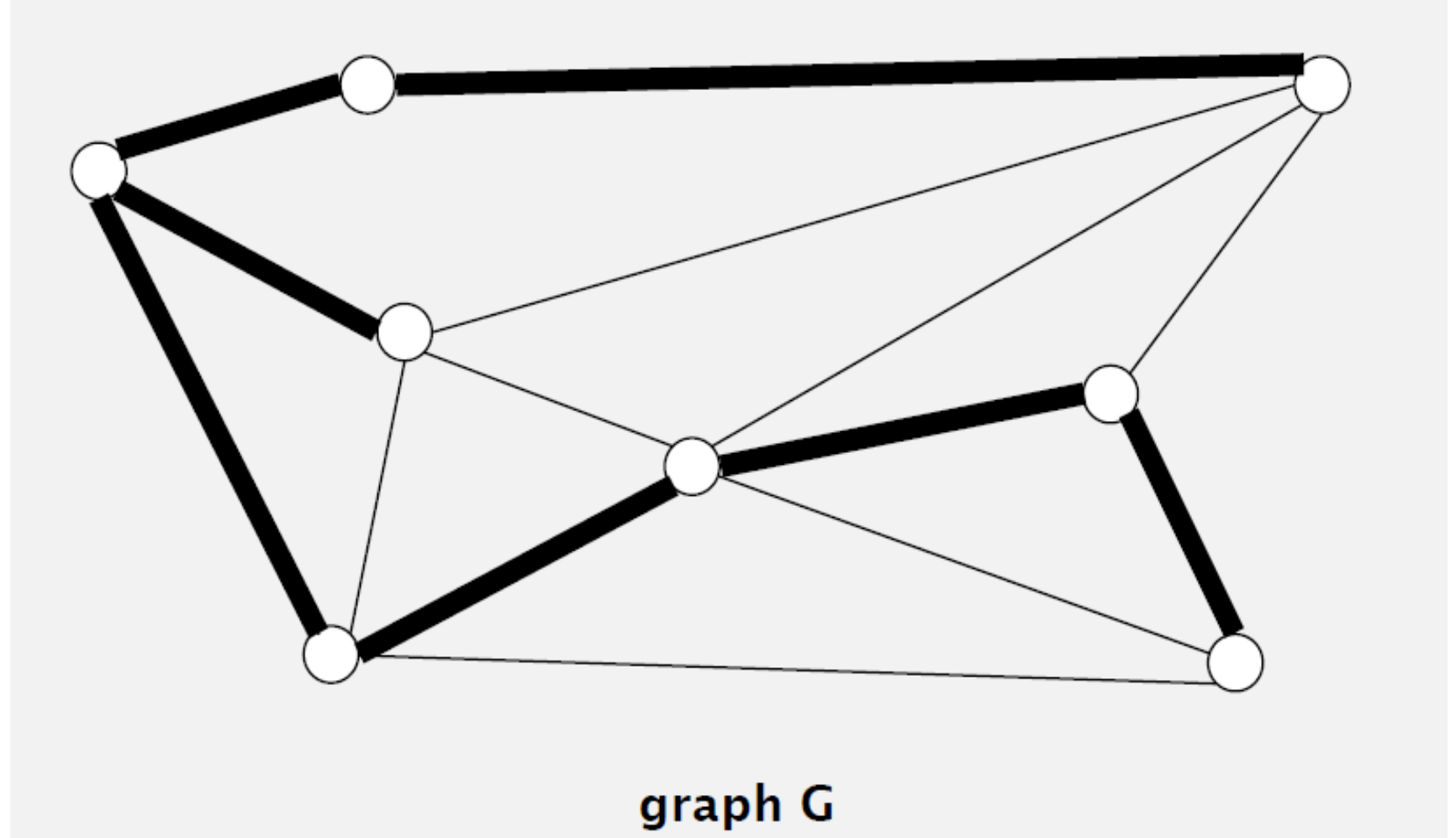
```
>>> g = Graph()
>>> for i in range(6):
...     g.addVertex(i)
>>> g.vertList
{0: <adjGraph.Vertex instance at 0x41e18>,
 1: <adjGraph.Vertex instance at 0x7f2b0>,
 2: <adjGraph.Vertex instance at 0x7f288>,
 3: <adjGraph.Vertex instance at 0x7f350>,
 4: <adjGraph.Vertex instance at 0x7f328>,
 5: <adjGraph.Vertex instance at 0x7f300>}
>>> g.addEdge(0,1,5)
>>> g.addEdge(0,5,2)
>>> g.addEdge(1,2,4)
>>> g.addEdge(2,3,9)
>>> g.addEdge(3,4,7)
>>> g.addEdge(3,5,3)
>>> g.addEdge(4,0,1)
>>> g.addEdge(5,4,8)
>>> g.addEdge(5,2,1)
```

```
>>> for v in g:
...     for w in v.getConnections():
...         print("( %s , %s )" % (v.getId(), w.getId()))
...
( 0 , 5 )
( 0 , 1 )
( 1 , 2 )
( 2 , 3 )
( 3 , 4 )
( 3 , 5 )
( 4 , 0 )
( 5 , 4 )
( 5 , 2 )
```


Graph's applications

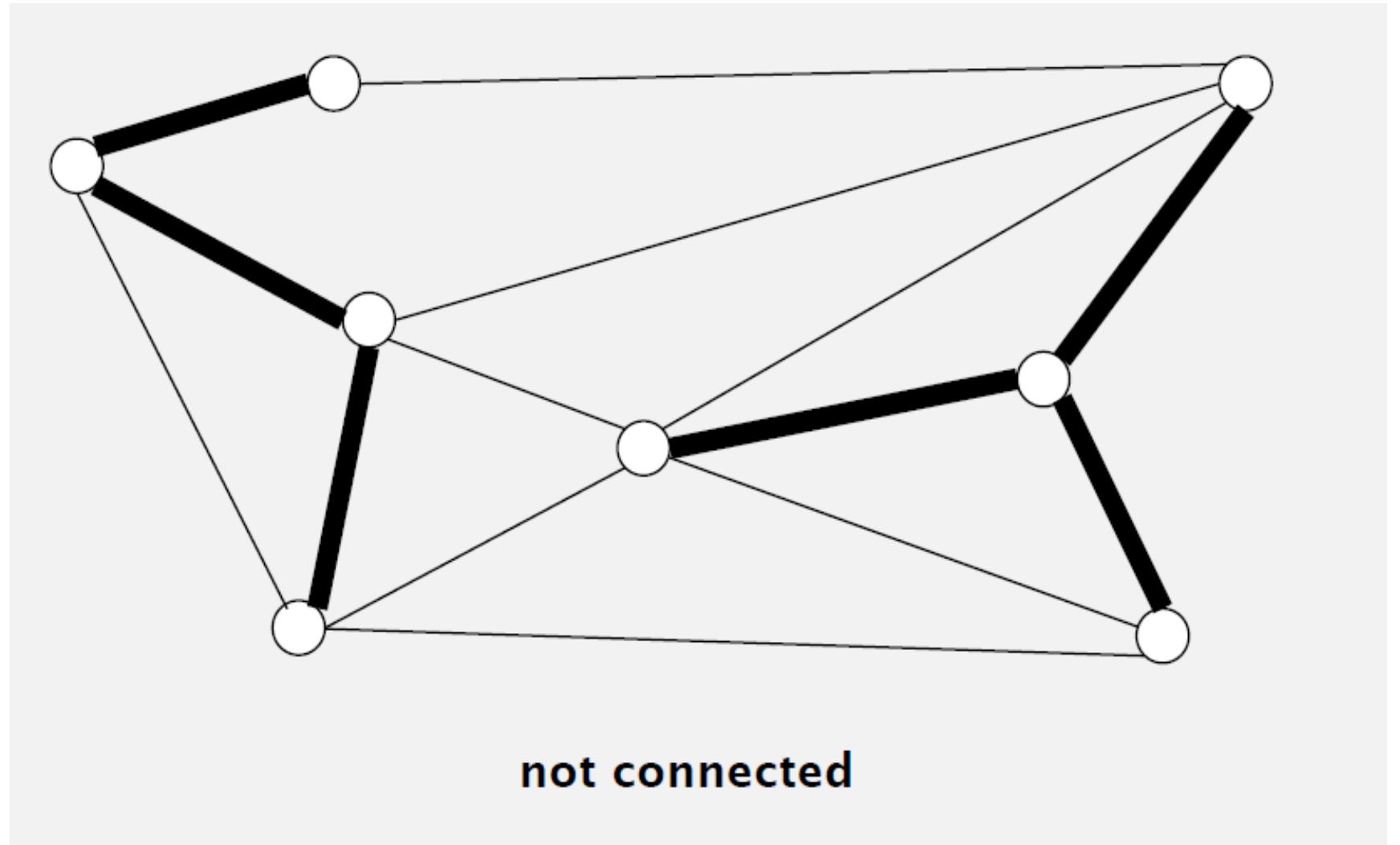
Minimum spanning tree

- กราฟย่อยที่
 - เชื่อมต่อกัน
 - ไม่เกิดวง
 - สร้างจากทุกจุดยอด



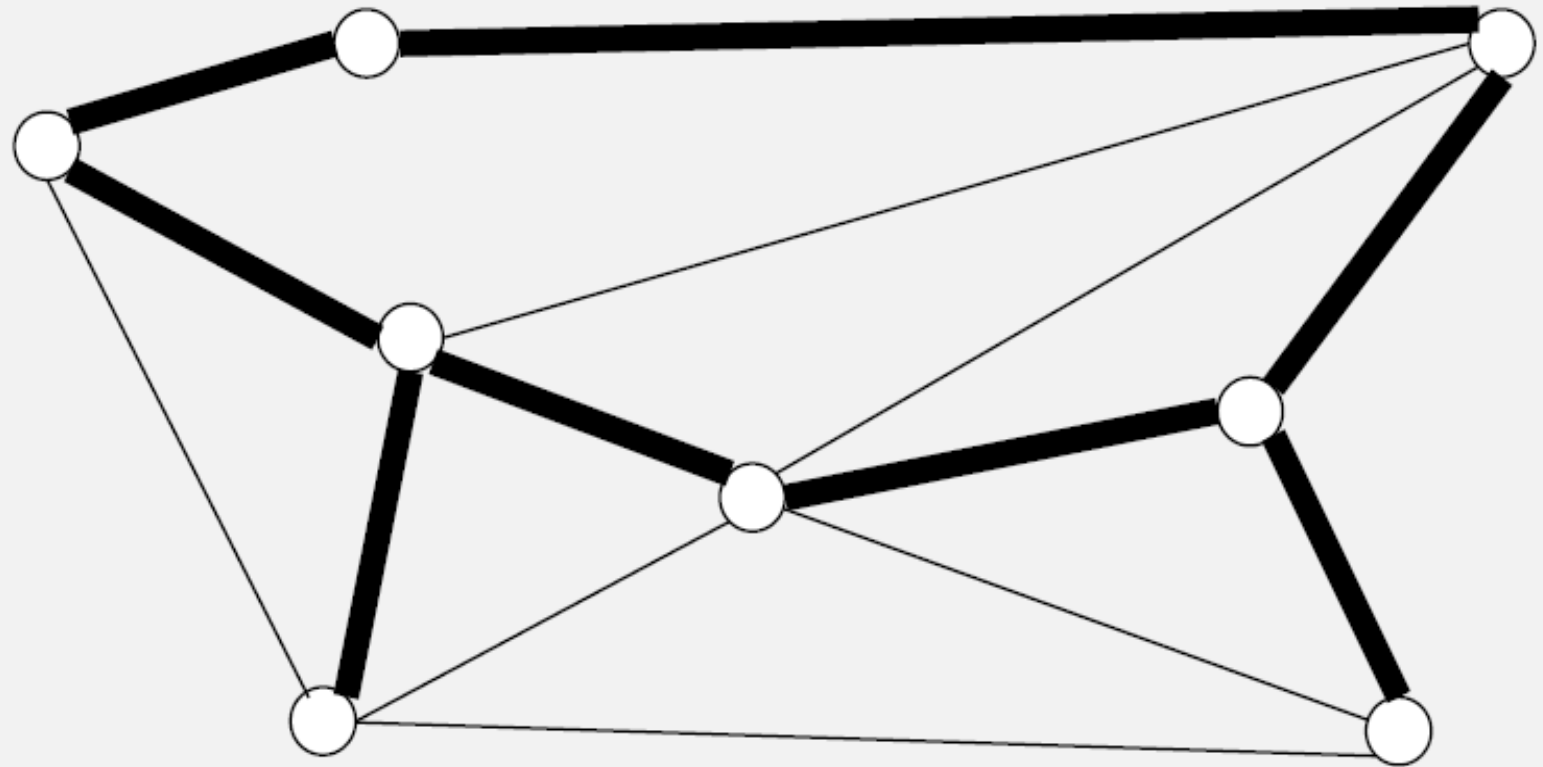
Minimum spanning tree

- กราฟย่อยที่
 - เชื่อมต่อกัน
 - ไม่เกิดวง
 - สร้างจากทุกจุดยอด



Minimum spanning tree

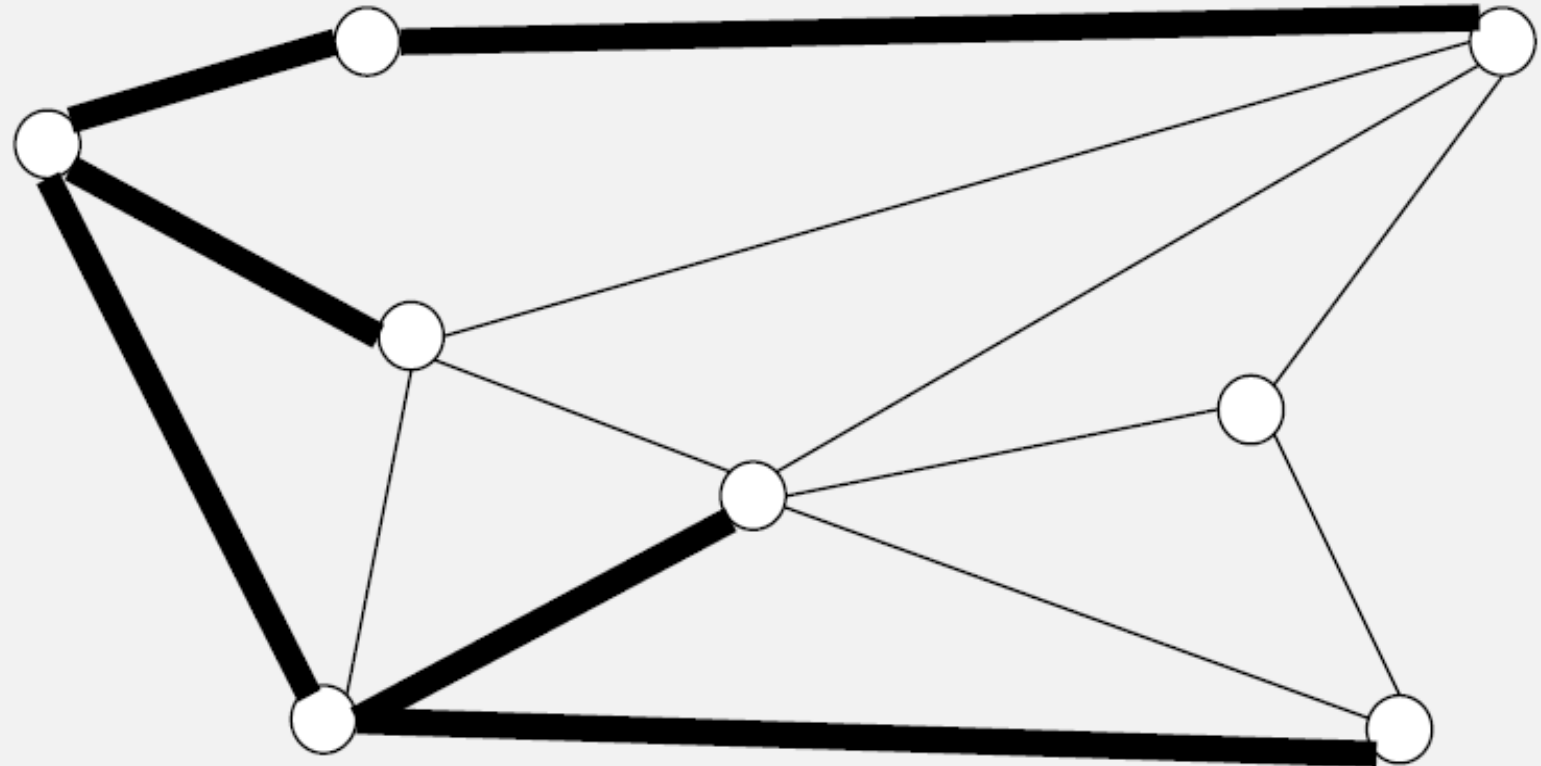
- กราฟย่อยที่
 - เชื่อมต่อกัน
 - ไม่เกิดวง
 - สร้างจากทุกจุดยอด



not acyclic

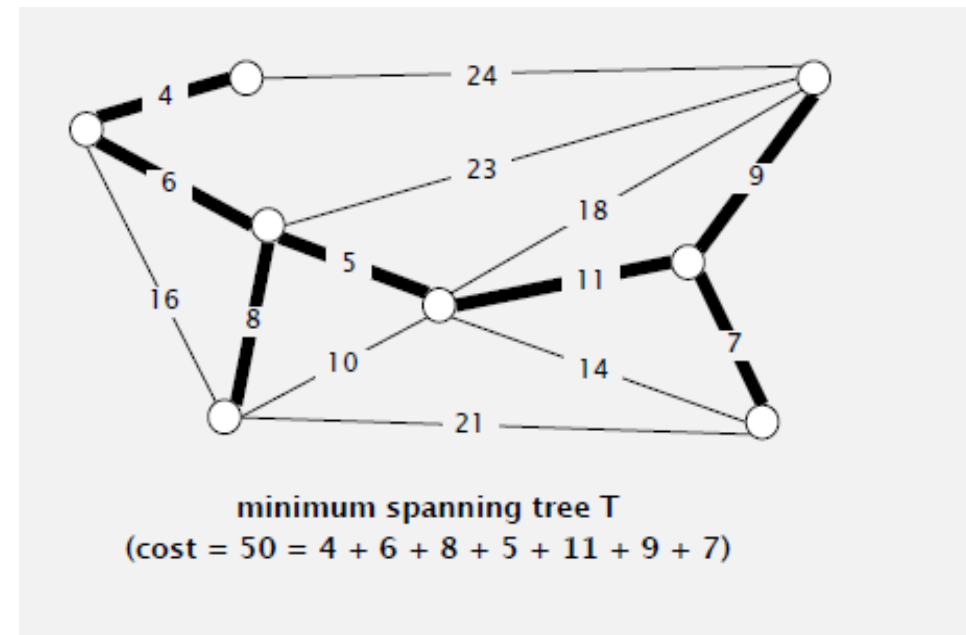
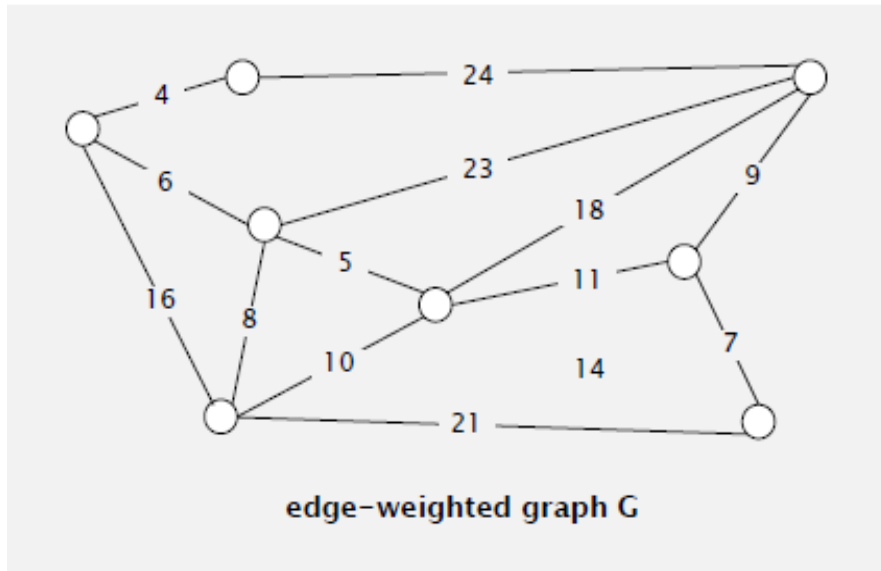
Minimum spanning tree

- กราฟย่อยที่
 - เชื่อมต่อกัน
 - ไม่เกิดวง
 - สร้างจากทุกจุดยอด



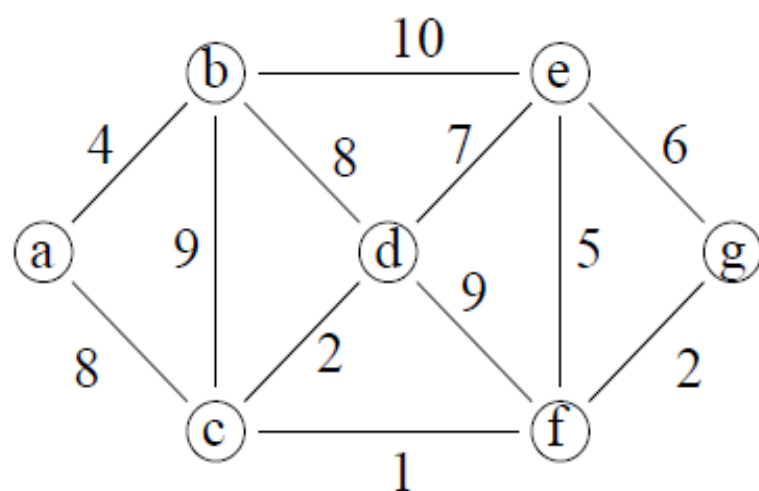
Minimum spanning tree

- มีกราฟ G ที่มีน้ำหนักของเส้นเชื่อมเป็นค่าบวก
- หา **spanning tree** ที่มีน้ำหนักรวมของเส้นเชื่อมน้อยที่สุด
- ต้นไม้ทอดข้ามน้อยสุด หรือ **MST**

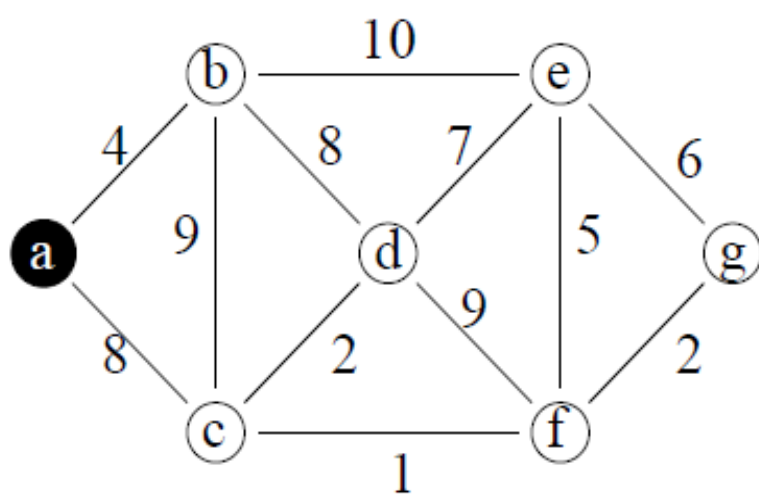


Prim's algorithm

- กำหนดให้ต้นไม้ที่เราจะสร้างคือ S
- เริ่มที่จุดยอดแรกของกราฟ G แล้วค่อยๆ ขยายต้นไม้ S
- การขยายต้นไม้ทำโดยเลือกเส้นเชื่อมที่มีค่าน้อยที่สุดที่มีเส้นเชื่อมกับ S
- ทำซ้ำจนได้จำนวนเส้นเชื่อมเท่ากับจำนวนจุดยอด - 1



Connected graph

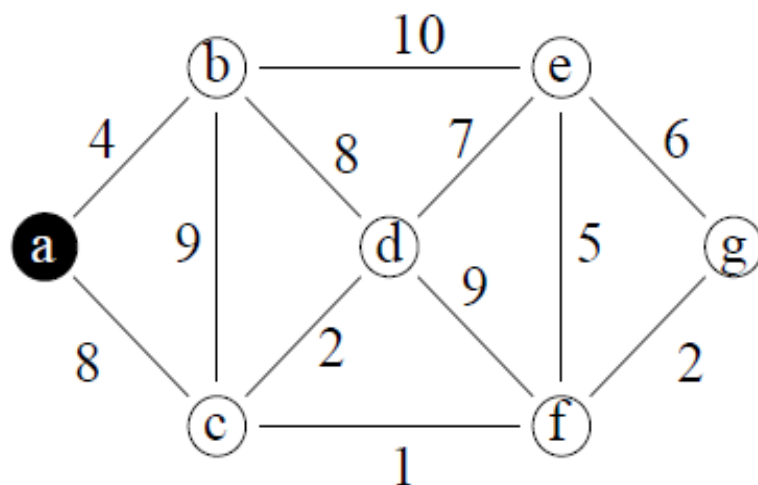


Step 0

$S = \{a\}$

$V \setminus S = \{b, c, d, e, f, g\}$

lightest edge = $\{a, b\}$



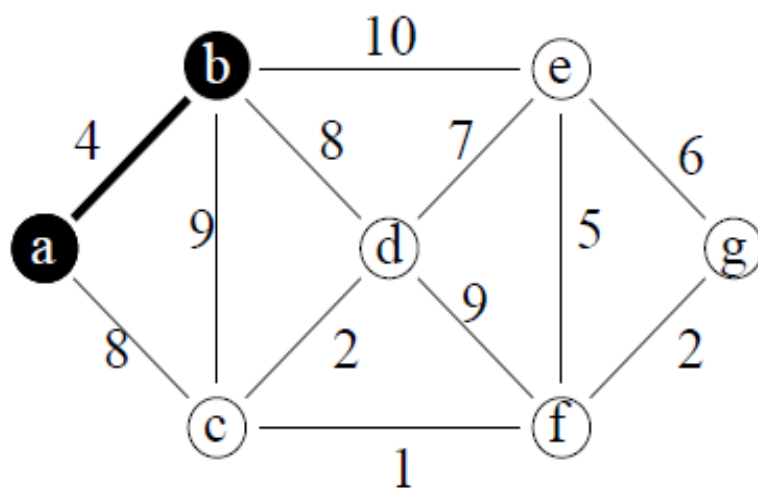
Step 1.1 before

$S = \{a\}$

$V \setminus S = \{b, c, d, e, f, g\}$

$A = \{\}$

lightest edge = $\{a, b\}$



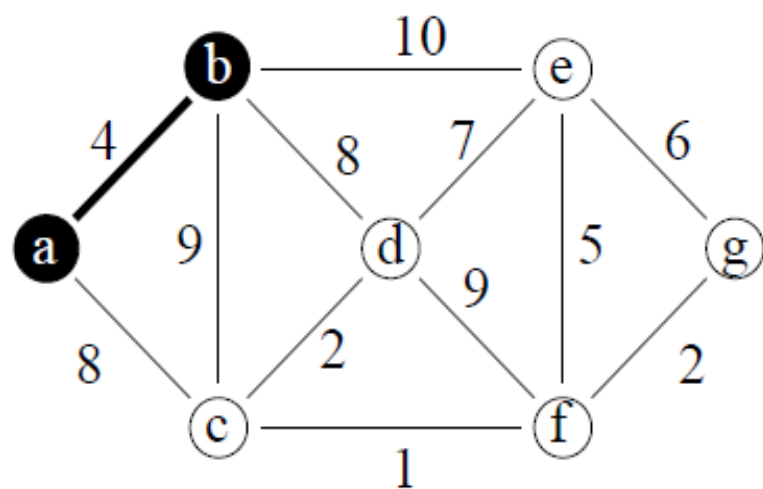
Step 1.1 after

$S = \{a, b\}$

$V \setminus S = \{c, d, e, f, g\}$

$A = \{\{a, b\}\}$

lightest edge = $\{b, d\}, \{a, c\}$



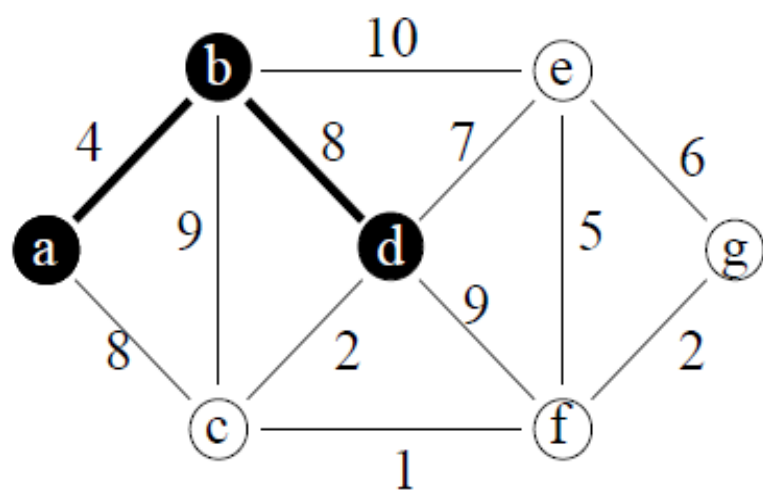
Step 1.2 before

$S = \{a, b\}$

$V \setminus S = \{c, d, e, f, g\}$

$A = \{\{a, b\}\}$

lightest edge = $\{b, d\}, \{a, c\}$



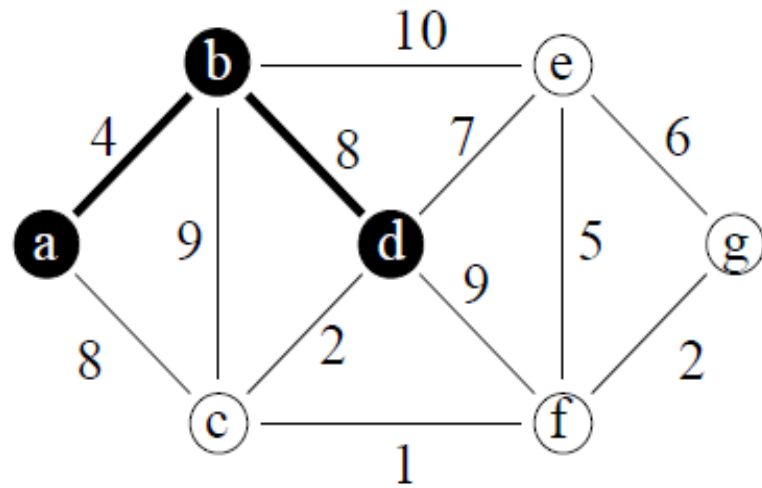
Step 1.2 after

$S = \{a, b, d\}$

$V \setminus S = \{c, e, f, g\}$

$A = \{\{a, b\}, \{b, d\}\}$

lightest edge = $\{d, c\}$



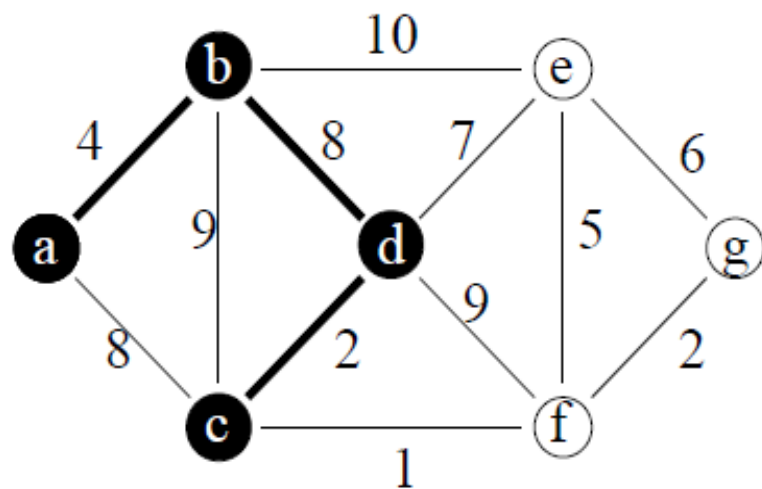
Step 1.3 before

$S = \{a, b, d\}$

$V \setminus S = \{c, e, f, g\}$

$A = \{\{a, b\}, \{b, d\}\}$

lightest edge = $\{d, c\}$



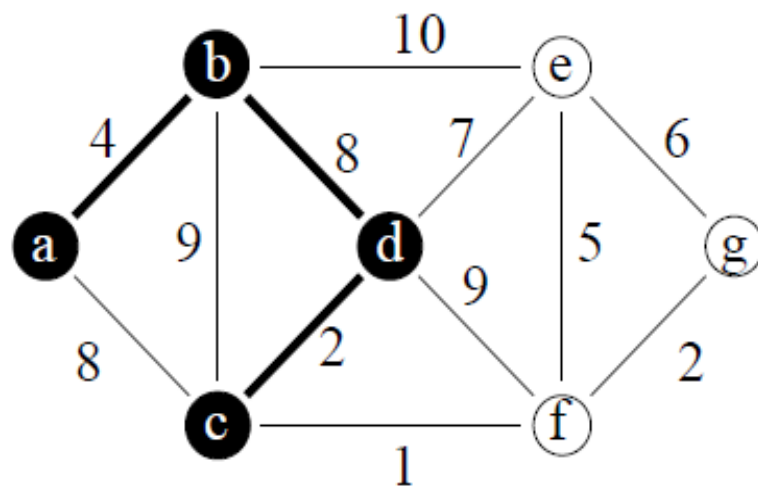
Step 1.3 after

$S = \{a, b, c, d\}$

$V \setminus S = \{e, f, g\}$

$A = \{\{a, b\}, \{b, d\}, \{c, d\}\}$

lightest edge = $\{c, f\}$



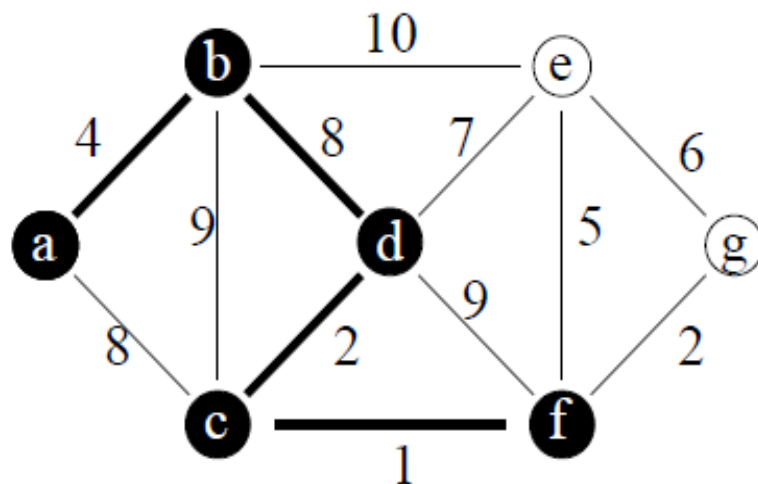
Step 1.4 before

$S = \{a, b, c, d\}$

$V \setminus S = \{e, f, g\}$

$A = \{\{a, b\}, \{b, d\}, \{c, d\}\}$

lightest edge = $\{c, f\}$



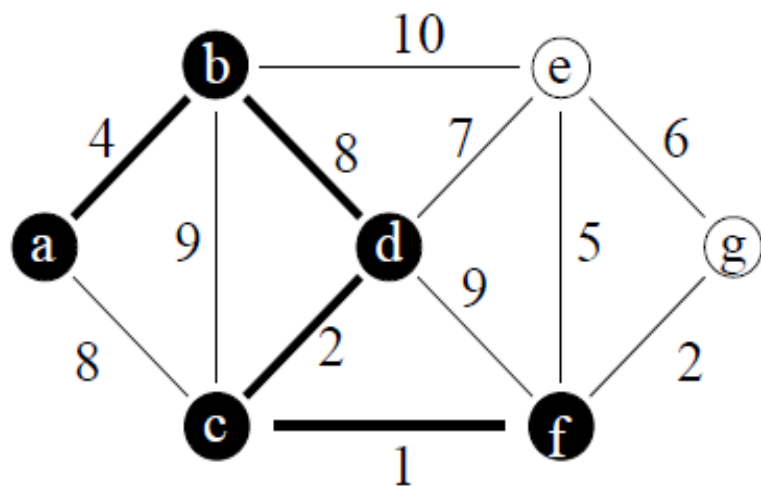
Step 1.4 after

$S = \{a, b, c, d, f\}$

$V \setminus S = \{e, g\}$

$A = \{\{a, b\}, \{b, d\}, \{c, d\}, \{c, f\}\}$

lightest edge = $\{f, g\}$



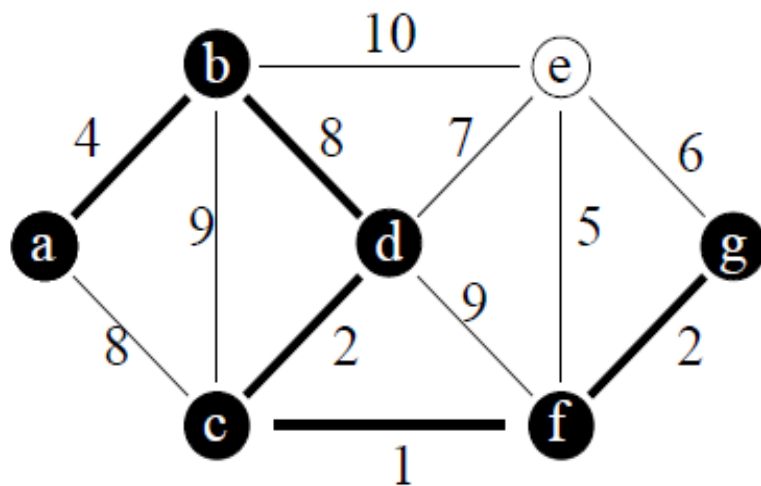
Step 1.5 before

$S = \{a, b, c, d, f\}$

$V \setminus S = \{e, g\}$

$A = \{\{a, b\}, \{b, d\}, \{c, d\}, \{c, f\}\}$

lightest edge = $\{f, g\}$



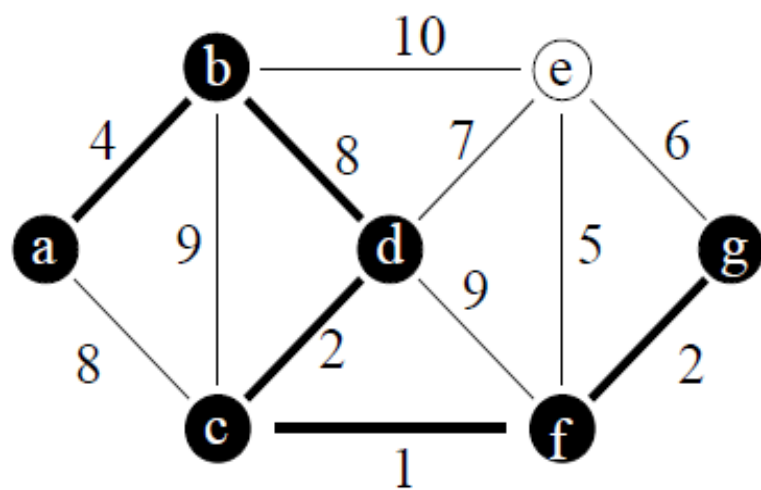
Step 1.5 after

$S = \{a, b, c, d, f, g\}$

$V \setminus S = \{e\}$

$A = \{\{a, b\}, \{b, d\}, \{c, d\}, \{c, f\},$
 $\{f, g\}\}$

lightest edge = $\{f, e\}$



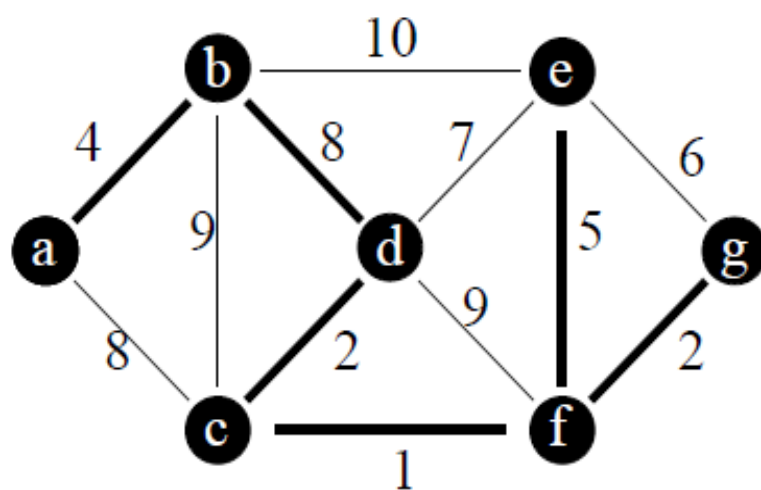
Step 1.6 before

$S = \{a, b, c, d, f, g\}$

$V \setminus S = \{e\}$

$A = \{\{a, b\}, \{b, d\}, \{c, d\}, \{c, f\},$
 $\{f, g\}\}$

lightest edge = $\{f, e\}$



Step 1.6 after

$S = \{a, b, c, d, e, f, g\}$

$V \setminus S = \{\}$

$A = \{\{a, b\}, \{b, d\}, \{c, d\}, \{c, f\},$
 $\{f, g\}, \{f, e\}\}$

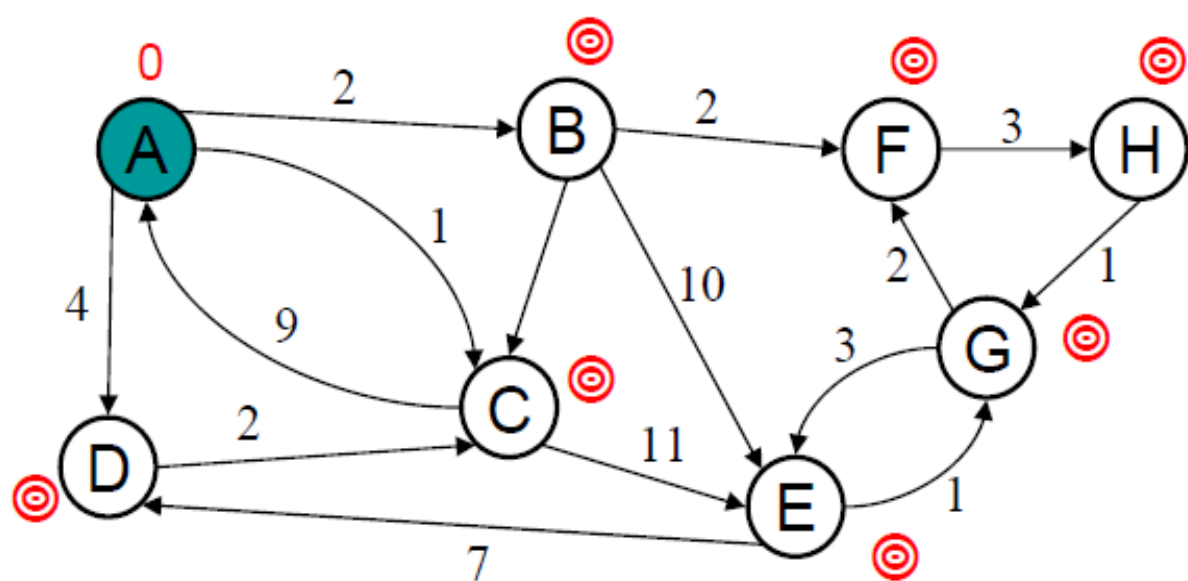
MST completed

Shortest path

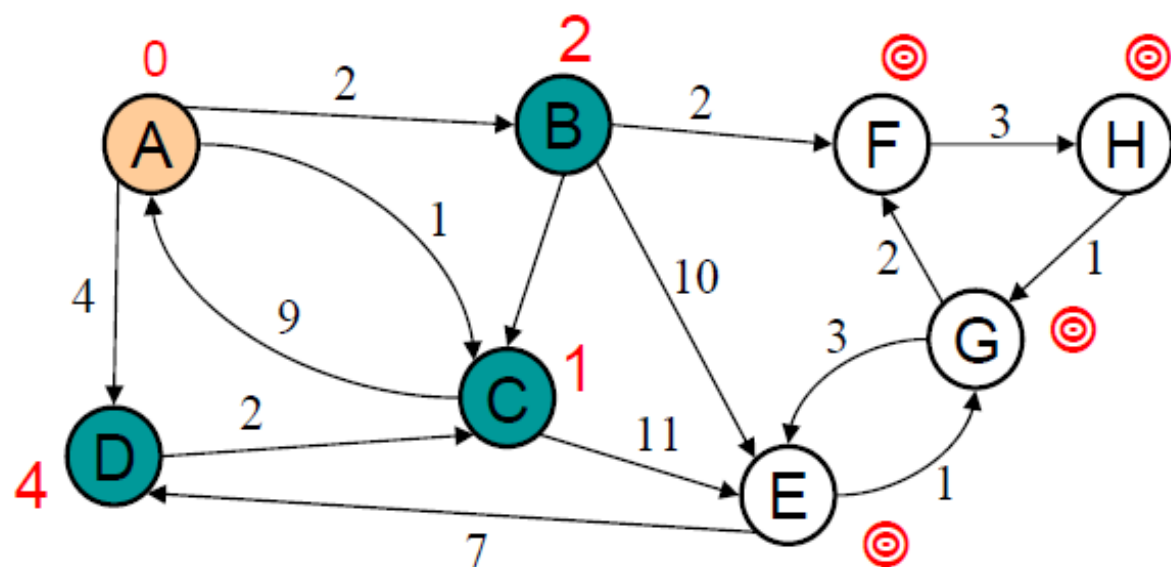
- หาเส้นทางสั้นสุดจากจุดยอดหนึ่งไปยังจุดยอดอื่นๆ ทั้งหมด
- Dijkstra's algorithm
- แบ่งจุดยอดเป็น 2 ประเภท
 - จุดยอดที่รู้ระยะทางแล้ว (known)
 - จุดยอดที่ยังไม่รู้ระยะทาง (unknown)
- หลักการ
 - หยิบ unknown ที่มีระยะสั้นสุด
 - เพิ่มเข้าในส่วน known
 - ปรับค่าระยะทาง

Dijkstra's algorithm

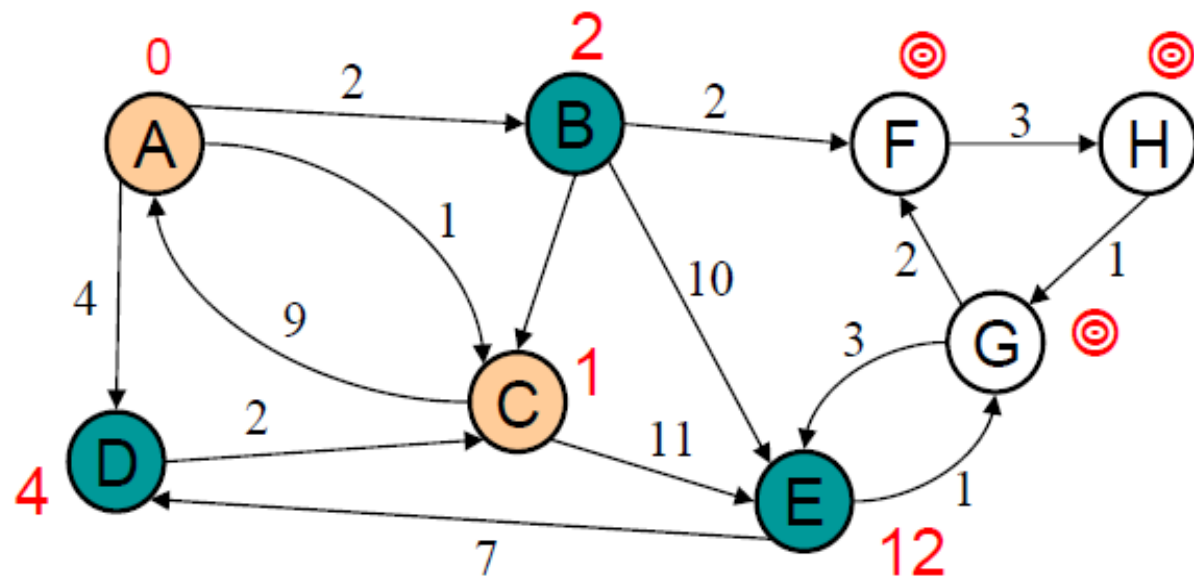
- เริ่มต้นระยะทางที่จะไปถึงแต่ละจุดยอดเป็น ∞
- เริ่มต้นระยะทางของจุดยอดเริ่มต้นเป็น 0
- While ยังมี unknown
 - หยิบ unknown ที่มีระยะทางน้อยสุด กำหนดให้ชื่อ b
 - เปลี่ยน b เป็น known
 - For each จุดยอด a ที่เชื่อมต่อกับ b
 - ปรับระยะ a เป็น $\min(\text{ระยะที่มาถึง } a \text{ เก่า}, \text{ระยะที่มาถึง } b + \text{ระยะจาก } b \text{ ไป } a)$
 - เปลี่ยนเส้นทางเข้าหา a หากระยะที่มาถึง $b + \text{ระยะจาก } b \text{ ไป } a$ สั้นกว่า



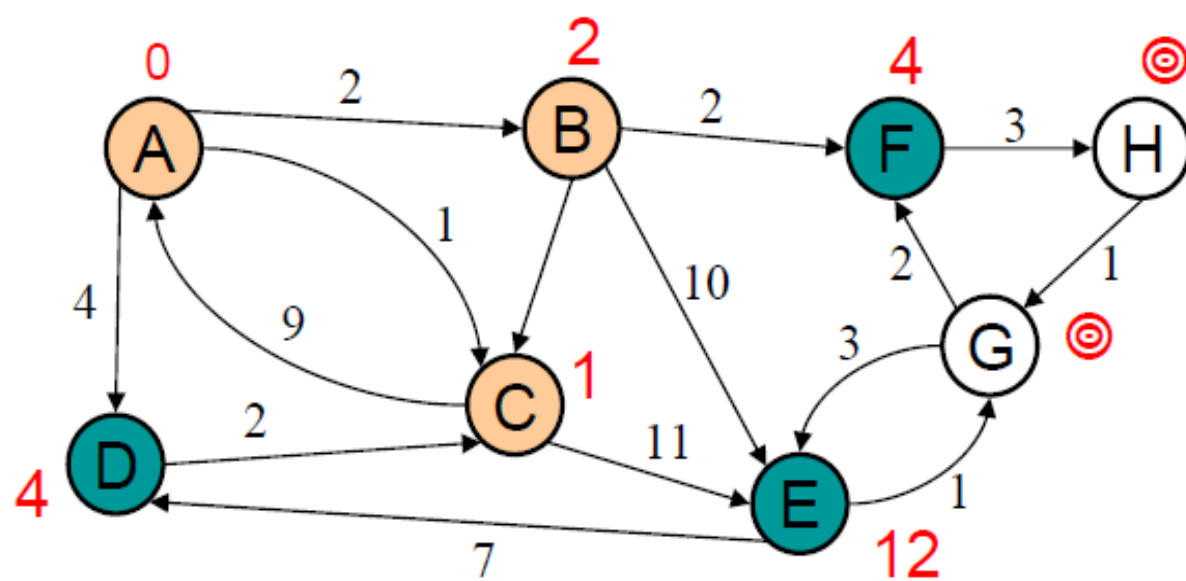
Vertex	Visited?	Cost	Found by
A		0	
B		??	
C		??	
D		??	
E		??	
F		??	
G		??	
H		??	



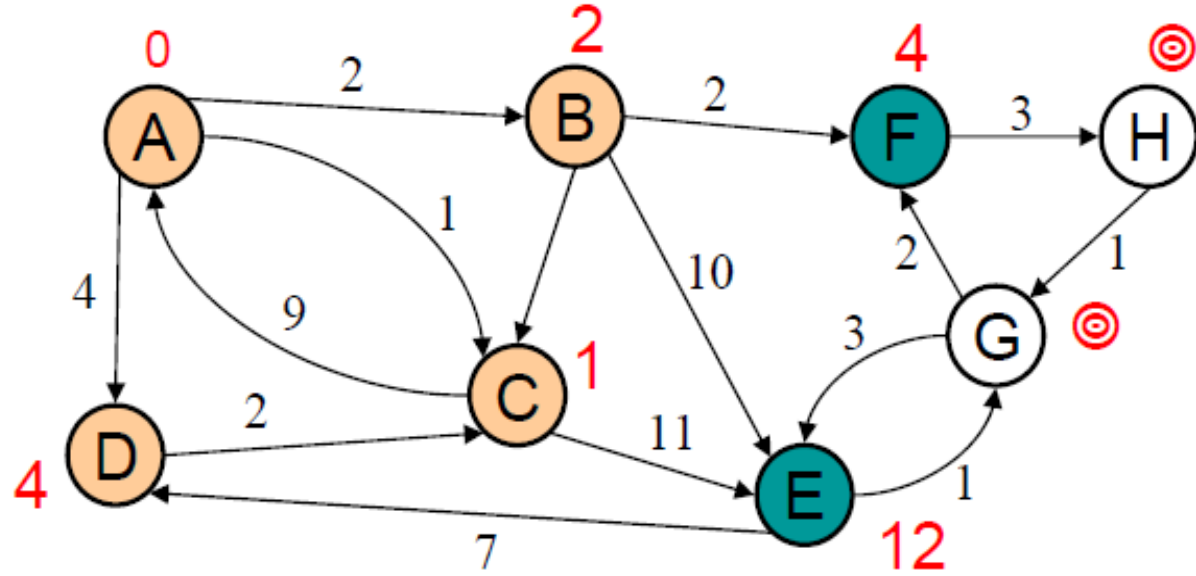
Vertex	Visited?	Cost	Found by
A	Y	0	
B		≤ 2	A
C		≤ 1	A
D		≤ 4	A
E		??	
F		??	
G		??	
H		??	



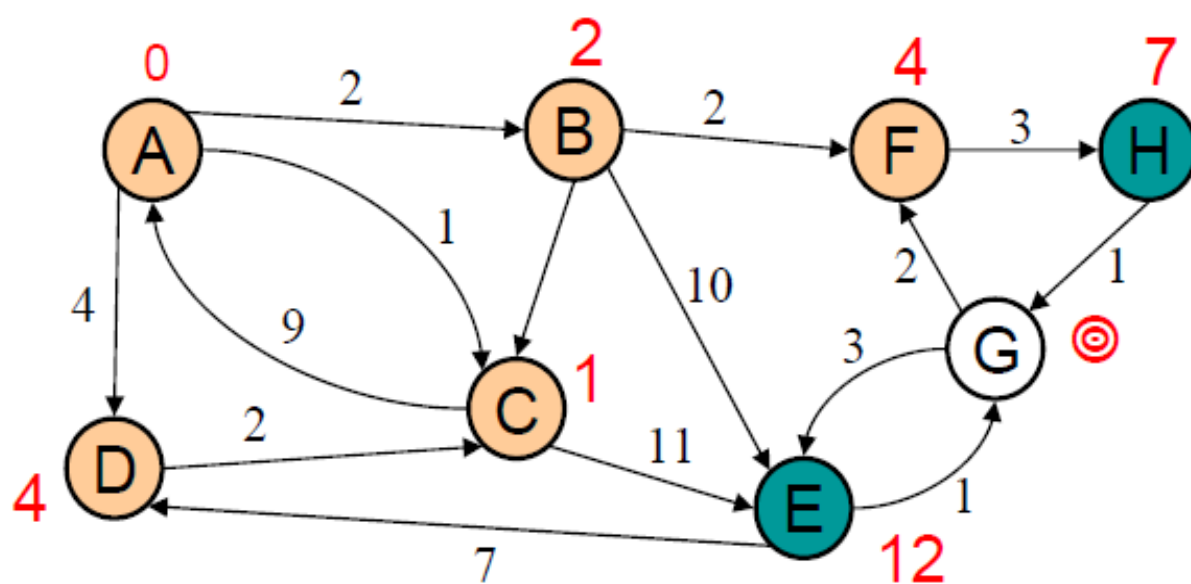
Vertex	Visited?	Cost	Found by
A	Y	0	
B		≤ 2	A
C	Y	1	A
D		≤ 4	A
E		≤ 12	C
F		??	
G		??	
H		??	



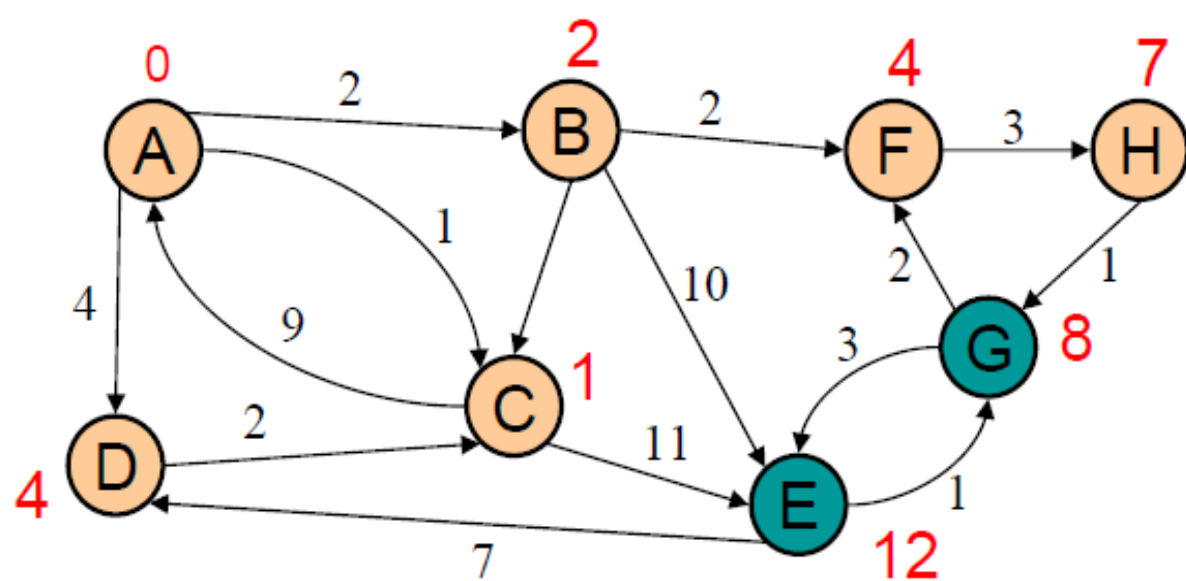
Vertex	Visited?	Cost	Found by
A	Y	0	
B	Y	2	A
C	Y	1	A
D		≤ 4	A
E		≤ 12	C
F		≤ 4	B
G		??	
H		??	



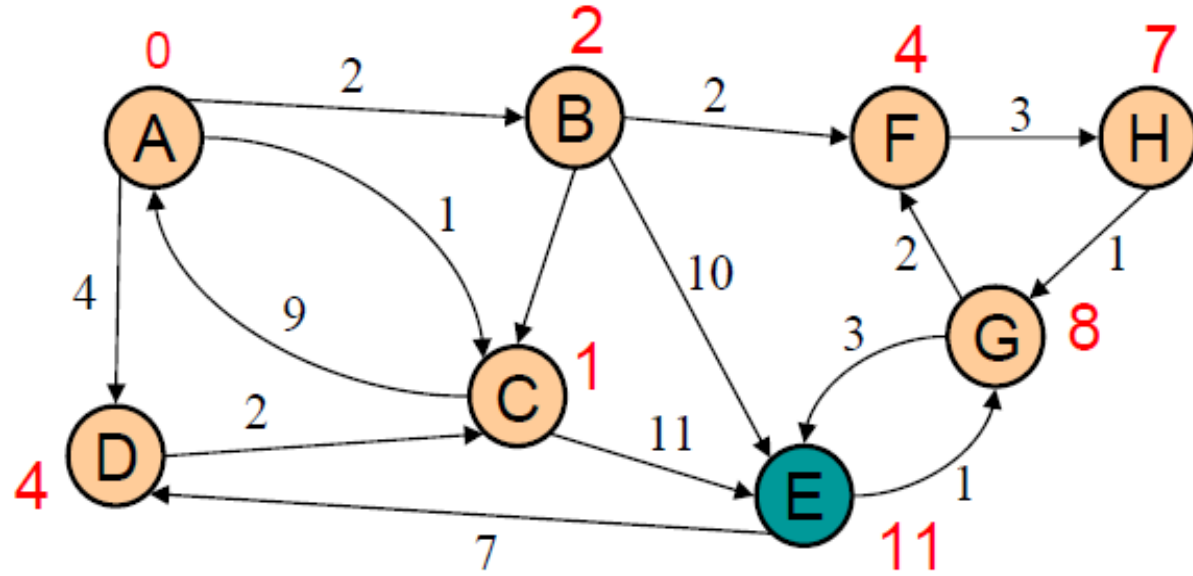
Vertex	Visited?	Cost	Found by
A	Y	0	
B	Y	2	A
C	Y	1	A
D	Y	4	A
E		≤ 12	C
F		≤ 4	B
G		??	
H		??	



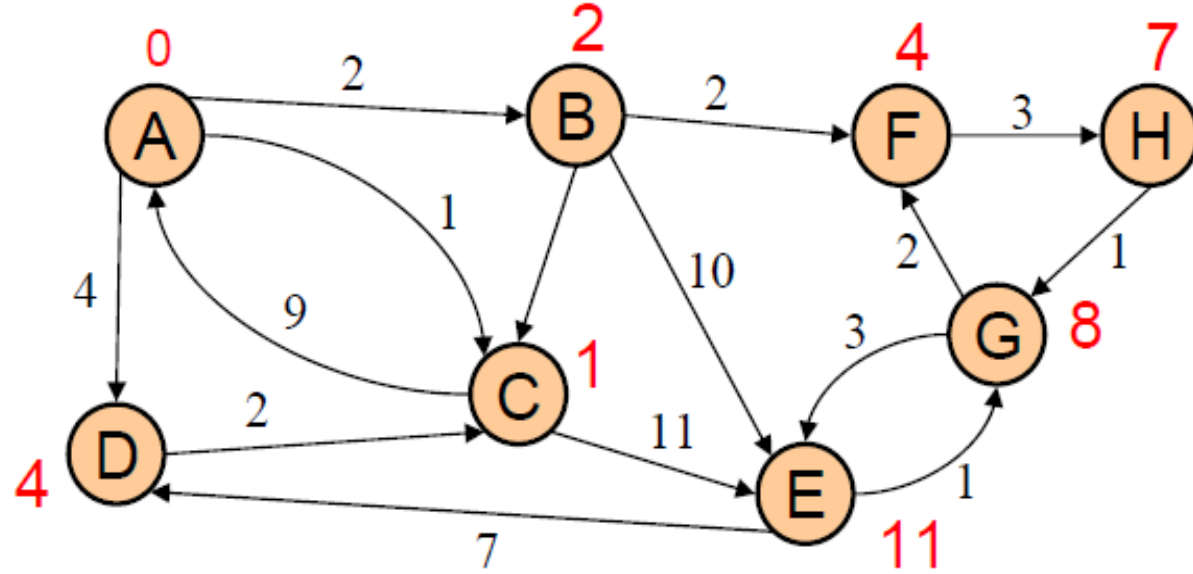
Vertex	Visited?	Cost	Found by
A	Y	0	
B	Y	2	A
C	Y	1	A
D	Y	4	A
E		≤ 12	C
F	Y	4	B
G		??	
H		≤ 7	F



Vertex	Visited?	Cost	Found by
A	Y	0	
B	Y	2	A
C	Y	1	A
D	Y	4	A
E		≤ 12	C
F	Y	4	B
G		≤ 8	H
H	Y	7	F



Vertex	Visited?	Cost	Found by
A	Y	0	
B	Y	2	A
C	Y	1	A
D	Y	4	A
E		≤ 11	G
F	Y	4	B
G	Y	8	H
H	Y	7	F



Vertex	Visited?	Cost	Found by
A	Y	0	
B	Y	2	A
C	Y	1	A
D	Y	4	A
E	Y	11	G
F	Y	4	B
G	Y	8	H
H	Y	7	F